

Содержание

1	Воспоминания о ФП	7
1.1	Термы и редукция	7
1.2	Типы	7
1.3	Функции в Haskell	10
1.4	Данные в Haskell	11
1.5	Классы типов в Haskell	13
1.6	Монады в Haskell	14
2	Параметрический полиморфизм	17
2.1	Параметрический полиморфизм в языке	17
2.1.1	First-class polymorphism	21
2.1.2	Higher-order/kinded polymorphism	23
2.1.3	Обобщённые алгебраические типы данных (GADTs)	24
2.1.4	Структуры на уровне типов, data promotion	25
2.1.5	Факультативно: эмуляция типовых абстракций и аппликаций (Proxy)	29
2.2	Реализация параметрического полиморфизма	29
2.2.1	Мономорфизация	30
2.2.2	Стирание типа	31
2.2.3	Гибридный подход	32
2.2.4	Использование виртуальной таблицы свойств типов	32
2.2.5	Сравнение подходов	33
2.3	Полиморфизм по конвенции вызова	34
2.3.1	Разновидности runtime представлений в Haskell	34
2.3.2	Классификация значений по runtime представлению	35
2.3.3	Representation polymorphism	36
3	Специальный (ad-hoc) полиморфизм	38
3.1	Классы типов в языке	39
3.1.1	Словари	39
3.1.2	Неявные аргументы	41
3.1.3	Вывод инстансов	42
3.1.4	Построение типа по значению	44

¹Автор Андрей Стоян (andrey.stoyan.csam@gmail.com).

²Спасибо Илье Колегову за первое внимательное прочтение и кучу комментариев. Спасибо моим студентам, которые своим интересом к предмету сделали эту работу возможной.

³Версия от 5 октября 2026 г., 01:28.

3.1.5	Имплициты и когерентность	45
3.1.6	Правила (rules) и специализация	46
3.1.7	Отступление: дефункционализация	47
3.1.8	Эмуляция полиморфизма высших порядков	48
3.2	Семейства	49
3.2.1	Data families	50
3.2.2	Synonym families	50
3.2.3	Инъективные семейства	51
3.2.4	Семейства первого класса	52
3.3	Кайнд Constraint	54
3.4	Использование ad-hoc полиморфизма	55
3.4.1	Сериализация	56
3.4.2	Экзистенциальные типы	56
3.4.3	Разрешение имён	57
3.4.4	Несинтаксические типовые эквивалентности, System FC	58
3.4.5	Коекции и роли	59
3.4.6	Type reflection	62
3.4.7	Data reflection	63
3.4.8	Открытые структуры	64
3.4.9	Исключения и открытая иерархия	64
3.4.10	Легковесные частичные стек-трейсы	66
3.4.11	Кастомизируемые ошибки типизации	67
3.4.12	Вычисление до нормальной формы	67
4	Типы данных	70
4.1	Вариантность	70
4.2	Изоморфизм	72
4.2.1	Кардинальность: суммы, произведения, экспоненты	73
4.2.2	Алгебраическое представление типа	74
4.3	Рекурсивные типы	76
4.3.1	Просто список	76
4.3.2	Неподвижная точка функтора	78
4.3.3	Схемы рекурсии	79
4.4	Всё через свёртки	81
4.4.1	Deforestation & list fusion	82
4.4.2	Visitor pattern	84
4.5	Всё через развёртку	85
4.5.1	Абстрактные типы данных	86
4.5.2	Stream fusion	87
4.6	Вездесущий дуализм	88
4.6.1	Push vs pull streaming	89

4.6.2	Data vs codata	89
4.7	Приложение: категория алгебр	91
5	Интерпретаторы	94
5.1	Интерпретаторы как основа основ	94
5.1.1	Башня интерпретаторов	94
5.1.2	Интерпретаторы повсюду	95
5.1.3	Интерпретаторы и семантика языков программирования	96
5.1.4	Встроенные доменно-специфичные языки (eDSL)	98
5.1.5	Пример: библиотека Accelerate	100
5.2	Типы значений	100
5.2.1	Untyped tagless interpreters	100
5.2.2	Typed tagged interpreters	101
5.2.3	Typed tagless interpreters	101
5.3	Связывания и функции первого класса	102
5.3.1	Семантика имён	102
5.3.2	Подстановки	103
5.3.3	Окружение	104
5.3.4	Замыкания	105
5.3.5	Типизированный контекст	106
5.3.6	Meta-circular интерпретация	106
5.3.7	Синтаксис высшего порядка	107
5.3.8	Сериализация функций	108
5.4	Tagless final интерпретаторы	109
5.4.1	Разные интерпретации для shallow embedding	109
5.4.2	Дойти до конца	110
5.4.3	Восстановление композиционности семантики	111
5.4.4	Typed tagless final interpreter	112
5.4.5	Встречаем старых друзей: Applicative, Monad	113
5.5	Expression problem	115
5.5.1	Копроизведение функторов	116
5.5.2	Произведение алгебр	117
6	Продолжения (continuations)	119
6.1	Концепция продолжений	119
6.1.1	Reduction semantics	120
6.1.2	Continuation semantics	122
6.1.3	Продолжения первого класса	123
6.2	Продолжения своими руками	126
6.2.1	Дефункционализация и аккумуляторы	127
6.2.2	Monad Cont	128
6.2.3	call/cc	130

6.3	Разграниченные продолжения (delimited continuations)	130
6.3.1	Реализация операторов	132
6.3.2	В Monad Cont	133
6.4	Приложения продолжений	134
6.4.1	Всё через продолжения	134
6.4.2	The mother of all monads	136
6.4.3	Генераторы и корутины	138
6.5	Эффективная работа с продолжениями	139
6.5.1	Contiguous stack	139
6.5.2	Сегментный стек	139
6.5.3	Finite state machine (FSM)	139
7	Эффекты и модульные интерпретаторы	141
7.1	Понятие эффекта	141
7.2	Хендлеры эффектов	144
7.2.1	Хендлеры через ограниченные продолжения	144
7.2.2	Эффективная реализация хендлеров	144
7.2.3	Встроенные хендлеры как явная клиент-серверная коммуникация . . .	145
7.2.4	Расширяемые сообщения и пересылка	146
7.2.5	Свободные монады	147
7.2.6	Хендлеры через свободные монады	148
7.2.7	Приложения хендлеров	150
7.2.8	Трансформеры монад	150
7.2.9	Алгебраичность и эффекты высших порядков	151
8	Системы эффектов TODO	153
9	Datatype-generic programming TODO	154
9.1	Специализация	154
9.2	Ad-hoc решения	154
9.3	Подходы к datatype-generic в Haskell	154
9.3.1	Template Haskell	154
9.3.2	Deriving strategies	154
9.3.3	GHC.Generics	155
9.3.4	Sum Of Products	155
9.3.5	Uniplate	155
10	Персистентность и оптика TODO	156
10.1	Персистентные структуры данных	156
10.2	Идея оптики	157
10.3	Использование оптики	159
10.4	Data optics	159

10.4.1	Линзы	159
10.4.2	Призмы	161
10.4.3	Композиция линз и призм	161
10.5	Путь к profunctor optics	162
10.6	Генерация оптики	162

Введение

Многие сложные концепции в дизайне языков и программ могут быть поняты как частные случаи некоторых простых фундаментальных принципов, которые, как правило, считаются общеизвестным фольклором, не требующим дополнительных пояснений. Однако, сложность в том, что эти знания рассеяны по книгам, статьям и “культовым” блог-постам, и требуется довольно много времени и сил для восстановления целостной картины.

Цель данного курса — собрать в одном месте такие фольклорные знания и организовать их в некоторую систему. Курс будет явным образом опираться на классические работы, исследующие принципы построения языков, и помогать в их изучении. Просмотр упоминаемых статей является важной частью самостоятельной работы в рамках курса.

Под функциональным программированием, вынесенным даже в заголовок курса, понимается трепетное отношение к понятию эффекта, которое в ФП, в отличие от других школ мысли, не считается аксиоматической данностью, но предметом для изучения, сознательного конструирования и аккуратного обращения. Этот подход оказывается очень полезным для изучения языков, построения выразительных языковых конструкций, а также является основой для продуктивного стиля программирования. Кроме того, функциональные языки сравнительно просты, в результате чего новые идеи и подходы нередко зарождаются в них и распространяются далее.

В качестве основного языка курса выбран Haskell, так как он, с одной стороны, воплощает в себе многие концепции, часто доведённые до некоторого логического завершения, и достаточно выразителен для кодирования других. С другой стороны, всё ещё является прикладным промышленным языком программирования.

В связи с широтой контекста, данный курс не всегда является глубоким. Так, детали реализации в GHC или теоретико-категорные основания вещей могут даваться в общем виде и без конкретики. В то же время, в плоскости языкового дизайна через оптику функционального программирования курс пытается быть максимально подробным.

Таким образом, данный курс может быть полезен тем, кто интересуется дизайном языков и красивыми обобщениями программистских концепций, хочет улучшить свои навыки проектирования API или планирует вести практическую деятельность на функциональных языках.

Пререквизитом к прохождению курса является знание основ функционального программирования: алгебраических типов данных, паттерн-матчинга, свёрток, параметрического полиморфизма, классов типов, базовых монад. Дополнительно будет полезным умение читать типовые дробы, знакомство с полиморфным λ -исчислением и кодированием Чёрча.

1 Воспоминания о ФП

В этом разделе мы вспомним основные концепции функционального программирования и языка Haskell.

1.1 Термы и редукция

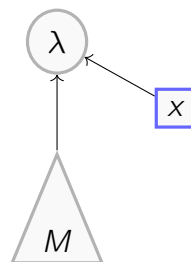
В ФП программы представляют собой выражения. Выполнение программ — редукция таких выражений до более “простых”. Выражения можно представлять как в виде линейной записи символов, так и в виде дерева, для понимания которого не требуется знания вспомогательных правил ассоциативности и проч.

Простейший функциональный язык — λ -исчисление. Выражения в нём называются λ -термами, которые состоят из вершин трёх видов (V — множество валидных идентификаторов, Λ — множество λ -термов):

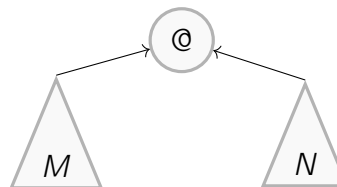
Переменные $x \in \Lambda$, если $x \in V$



Абстракция $(\lambda x. M) \in \Lambda$, если $x \in V, M \in \Lambda$



Аппликация $(M N) \in \Lambda$, если $M \in \Lambda, N \in \Lambda$



В произвольном выражении можно заменить некоторый его фрагмент на формальный параметр, который должен быть задекларирован выше по дереву с помощью специальной вершины λ . Вместо формального параметра можно в дальнейшем подставлять различные конкретные параметры с помощью вершины-аппликации $@$, то есть переиспользовать это выражение для различных целей (например, см. рис. 1). Редукция как раз определяется как следующее правило переписывания: ищется применение λ -функции к аргументу и в её тело осуществляется подстановка аргумента во все свободные вхождения переменной, связанной лямбдой (рис. 2).

1.2 Типы

Программное обеспечение — это сложно. Поэтому постоянно и неизбежно в программах возникают ошибки. Их можно искать, в том числе, статически, то есть без запуска программы. Одним из видов статического анализа является анализ типов.

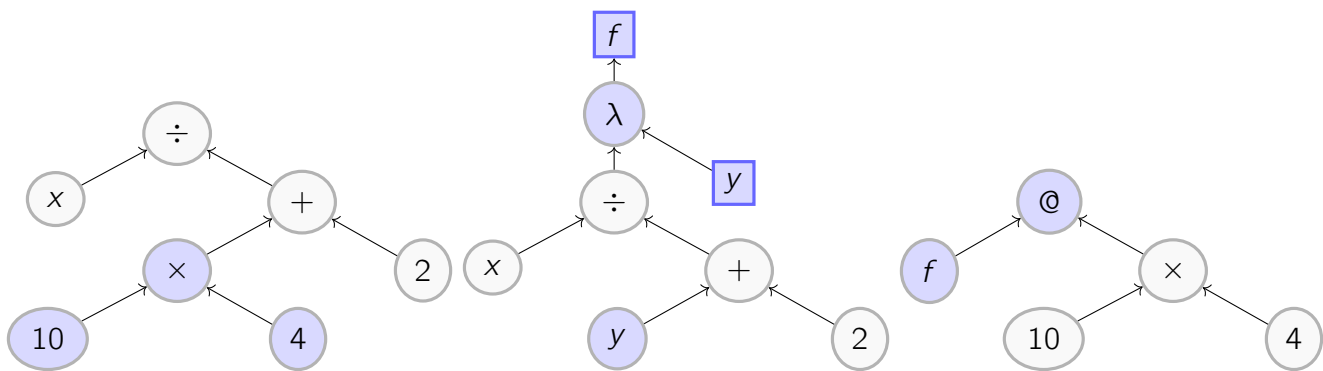


Рис. 1: Выражение с помощью λ вершины преобразуется в функцию одного аргумента.

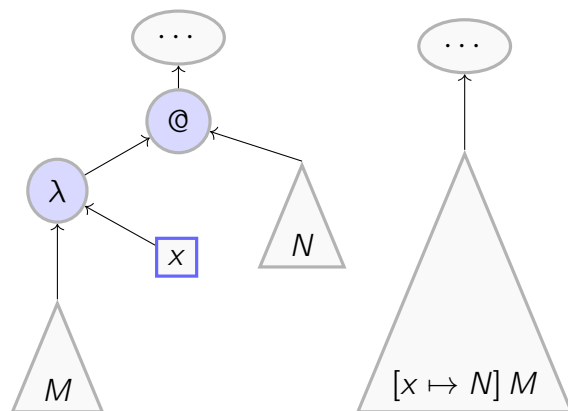


Рис. 2: Редукция переписывает дерево путём подстановки конкретного аргумента вместо формального параметра.

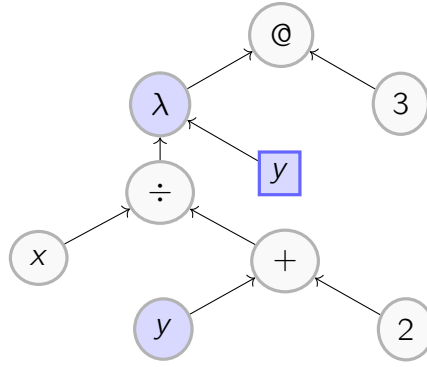


Рис. 3: Дерево соответствующее выражению $(\lambda y. x \div (y + 2)) 3$.

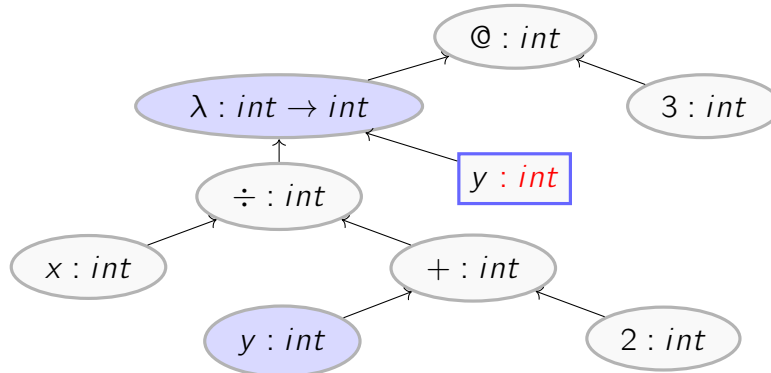


Рис. 4: Дерево выражения $(\lambda y. x \div (y + 2)) 3$ после присписывания типовых меток.

Идея анализа типов состоит в том, что каждой вершине дерева программы мы пытаемся присписать некоторую синтаксическую метку по определённым правилам. Если таким образом каждой вершине можно присписать метку, то мы считаем, что программа проходит проверку типов, и она “хорошая”. Например, на рисунке 3 представлено выражение, а на рисунке 4 каждой вершине присписаны метки в согласии с некоторой системой типов.

Система типов определяет синтаксис типовых меток и правила, по которым их можно присписывать. Синтаксис обычно описывается в классических нотациях а ля BNF, а правила в виде типовых дробей. Например, так выглядят дроби для просто-типизированного λ -исчисления:

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

Типовые метки имеют чисто-синтаксическую природу, однако их можно проинтерпретировать. Самая популярная интерпретация — воспринимать типовую метку как множество. Так, метке $int \rightarrow int$ можно поставить в соответствие множество функций между множествами целых чисел $\mathbb{Z} \rightarrow \mathbb{Z}$.

1.3 Функции в Haskell

В своей основе Haskell представляет собой расширенное типизированное λ -исчисление, дополненное примитивными типами, возможностью декларировать новые имена, структурами данных и классами типов.

Примеры λ -абстракций в REPL окружении GHCi:

```
1 ghci> (\x -> x + 1) 4
2 5
```

Можно узнать тип функции в интерпретаторе (в реальности числа полиморфные, но об этом далее):

```
1 ghci> :t \x -> x + 1
2 \x -> x + 1 :: Int -> Int
```

Функциям можно давать имена. Именам можно приписывать типы, это рекомендуется делать явно для деклараций на верхнем уровне файлов исходного кода:

```
1 f :: Int -> Int
2 f x = x + 1
```

Если имя типа начинается с маленькой буквы, то это не конкретный тип, а типовая переменная, способная принимать различные значения в зависимости от места вызова. Такая возможность называется **параметрическим полиморфизмом**. Так, функция, которая просто возвращает свой аргумент, никак не ограничивает тип аргумента. Но в то же время тип результата должен совпадать с типом аргумента:

```
1 id :: a -> a
2 id x = x

3 ghci> :t id 'x'
4 id 'x' :: Char
```

Функции могут принимать другие функции в качестве аргументов (такие функции называются **функциями высших порядков (higher-order functions)**). Имя функции может состоять из специальных символов, тогда она считается оператором и может применяться к своим операндам в инфиксном стиле:

```
1 ($) :: (a -> b) -> a -> b
2 f $ x = f x
```

Пример рекурсивной функции, использующей охранные выражения для отличия базового случая рекурсии:

```

1 factorial :: Int -> Int
2 factorial n
3   | n < 1 = 1
4   | otherwise = n * factorial (n - 1)

```

Упражнение 1 Что выведет запрос `ghci> :t uncurry (flip const)?`

Упражнение 2 Что выведет запрос `ghci> :t first . first` при

```

1 first :: (a -> a') -> (a, b) -> (a', b)

```

Упражнение 3 Реализуйте факториал с помощью техники аккумулирующего параметра.

1.4 Данные в Haskell

В Haskell есть встроенная возможность объявлять новые типы данных на основании других типов, а также создавать их экземпляры.

Зададим тип данных, описывающий животных:

```

1 data Animal
2   = Cat String Int
3   | Dog String

```

Мы задали тип данных `Animal` и два способа создать значения этого типа: для кошек и собак. `Cat` и `Dog` — это **конструкторы данных**. Они представляют собой функции, реализация которых находится на стороне языка. Они выделяют память под экземпляры данного типа и позиционно размещают компоненты. Кошек мы описываем именем и оставшимся количеством жизней, а собак — только именем.

```

1 Cat :: String -> Int -> Animal
2 Dog :: String -> Animal

```

Чтобы воспользоваться информацией, сохранённой в структуре данных, требуется деконструировать её с помощью паттерн-матчинга. Мы сопоставляем значение типа с образцом. Если образец похож на то, как было сконструировано значение, то он выбирается среди других образцов и переменные, задекларированные в нём, начинают ссылаться на соответствующее позиционно содержимое структуры данных:

```

1 describeAnimal :: Animal -> String
2 describeAnimal animal = case animal of
3   Cat name nLives -> "This is cat " ++ name ++ " " ++ show nLives
4   Dog name -> "This is dog " ++ name

```

В Haskell есть специальный синтаксис для объявления полей с именованными метками.

```

1 data Penguin = Penguin { getName :: String, getAge :: Int }
2 penguin = Penguin { getName = "Andrey", getAge = 500 }

```

Haskell генерирует функции-аксессоры для доступа к полям значения:

```

1 ghci> :t getName
2 getName :: Penguin -> String

```

Часто функции в программировании частичные — при некоторых значениях аргументов они могут вернуть результат, а при некоторых — нет. Давайте моделировать это с помощью специального типа данных. Если есть вещественный результат, будем возвращать его. Если нет, будем возвращать специально выделенное константное значение этого типа.

```

1 data MaybeD = NothingD | JustD Double
2 safeSqrtD :: Double -> MaybeD
3 safeSqrtD x = if x < 0 then NothingD else JustD (calcSqrt x)

```

Можно заметить, что так нам придётся объявлять отдельный тип вроде `MaybeD` для каждого типа `D`. Поэтому Haskell позволяет абстрагироваться в типе, аналогично тому как можно абстрагироваться по значениям в терме.

```

1 data Maybe a = Nothing | Just a
2 safeSqrt :: Double -> Maybe Double
3 safeSqrt x = if x < 0 then Nothing else Just (calcSqrt x)

```

Заметьте, что сейчас `Maybe` — это не совсем тип, так как теперь нужно передать типовой параметр, чтобы получить конкретный тип. `Maybe` называют **типовым конструктором**.

Вместе с абстракцией на уровне типов появилась и аппликация типа к типу. А что если дать меньше параметров типовому конструктору, чем ожидается? А что если больше? Контроль за корректностью типовых аппликаций обеспечивает **система кайндов**⁴. Это простейшие “типы для типов”, то есть синтаксические метки, контролирующие корректность записанных программистом типов. Так, обычные типы имеют метку (кайнд) `Type`⁵. Типовые конструкторы имеют стрелочные кайнды. Например, `Maybe :: Type -> Type`. Аппликация типового конструктора к типу подходящего кайнда убирает одну стрелку:

```

1 ghci> :k Int
2 Int :: Type
3 ghci> :k Maybe
4 Maybe :: Type -> Type
5 ghci> :k Maybe Int
6 Maybe Int :: Type

```

⁴Иногда в русскоязычной литературе кайнды называют родами типов, но мы не будем так говорить.

⁵Кайнд `Type` экспортируется из модуля `Data.Kind`. В старых текстах и в выводе GHCi по умолчанию он обозначается звёздочкой: `Maybe :: * -> *`. Это одно и то же; чтобы GHCi печатал `Type`, включите `:set -XNoStarIsType`.

Кроме совершенно новых типов данных, в Haskell можно объявлять типовые синонимы. Это имена, которые можно использовать вместо других типов, если, например, запись оригинального типа слишком длинная для повсеместного написания:

```
1 type T a = VeryLongType Int (a -> AnotherLongType a)
```

Если тип данных содержит только один конструктор и только одно поле, то отсутствует необходимость в аллокации новой памяти, содержащей тег конструктора и набор ссылок на поля. В таком случае, в качестве значения такого типа можно всегда просто использовать значение оборачиваемого типа, оставляя новый тип присутствовать исключительно во время компиляции, снижая нагрузку во время исполнения. Для объявления таких типов-обёрток нужно воспользоваться ключевым словом `newtype` вместо `data`:

```
1 newtype CourseId = CourseId Int64
2 newtype ModuleId = ModuleId Int64
```

Упражнение 4 Определите кайнд конструктора типа

```
1 data Free f a = Pure a | Free (f (Free f a))
```

1.5 Классы типов в Haskell

Параметрический полиморфизм позволяет использовать один и тот же код для различных типов входных данных. Классы типов же позволяют одному идентификатору ссылаться на разные реализации для разных типов данных (что аналогично механизму перегрузки (overloading) в других языках). Классы типов, как говорят, являются механизмом **специального (ad-hoc) полиморфизма**. Так, мы можем задекларировать символ `==`, выбор реализации которого зависит от выбора типа аргументов `a`:

```
1 class Eq a where
2   (==) :: a -> a -> Bool
```

Для каждого типа можно объявить свою собственную реализацию `Eq`:

```
1 instance Eq CourseId where
2   CourseId x == CourseId y = x == y

3 instance Eq a => Eq [a] where
4   [] == [] = True
5   x:xs == y:ys = x == y && xs == ys
6   _ == _ = False
```

Теперь в зависимости от конкретного типа `a` в месте вызова, будет выбрана подходящая реализация для этого типа:

```

1 ghci> CourseId 1 == CourseId 2
2 False
3 ghci> [CourseId 1, CourseId 2] == [CourseId 1, CourseId 2]
4 True

```

Рассмотренные ранее параметрически-полиморфные функции ничего не могли делать со своими аргументами, кроме как возвращать их в качестве результата или передавать в другие полиморфные функции. Чтобы уметь делать что-то ещё, нужна какая-то дополнительная информация про тип, потому что иначе нет никакой гарантии, что над объектом данного типа можно делать все необходимые операции. Так, функция `suc n = n + 1` не будет работать для строчек, потому что для них, очевидно, не определена операция сложения. Поэтому некорректно будет приписать полиморфный тип `suc :: a -> a`.

Классы типов, в отличие от перегрузки, в том числе являются механизмом ограничения полиморфности функций. Мы можем явно задать, что функция требует не произвольный тип на вход, а произвольный тип, для которого определены обязательно нужные нам операции. Так, для типа `suc` достаточно ограничить тип условием наличия плюса для него (операция обозначаемая символом `+` объявлена в классе типов `Num`):

```

1 suc :: Num a => a -> a

```

Упражнение 5 Реализуйте функцию, проверяющую равенство всех элементов данного списка.

Упражнение 6 Реализуйте инстанс полугруппы для функций.

Упражнение 7 Реализуйте инстанс `Eq` для функций. Подсказка: для произвольных функций это невозможно; подумайте, какие ограничения нужно наложить на тип аргумента, чтобы равенство стало проверяемым.

1.6 Монады в Haskell

Класс типов `Functor` объявляется для конструкторов типов и позволяет заменить в некотором контейнере все элементы одного типа на все элементы другого, оставляя структуру контейнера неизменной.

```

1 class Functor (f :: Type -> Type) where
2   fmap :: (a -> b) -> f a -> f b

3 instance Functor [] where
4   fmap :: (a -> b) -> [a] -> [b]
5   fmap _ [] = []
6   fmap f (x:xs) = f x : fmap f xs

```

В Haskell любая функция просто вычисляет результат некоторого типа. Однако в программировании часто требуются функции, которые не только вычисляют результат, но и делают что-то ещё. Например, изменяют какое-то состояние или пишут в консоль. Иными словами, производят побочные эффекты. В любом случае в Haskell мы можем вернуть из функции только результат, поэтому такие побочные эффекты мы кодируем в качестве дополнительной структуры, оборачивающей чистый результат. Т.е. если функция без побочных эффектов возвращала какой-то тип *a*, то после добавления побочных эффектов в её реализацию, она будет возвращать некоторый тип *вычислений* *f a*.

- Если функция кидает ошибку, то *f = Maybe*.
- Если функция читает глобальное состояние типа *e*, то *f = e -> _*.
- Если функция читает глобальное состояние *s* и обновляет его, то *f = s -> (s, _)*.

Стандартная библиотека Haskell предоставляет несколько классов типов для работы со значениями вида *f a*. Они позволяют абстрагироваться от структуры *f* и работать со значениями *a* внутри, как будто нет никакой дополнительной структуры.

Первый такой класс типов позволяет писать выражения над вычислениями *f a*.

```
1 class Functor f => Applicative (f :: Type -> Type) where
2   pure :: a -> f a
3   liftA2 :: (a -> b -> c) -> f a -> f b -> f c

4 instance Applicative Maybe where
5   pure :: a -> Maybe a
6   pure = Just

7   liftA2 :: (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c
8   liftA2 _ Nothing _ = Nothing
9   liftA2 _ _ Nothing = Nothing
10  liftA2 f (Just x) (Just y) = Just (f x y)
```

Второй класс типов позволяет делать последовательную композицию вычислений в императивном стиле⁶⁷:

```
1 class Applicative m => Monad (m :: Type -> Type) where
2   (>=) :: m a -> (a -> m b) -> m b

3 newtype State s a = State { runState :: s -> (s, a) }
```

⁶В коде ниже лямбда передаётся последним аргументом без скобок: `State \s -> ...` вместо `State (\s -> ...)`. Так позволяет писать расширение `BlockArguments`, мы будем пользоваться им повсеместно.

⁷В библиотеке `mtl` компоненты пары идут в обратном порядке: `s -> (a, s)`. Мы в конспекте ставим состояние первым.

```

4 instance Monad (State s) where
5   (>>=) :: State s a -> (a -> State s b) -> State s b
6   m >>= k = State \s ->
7     let (s', x) = runState m s in
8     runState (k x) s'

```

Теперь если мы определим базовые операции работы с состоянием, мы сможем писать код в императивном стиле с побочными эффектами.

```

1 get :: State s s
2 get = State \s -> (s, s)

3 put :: s -> State s ()
4 put newS = State \oldS -> (newS, ())

5 example :: State Int Int
6 example =
7   get >>= \x ->
8   put 42 >>= \() ->
9   get >>= \y ->
10  pure (x + y)

11 ghci> runState example 1
12 (42, 43)

```

Для таких монадических цепочек существует специальный синтаксический сахар:

```

1 example :: State Int Int
2 example = do
3   x <- get
4   put 42
5   y <- get
6   pure (x + y)

```

Упражнение 8 Реализуйте `liftA3` через `liftA2`.

Упражнение 9 Реализуйте `>>=` через `join` и наоборот.

Упражнение 10 Прочитайте два числа с консоли, поделите одно на другое нацело и распечатайте результат; если остаток не нулевой, распечатайте его тоже.

2 Параметрический полиморфизм

Никакое нетривиальное семантическое свойство программ не может быть алгоритмически проверено⁸. Чтобы оставаться разрешимыми (в смысле проверки типов и/или вывода), многие системы типов жертвуют полнотой и, помимо некорректных программ, отвергают много корректных. В то же время системы типов также стараются предоставлять различные возможности, позволяющие протипизировать как можно больше корректных программ. Одна из них — параметрический полиморфизм.

Под **параметрическим полиморфизмом** мы будем подразумевать возможность кода единообразно работать с произвольными типами данных Strachey [2000], Cardelli and Wegner [1985], что позволяет во многих случаях избегать дублирования кода.

В этой главе мы рассмотрим, как описывают полиморфизм в самом простом виде — в типизированном λ -исчислении. Изучим различные формы параметрического полиморфизма и сопутствующие техники безопасного программирования. Проанализируем возможные способы эффективной реализации параметрического полиморфизма. И в завершение рассмотрим полиморфизм по рантайм-представлению, “полиморфизм по полиморфизму”.

2.1 Параметрический полиморфизм в языке

Начнём со знакомого примера из предыдущей главы:

```
1 id :: a -> a
2 id x = x
```

Без полиморфизма нам пришлось бы написать отдельные `idInt`, `idChar` и так далее: корректные, но совершенно одинаковые программы. Типовая переменная `a` избавляет от дублирования, но оставляет вопрос: *кто и когда выбирает конкретный тип вместо `a`?*

Тип выбирает вызывающий код, причём отдельно в каждом месте вызова. Иначе говоря, у `id` есть ещё один, скрытый аргумент — сам тип. В Haskell этот аргумент можно сделать явным с обеих сторон: и принять его в определении функции, и передать в месте вызова⁹:

```
1 id :: forall a . a -> a
2 id @a (x :: a) = x

3 ghci> :t id @Int
4 id @Int :: Int -> Int
5 ghci> id @Char 'x'
6 'x'
```

Чтобы понять, откуда берётся скрытый аргумент, посмотрим, как полиморфизм описывают в самом простом виде — в типизированном λ -исчислении. Вспомним правила просто

⁸https://en.wikipedia.org/wiki/Rice%27s_theorem

⁹Расширения `TypeAbstractions` и `TypeApplications`; явный квантор в сигнатуре — `ExplicitForAll`.

типизированного λ -исчисления из предыдущей главы. λ -абстракция позволяет обобщать выражения по значениям, каждая абстракция добавляет стрелку в тип выражения, аппликация же снимает стрелку:

$$\frac{x : \tau, \Gamma \vdash M : \sigma}{\Gamma \vdash \lambda x : \tau. M : \tau \rightarrow \sigma} \text{Lam} \quad \frac{\Gamma \vdash M : \tau \rightarrow \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash M N : \sigma} \text{App}$$

Полиморфное λ -исчисление (*System F*) добавляет точно такую же пару конструкций, только для типов [Pierce, 2002, глава 23]. λ -абстракция $\Lambda\alpha. M$ обобщает выражение по типу: это функция, аргументом которой является тип. Она добавляет в тип выражения квантор $\forall$ подобно тому, как λ добавляет стрелку¹⁰. Применение терма к типу $M \sigma$ (**универсальная аппликация (universal application)**) снимает квантор: вызывающий код выбирает, с каким конкретным типом σ он будет использовать терм, и σ подставляется в тип вместо α :

$$\frac{\alpha, \Gamma \vdash M : \tau}{\Gamma \vdash \Lambda\alpha. M : \forall\alpha. \tau} \text{TLam} \quad \frac{\Gamma \vdash M : \forall\alpha. \tau}{\Gamma \vdash M \sigma : [\alpha \mapsto \sigma] \tau} \text{TApp}$$

Таким образом, например, функция *id* фактически принимает *два аргумента: тип и значение*:

$$\begin{aligned} id &: \forall\alpha. \alpha \rightarrow \alpha \\ id &= \Lambda\alpha. \lambda x : \alpha. x \\ id \text{ nat} &: \text{nat} \rightarrow \text{nat} \\ id \text{ nat } 42 &: \text{nat} \end{aligned}$$

Сравните с записью на Haskell выше: определение `id @a (x :: a) = x` — это в точности $\Lambda\alpha. \lambda x : \alpha. x$, а вызов `id @Int` — это *id nat*.

В обычном коде на Haskell типовые абстракции и аппликации, как и кванторы в начале типа, расставляются неявно механизмом вывода типов. Достаточно конвенции именования: конкретные типы начинаются с большой буквы, а типовые переменные — с маленькой. Писать их вручную приходится, когда информации из терма недостаточно, чтобы вывести тип, когда нужно задать порядок типовых параметров, или чтобы сослаться на абстрагированный тип в теле функции¹¹.

Упражнение 11 Какой тип у выражения `const @Int`? А у `const @Int @Char`? От чего зависит порядок, в котором передаются типы?

Цена такого полиморфизма на уровне типов невелика: пока кванторы стоят только снаружи типа, алгоритм Хиндли–Милнера выводит все типовые абстракции и аппликации сам. Платить приходится, когда кванторы появляются в других местах (2.1.1), а также во время исполнения (2.2).

¹⁰В теории зависимых типов и стрелка, и квантор $\forall$ являются частными случаями одной конструкции — Π -типа.

¹¹Ссылаться из тела функции на типовые переменные из её сигнатуры позволяет расширение `ScopedTypeVariables`.

Полиморфными бывают не только функции, но и типы данных. В предыдущей главе мы видели, что `Maybe` — это не тип, а типовой конструктор: чтобы получить тип, ему нужно передать аргумент, например, `Maybe Int`. Если ранее мы управляли типом с уровня термов универсальной аппликацией, то теперь мы управляем типом на уровне типов. Иначе говоря, на уровне типов появляется своё маленькое λ -исчисление: λ -абстракция в типах, аппликация в типах и, соответственно, β -редукция.

Для примера определим тип пары по Чёрчу, абстрагированный от конкретных типов компонент, чтобы пользователь мог выбрать нужные ему. Сначала на Haskell, где вычислительную семантику типов можно проследить в синонимах типов:

```
1 type Pair a b = forall c . (a -> b -> c) -> c
2 pair :: a -> b -> Pair a b
3 pair x y = \f -> f x y
4 fst :: Pair a b -> a
5 fst p = p (\x _ -> x)
```

Пара здесь закодирована по Чёрчу: это функция, которая отдаёт обе компоненты любому, кто скажет, что с ними делать. Синоним `Pair` — функция на типах: `Pair Int Int` вычисляется в `forall c . (Int -> Int -> c) -> c`. Обычные конструкторы типов, напротив, номинативны: `(Int, Int)` или `Maybe Int` никуда далее не вычисляются.

А раз на уровне типов есть функции и аппликации, то и типы можно составить неправильно: `Maybe Maybe` или `Int Bool`. Значит, типам нужна своя система типов. Это уже знакомая нам система кайндов: она (пока) представляет собой простейшую “систему типов для типов” и обеспечивает *well-formedness* типов и строгую нормализуемость¹². Получается двухэтажная конструкция (рис. 5): вершинам дерева терма приписаны типы, но типы — тоже деревья, и уже их вершинам приписаны кайнды. Кайнды, в свою очередь, тоже деревья (например, `Type -> Type`); что приписывается их вершинам, мы увидим в конце этого раздела (рис. 7).

Правила приписывания кайндов устроены в точности как правила *Lam* и *App* для термов, только этажом выше: абстракция по типовой переменной добавляет стрелку в кайнд, аппликация типа к типу её снимает [Pierce, 2002, глава 29]¹³:

$$\frac{\alpha : \kappa_1, \Gamma \vdash \tau : \kappa_2}{\Gamma \vdash \lambda \alpha : \kappa_1. \tau : \kappa_1 \rightarrow \kappa_2} KLam \quad \frac{\Gamma \vdash \tau : \kappa_1 \rightarrow \kappa_2 \quad \Gamma \vdash \sigma : \kappa_1}{\Gamma \vdash \tau \sigma : \kappa_2} KApp$$

Аппликация абстракции к аргументу редуцируется подстановкой, как и на уровне термов:

$$(\lambda \alpha : \kappa. \tau) \sigma \rightsquigarrow [\alpha \mapsto \sigma] \tau$$

Кроме того, нужны правила для конструкций, которые строят обычные типы: стрелка и квантор из типов кайнда * делают тип кайнда *. Квантор при этом связывает типовую переменную

¹²Строгая нормализуемость — любой порядок редукций приводит к нормальной форме.

¹³В формулах кайнд `Type` обозначаем традиционной звёздочкой, как это принято в литературе Pierce [2002].

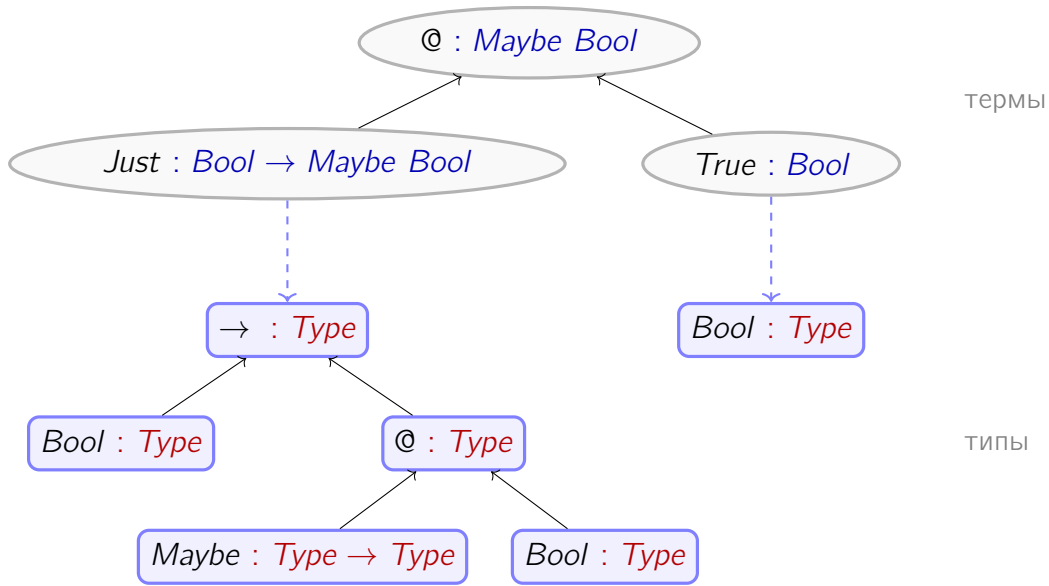


Рис. 5: Два этапа типизации выражения `Just True`. Вершинам дерева терма приписаны типы (синим). Каждый тип сам является деревом (пунктирные стрелки), и его вершинам приписаны кайнды (красным).

любого кайнда:

$$\frac{\Gamma \vdash \tau : * \quad \Gamma \vdash \sigma : *}{\Gamma \vdash \tau \rightarrow \sigma : *} \text{KArrow} \quad \frac{\alpha : \kappa, \Gamma \vdash \tau : *}{\Gamma \vdash \forall \alpha : \kappa. \tau : *} \text{KAll}$$

В Haskell кайнды, как и типы, выводятся автоматически, а посмотреть их можно командой `:k`, как мы делали в предыдущей главе.

В исчислении то же самое записывается с явными абстракциями и аппликациями на обоих уровнях (кайнды связанных типовых переменных указаны верхним индексом):

$$\begin{aligned} \text{Pair} &: * \rightarrow * \rightarrow * \\ \text{Pair} &= \lambda \tau^* \sigma^*. \forall \gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma \\ \text{pair} &: \forall \alpha \beta. \alpha \rightarrow \beta \rightarrow \text{Pair } \alpha \beta \\ \text{pair} &= \Lambda \alpha^* \beta^*. \lambda x^\alpha y^\beta. (\Lambda \gamma^*. \lambda f^{\alpha \rightarrow \beta \rightarrow \gamma}. f \ x \ y) \\ \text{fst} &: \forall \alpha \beta. \text{Pair } \alpha \beta \rightarrow \alpha \\ \text{fst} &= \Lambda \alpha^* \beta^*. \lambda p^{\text{Pair } \alpha \beta}. p \ \alpha \ (\mathbf{K} \ \alpha \ \beta) \end{aligned}$$

Прочитаем определение `fst`. Принимаем два типа и пару p . Пара — полиморфная функция, поэтому сначала выбираем для неё тип результата α , а затем передаём ей функцию $\mathbf{K}$, возвращающую первый из двух аргументов.

Упражнение 12 Постройте по правилам кайндизации дерево вывода $\text{Pair} : * \rightarrow * \rightarrow *$. Какие правила и в каком порядке в нём применяются? Поредуцируйте тип `Pair nat bool` до нормальной формы.

Упражнение 13 Реализуйте на Haskell функции `snd` и `swap :: Pair a b -> Pair b a`. Запишите `snd` в исчислении с явными типовыми абстракциями и аппликациями.

Haskell не позволяет создавать функции на типах по месту с помощью явной типовой лямбды¹⁴ ввиду проблематичности этой конструкции для вывода типов. Однако именованные функции на типах есть, и мы рассмотрим их далее 3.2. В Scala существует нетривиальный трюк¹⁵¹⁶, который позволяет этого добиться. Scala3, однако, включила эту возможность непосредственно в язык¹⁷.

2.1.1 First-class polymorphism

Рассмотрим функцию `f g = (g 42, g 'a')`. Программа корректна, если передать в неё, например, `id`. Однако алгоритм Хиндли–Милнера её отвергает: аргумент функции он считает мономорфным, а `g` применяется и к числу, и к символу. Чтобы протипизировать такую программу, нужно разрешить квантору стоять не только снаружи типа, но и слева от стрелки.

Иначе говоря, нужна возможность писать функции, которые принимают другие полиморфные функции в качестве аргументов. Типы таких функций называются **типами высшего ранга (higher-rank types)**¹⁸. Так, типовой параметр функции `g` определяет функция `f`, а не вызывающий функцию `f`:

```
1 f :: (forall a . a -> a) -> (Int, Char)
2 f g = (g @Int 42, g @Char 'a') -- универсальная аппликация для наглядности
3 ghci> f (\x -> x)
```

Цена типов высшего ранга в том, то есть глобальный вывод типов Haskell в этом случае перестаёт работать. Но если типы высшего ранга приписать вручную, остальной вывод будет работать как раньше. Например, тип числа Чёрча полиморфен, поэтому функции над числами Чёрча имеют высший ранг¹⁹²⁰:

```
1 suc :: (forall a . (a -> a) -> a -> a) -> (b -> b) -> b -> b
2 suc n s z = s (n s z)
```

Упражнение 14 Какую ошибку выдаст GHC, если убрать у функции `f` сигнатуру и типовые аппликации? Какой тип для `g` он пытался вывести?

Упражнение 15 Какой ранг имеет тип `Int -> (forall a . a -> a)`?

¹⁴<https://stackoverflow.com/questions/4069840/lambda-for-type-expressions-in-haskell>

¹⁵(stackoverflow) Scala type lambdas.

¹⁶<https://stackoverflow.com/questions/9443004/what-does-the-operator-mean-in-scala>

¹⁷<https://docs.scala-lang.org/scala3/reference/new-types/type-lambdas.html>

¹⁸В Haskell их включает расширение RankNTypes.

¹⁹<https://okmij.org/ftp/tagless-final/course/Boehm-Berarducci.html>

²⁰Любому интересующемуся языками программирования предлагается провести на сайте Олега Киселёва не один месяц жизни: <https://okmij.org/ftp/README.html>.

От многих проблем сопутствующих типам высших рангов можно избавиться, если создавать для них обёртки. Например, для чисел Чёрча можно создать обёртку `newtype Church`. Теперь код, работающий с обёрткой, может быть протипизирован типами первого ранга, только конструктор имеет тип высшего ранга:

```
1 newtype Church = Church (forall a . (a -> a) -> a -> a)
2 (+) :: Church -> Church -> Church -- rank 1
```

Аналогичный код можно написать и на Java или Kotlin:

```
1 interface Church { fun <a> fold(s: (a) -> a, z: a): a }
2 fun plus(n: Church, m: Church): Church = object : Church {
3     override fun <a> fold(s: (a) -> a, z: a): a = n.fold(s, m.fold(s, z))
4 }
```

По умолчанию типовые параметры можно специализировать только на конкретные типы. Специализацию типовых параметров на полиморфные типы (включающие `forall`'ы внутри себя) называют **импредикативным применением**²¹.

```
1 runST :: (forall s. ST s c) -> c
2 ($) :: forall a b . (a -> b) -> a -> b
3 foo = runST $ ... -- a специализируется на (forall s. ST s c)
```

Этот пример настолько частый, что GHC делает для оператора `$` исключение и типизирует его без расширения. Для любой другой функции с таким же типом, например, собственного оператора аппликации `app`, потребуется само расширение; с ним GHC применяет алгоритм Quick Look, о котором ниже.

Higher-rank типы можно использовать как type-based escape analysis, иначе говоря, не позволять пользователю передавать некоторое значение вовне определённого скоупа. Так, например, Haskell предоставляет эффективную монаду `ST`, позволяющую в рамках ограниченного скоупа работать с мутабельными ячейками памяти Launchbury and Peyton Jones [1995][Maguire, a, 7.2, ST trick]:

```
1 newtype ST s a = ST (IO a) -- упрощённо, в GHC устроено сложнее
2 runST :: (forall s. ST s a) -> a

3 sumTo :: Int -> Int
4 sumTo n = runST do
5     ref <- newSTRef 0
6     forM [0..n] \i -> modifySTRef ref (+ i)
7     readSTRef ref
```

²¹В Haskell его включает расширение `ImpredicativeTypes`.

Заметим, что если попытаться вернуть из `runST` ссылку на мутабельную ячейку, то тип результата должен будет содержать фантомный параметр `s`. Но в сигнатуре `runST` тип результата `a` стоит вне области действия квантора по `s`: этот квантор связан внутри аргумента, и снаружи `s` просто не существует. Поэтому такая программа не проходит проверку типов:

```
1 newSTRef :: v -> ST s (STRef s v)
2 ghci> runST (newSTRef True) -- ошибка типизации
```

На практике, чтобы отличать такие локально связанные типовые переменные, используют концепцию уровней²² Peyton Jones [2019].

Типы высших рангов вместе с импредикативным применением образуют **полиморфизм первого класса (first-class polymorphism)**, когда полиморфные типы могут использоваться почти так же свободно, как и любые другие. Классический алгоритм глобального вывода Хиндли-Милнера не справляется (и в общем случае задача неразрешима), так что существует большое количество решений, делающих различные компромиссы. Можно сделать вывод типов локальным, опирающемся только на соседние ноды AST и вспомогательные типовые аннотации Pierce and Turner [2000], Christiansen [2013], Dunfield and Krishnaswami [2019]. Либо же можно попытаться помочь глобальному выводу дополнительной предобработкой (Quick Look²³ Serrano et al. [2020], реализованный в GHC начиная с версии 9.2 и включаемый расширением `ImpredicativeTypes`) или дополнительными регулируемыми конструкциями (FreezeML Emrich et al. [2020]).

2.1.2 Higher-order/kinded polymorphism

Функция `fmap` работает одинаково для списков, `Maybe` и деревьев. Чтобы записать её тип, нужно абстрагироваться уже не по типу элементов, а по самому контейнеру, то есть по типовому конструктору кайнда `Type -> Type`.

Haskell позволяет также абстрагироваться по типам произвольных кайндов, а не только `Type`, как в `data` декларациях (**higher-order/kinded types (HKT)**²⁴), так и в полиморфных функциях. Далее мы встретим немало примеров. Так, тип `Fix`, с которым мы подробно познакомимся в 4.3.2, параметризован не типом, а типовым конструктором, и потому имеет кайнд `(Type -> Type) -> Type`. Катаморфизм же абстрагирован по типу стрелочного кайнда:

```
1 newtype Fix f = Fix (f (Fix f))
2 cata :: forall (f :: Type -> Type) a . Functor f => (f a -> a) -> Fix f -> a
```

Упражнение 16 Какой кайнд у `Fix Maybe`? Приведите несколько значений этого типа. Какой знакомый тип они напоминают?

Цена: чтобы вывод типов оставался простым, Haskell требует, чтобы на месте `f` стоял именно типовой конструктор, а не произвольная функция на типах (мы вернёмся к этому

²²<https://okmij.org/ftp/ML/generalization.html>

²³(youtube) A Quick Look at Impredicativity (Simon Peyton Jones)

²⁴<https://serokell.io/blog/kinds-and-hkts-in-haskell>

в 3.2). Во многих же промышленных языках полиморфизма высших порядков нет вовсе, и далее мы рассмотрим технику, позволяющую закодировать его в таком языке (см. 3.1.8).

2.1.3 Обобщённые алгебраические типы данных (GADTs)

Обобщённые алгебраические типы данных (generalized algebraic data types, GADTs) позволяют приписывать данным на уровне типов больше информации. В качестве модельного примера возьмём синтаксис крошечного языка программирования. Зададимся целью не допустить возможности конструирования в Haskell некорректных с точки зрения типов синтаксических деревьев.

```
1 data Expr = Const Int | IsZero Expr | If Expr Expr Expr
```

Сейчас ничто не мешает составить бессмысленную программу вроде `IsZero (IsZero (Const 1))`: некорректная программа на нашем языке является совершенно корректным значением Haskell.

Как мы знаем, конструкторы данных в Haskell — это обычные функции с той лишь разницей, что их реализация генерируется компилятором (аллокация памяти, размещение полей...). У функций есть тип. Например, `IsZero :: Expr -> Expr`.

В Haskell есть синтаксис определения `data` через задание типов конструкторов²⁵. Он совершенно аналогичен рассмотренному ранее, только гораздо более удобен для сложно организованных структур данных. Рассмотренный ранее тип термов `Expr` будет выглядеть следующим образом:

```
1 data Expr where
2   Const :: Int -> Expr
3   IsZero :: Expr -> Expr
4   If :: Expr -> Expr -> Expr -> Expr
```

Для полиморфных структур данных, на примере списка, используется следующий синтаксис. Имя `elem` нужно исключительно для документации и больше никак его использовать нельзя, оно только маркирует наличие типового параметра и позволяет ему вручную задать кайнд²⁶.

```
1 data List (elem :: Type) where
2   Nil :: List a
3   Cons :: a -> List a -> List a
```

Добавим к `Expr` фантомный типовой параметр `ty`, обозначающий тип Haskell, в который должно быть проинтерпретировано данное выражение, и с помощью GADT зададим конкретные значения `ty` результирующим типам конструкторов. Так, мы говорим, что программа сконструированная с помощью `Const` вычисляется в число, `IsZero` вычисляется в булево значение, а условное выражение — в тип веток:

²⁵https://downloads.haskell.org/ghc/latest/docs/users_guide/exprs/gadt_syntax.html#gadt-style

²⁶Кайнд можно не писать. Либо можно не писать имена параметров и просто приписать кайнд типовому конструктору: `data List :: Type -> Type where ...`

```

1 data Expr ty where
2   Const :: Int -> Expr Int
3   IsZero :: Expr Int -> Expr Bool
4   If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty

5 eval :: Expr ty -> ty

```

Теперь мы можем написать безопасный типизированный интерпретатор. Обратите внимание, что при сопоставлении с образцами конструкторов, у нас уточняется информация о типовом параметре.²⁷

```

1 eval :: Expr ty -> ty
2 eval = \case
3   Const x   -> x           -- ty ~ Int
4   IsZero t  -> eval t == 0 -- ty ~ Bool
5   If c t e  -> if eval c then eval t else eval e

```

Цена: вывод типов при сопоставлении с образцом по GADT без подсказки не работает, поэтому функциям вроде `eval` сигнатура обязательна. Далее мы рассмотрим, как GADT в Haskell выражаются через более базовые механизмы языка и почему вывод типов для них сложнее (3.4.4).

Упражнение 17 Добавьте в язык пары: конструкторы `MkPair :: Expr a -> Expr b -> Expr (a, b)` и `Fst :: Expr (a, b) -> Expr a`. Допишите `eval`. Какие равенства типов возникают в новых ветках?

2.1.4 Структуры на уровне типов, data promotion

Чтобы обрести больший контроль корректности программ, научимся кодировать произвольные структуры данных на уровне типов. В качестве модельной задачи зададим структуру данных, моделирующую вектор, но с контролем длины.

Для начала определим натуральные числа на уровне типов в стиле Пеано:

```

1 data Zero
2 data Suc n

```

Упражнение 18 Сколько обитателей типа `Suc (Suc Zero)`?

Теперь мы можем задать тип вектора, содержащий информацию о длине:

```

1 data Vec (size :: Type) (elem :: Type) where
2   VNil :: Vec Zero a
3   VCons :: a -> Vec n a -> Vec (Suc n) a

```

²⁷Тут используется удобное расширение `LambdaCase`, позволяющее не вводить лишние имена.

```

4 example :: Vec (Suc (Suc Zero)) Int
5 example = VCons 1 (VCons 2 VNil)

```

Для такого типа, например, можно написать безопасную функцию `zip`, работающую только на векторах одинаковой длины:

```

1 vzip :: Vec n a -> Vec n b -> Vec n (a, b)
2 vzip VNil VNil = VNil -- n ~ Zero
3 vzip (VCons x xs) (VCons y ys) = VCons (x, y) (vzip xs ys) -- n ~ Suc n'

```

Заметьте, что в остальных ветках `vzip` должны возникнуть эквивалентности, начинающиеся с различных конструкторов, например, `Zero ~ Suc n`. Поскольку невозможно построить такие аргументы функции, Haskell позволяет соответствующие ветки не рассматривать.

Упражнение 19 Напишите функцию добавления в конец элемента вектора. Двигайтесь последовательно, заполняя типовые дыры и отслеживая возникающие эквивалентности.

Сейчас язык кайндов, который должен контролировать типы, слишком беден. Действительно, кайнд `Suc` — `Suc :: Type -> Type`, соответственно ничто не мешает написать, например, `Suc (Maybe Int)`. В то же время он слишком ограничивающий, поскольку не поддерживает полиморфизм, что дало начало большому количеству дублирований а ля `Typeable (ty :: Type)`, `Typeable1 (ty :: Type -> Type)`...

Современный Haskell позволяет объявлять новые типы и кайнды подобно тому, как `data` позволяет объявлять новые типы²⁸.

```

1 type data Nat = Zero | Suc Nat

```

Теперь вектору можно приписать более точный кайнд:

```

1 data Vec (size :: Nat) (elem :: Type) where
2   VNil :: Vec Zero a
3   VCons :: a -> Vec n a -> Vec (Suc n) a

```

Упражнение 20 Что выведет `ghci> :k Vec`?

Другой вариант добиться того же самого — автоматически продвигать (promotion) все `data` декларации на уровень выше²⁹ Yorgey et al. [2012]. А именно: любой конструктор типа также становится кайндом, а конструктор данных — конструктором типа. Так, в примере с числами, мы можем задекларировать натуральные числа как обычно и использовать на уровне типов:

²⁸Расширение `TypeData`.

²⁹Расширение `DataKinds`.

```

1 data Nat = Zero | Suc Nat
2 ghci> :k Suc -- тут понятно, что Suc используется как тип
3 Suc :: Nat -> Nat

```

Поскольку типы и термы в Haskell живут в разных пространствах имён, можно называть конструкторы типов и данных одинаково. Однако если продвинуть такой тип данных, возникнет неоднозначность: мы имеем в виду тип или продвинутый конструктор. Haskell позволяет указать явно, что речь идёт о продвинутом конструкторе с помощью одинарной кавычки.

```

1 data T = T Nat
2 ghci> :k T
3 T :: Type      -- про конструктор типа
4 ghci> :k 'T
5 'T :: Nat -> T -- про продвинутый конструктор данных

```

Не любые `data` декларации подходят для продвижения, в то же время `type data` декларации позволяют явно запросить структуру уровня типов и получить внятные ошибки, если декларация написана неправильно.

В случае продвижения полиморфного типа, мы получаем полиморфные кайнды³⁰:

```

1 data [a] = [] | (:) a [a]
2 ghci> :k '(:)
3 '(:) :: forall k . k -> [k] -> [k]

```

Term	Type	Kind
<code>Zero</code>	<code>Nat</code>	<code>Type</code>
<code>[Zero, Suc Zero]</code>	<code>[Nat]</code>	<code>Type</code>
<code>[]</code>	<code>forall a. [a]</code>	<code>Type</code>
<code>(:)</code>	<code>forall a. a -> [a] -> [a]</code>	<code>Type</code>
	<code>'Suc 'Zero</code>	<code>Nat</code>
	<code>'['Zero, 'Suc 'Zero]</code>	<code>[Nat]</code>
	<code>'[Int, Double]</code>	<code>[Type]</code>
	<code>'[]</code>	<code>forall k. [k]</code>
	<code>'(:)</code>	<code>forall k. k -> [k] -> [k]</code>

Рис. 6: Пример продвижений в Haskell.

Примеры продвижения различных конструкций можно увидеть в таблице 6.

В качестве примера, зададим гетерогенный список, индексированный типами элементов:

```

1 data HList (tys :: [Type]) where
2   HNil :: HList '[]

```

³⁰Расширение PolyKinds.

```

3   HCons :: ty -> HList tys -> HList (ty ': tys)

4   example :: HList '[Int, Bool, Double]
5   example = HCons 42 $ HCons True $ HCons 12.5 HNil

```

Структуры данных тоже могут быть полиморфными по кайндам. Рассмотрим тип `Tagged`, позволяющий дополнить тип значения дополнительным типовым тегом. Кайнд тега может быть произвольным, поэтому, например, можем использовать встроенные в систему типов строковые и числовые константы³¹ (другой пример использования полиморфных кайндов — тип `Proxy`, см. 2.1.5):

```

1   newtype Tagged (tag :: k) (a :: Type) = Tagged a
2   ghci> :t Tagged
3   Tagged :: forall k (tag :: k) a. a -> Tagged tag a

4   example :: Tagged ("dbId" :: Symbol) Int
5   example = Tagged 42

```

Современный Haskell в итоге пришёл к тому, что система типов не делает различий между типами и кайндами (рис. 7). В частности, `Type :: Type`. Это нужно для расширения возможностей Haskell в сторону программирования с зависимыми типами путём добавления несинтаксических эквивалентностей для кайндов³². Соответствующее расширение *System FC*, внутреннего языка GHC (см. далее 3.4.4), равенствами кайндов описано в работе Weirich et al. [2013]³³³⁴.

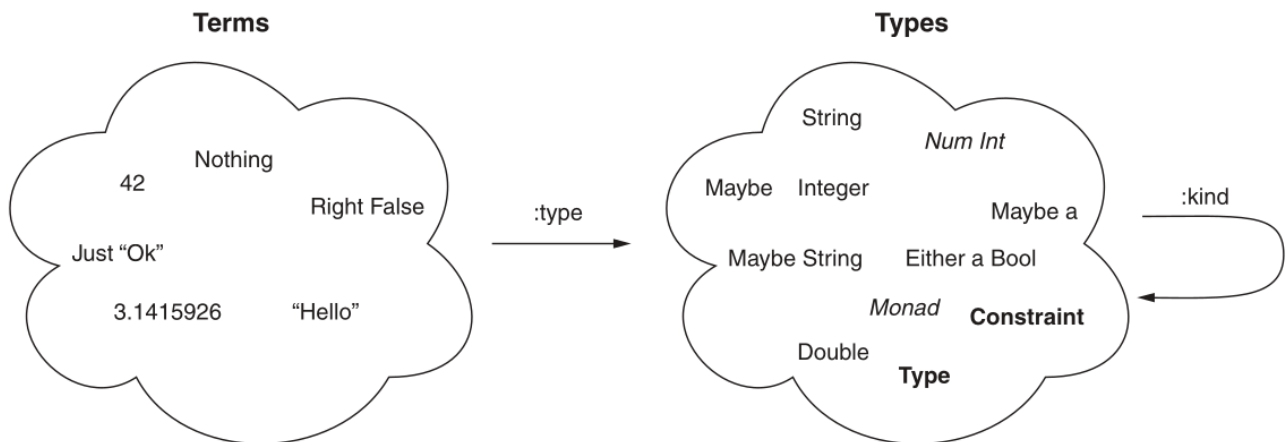


Рис. 7: Типы и кайнды — одно (Bragilevsky).

³¹TypeLits: литералы уровня типов кайндов `Symbol` и `Nat`.

³²Расширение `TypeInType`, ныне часть `PolyKinds`.

³³(youtube) Мини-курс на русском языке про развитие Haskell в сторону зависимой типизации.

³⁴(youtube) Мини-курс на русском языке — система вывода типов Haskell.

2.1.5 Факультативно: эмуляция типовых абстракций и аппликаций (`Proxy`)

Это факультативный и специфичный для Haskell материал. Его можно пропустить при первом чтении и вернуться, когда тип `Proxy` встретится в коде следующих глав.

Проблема здесь историческая: расширения, позволяющие вручную задавать типовые аппликации и абстракции, появились в Haskell сравнительно недавно³⁵. До этого явно передать тип в функцию было нельзя, и пользовались следующей техникой, которая до сих пор встречается во многих библиотеках.

В стандартной библиотеке определён тип `Proxy` с одним параметром. Это **фантомный типовой параметр** — значения соответствующего типа не хранятся в структуре данных, он только позволяет размещать дополнительную информацию на уровне типов³⁶. Соответственно, неинформативную константу `Proxy` можно проаннотировать нужным типом и передать в функцию, чтобы специализировать типовой параметр на нужный тип. Или можно принять `Proxy` и воспользоваться `ScopedTypeVariables` для типовых сигнатур в паттернах³⁷.

```
1 data Proxy a = Proxy

2 id :: Proxy a -> a -> a
3 ghci> :t id (Proxy :: Proxy Int)
4 id (Proxy :: Proxy Int) :: Int -> Int

5 id (Proxy :: Proxy [a]) x = (x :: [a])
```

Иногда прокси-тип оставляют полиморфным, чтобы не зависеть от `Data.Proxy`. Вместо конкретного значения иногда передают специализированное значение $\perp$, а получатель, не зная тип, не сможет его форсировать (однако, любые вхождения $\perp$ в терм слишком раздражают, поэтому это скорее не очень хорошая практика).

```
1 id :: proxy a -> a -> a
2 id (_ :: proxy a) x = (x :: a)

3 ghci> :t id (undefined :: Proxy Int)
4 id (undefined :: Proxy Int) :: Int -> Int
```

2.2 Реализация параметрического полиморфизма

До сих пор полиморфизм был для нас свойством системы типов: мы обсуждали, какие программы он позволяет протипизировать. Теперь посмотрим, сколько он стоит во время исполнения.

Конвенция вызова³⁸ представляет собой набор соглашений между тем как функция ком-

³⁵TypeApplications, TypeAbstractions.

³⁶https://wiki.haskell.org/Phantom_type

³⁷Типовой параметр на самом деле имеет полиморфный кайнд: `data Proxy (a :: k) = Proxy`, чтобы эта техника работала с типами произвольных кайндов (см. 2.1.4).

³⁸https://en.wikipedia.org/wiki/Calling_convention

пилируется и как должна вызываться. Например, функция принимает два аргумента, каждый размером в машинное слово, и возвращает один результат размером в машинное слово. Тогда сгенерированный низкоуровневый код этой функции может, например, ожидать, что оба аргумента передаются через специальную пару регистров, а складывать результат он будет в третий. В таком случае вызывающий код обязан предоставить аргументы в правильных регистрах и ожидать результата в некотором третьем, заранее оговоренном регистре.

В общем случае, конвенция вызова функции зависит от типов аргументов и результата. Нужно знать как минимум их размер, чтобы понять, размещать их в регистрах или на стеке. Нужно знать, это указатель (**reference type**) или значение само по себе (**value type**), чтобы понимать, как с ним работать. В структурах данных нужно знать смещения полей.

Таким образом, реализация параметрического полиморфизма в языке — это не тривиальная задача. Разные языки используют различные подходы, все со своими достоинствами и недостатками.

2.2.1 Мономорфизация

Мономорфизация — самый прямолинейный подход: полиморфные функции и структуры компилируются для каждого набора типовых аргументов. Так, если различных наборов типовых аргументов, с которыми эта функция вызывается, например, 100 (что запросто может быть), то её код будет компилироваться сто раз и занимать в бинарнике в сто раз больше места. Так делают, например, C++ и Rust.

На самом деле всё ещё хуже. Если проект многомодульный и состоит из множества единиц компиляции (кусков, которые компилируются отдельно), то одна и та же специализация функции на типовые аргументы будет компилироваться заново во всех единицах компиляции, где такая специализация нужна. А затем, линкер будет заниматься удалением дубликатов, что тоже не самый быстрый и эффективный процесс.

- + Порождаемый код максимально эффективен для каждого типа;
- + Легко на этапе компиляции отработывают `is`-проверки значений на принадлежность определённому типу (в остальных подходах с этим всё сложно);
- Время компиляции крайне велико;
- Существенно увеличивается размер результирующего бинарного файла, что может быть критично для некоторых приложений;
- Может неэффективно работать из-за засорения кеша кода в процессоре;
- В интерфейсах не может быть полиморфных методов, так как мы не знаем в месте вызова, к какому именно наследнику относится вызываемый метод, и какой код нужно специализировать (аналогично, не работает `higher-rank` полиморфизм);
- К полиморфным функциям нельзя динамически линковаться (у них нет кода до специализации);
- В общем случае нельзя поддержать `variance`, потому что код компилируется для конкретного типа и в общем случае не может работать для произвольного подтипа или

супертипа (если `reference` и `value` типы могут находиться в одной иерархии подтипизации).

Некоторые языки (например, C++ и Zig) не делают инстанциацию скрытой деталью реализации языка, а предоставляют её как инструмент пользователям. Это даёт следующие возможности:

- Если разрешить использовать значения в типах, инстанциация может использоваться как механизм вычислений на этапе компиляции.
- Если отложить проверку ошибок на стадию инстанцирования, то мы получим своего рода статическую утиную типизацию. Это позволит не описывать сложные сигнатуры полиморфных функций. Однако тогда функции для тестирования придётся вручную инстанцировать для всевозможных типов, иначе нельзя будет статически понять, компилируется ли она хотя бы для этих типов.

2.2.2 Стирание типа

Можно всё сделать наоборот, унифицировав значения, которые приходят на вход полиморфным функциям и хранятся в полиморфных структурах данных, вместо того, чтобы компилировать код под каждый тип.

Пусть каждое значение будет аллоцировано в куче и передаваться по указателю. Тогда мы сможем переиспользовать один и тот же код для разных типовых аргументов — он просто будет ожидать указатели.

- + Каждая функция компилируется ровно один раз — быстро;
- + Можно динамически загружать новые полиморфные функции и типы и использовать их друг с другом;
- + Гибкость — вариантность, полиморфные методы в интерфейсах, `higher-rank types` и т.д. просто работают;
- Аллокация в куче и разыменование указателя может очень сильно замедлить код;
- Поскольку информация о типах стирается, нельзя ничего сделать с типовым аргументом, не имея его обитателей (например, запросить рефлексией информацию или сделать `is` проверку).

Такого подхода придерживаются JVM, Haskell и, как правило, другие функциональные языки ввиду его гибкости и скорости компиляции.

Особую проблему вызывает работа с примитивами и другими `value`-типами, потому что каждое значение приходится сначала боксить (переносить в кучу), а потом уже использовать в полиморфном контексте. Поэтому языки борются с этим как могут. Некоторые языки урезают диапазоны значений примитивов, чтобы зарезервировать бит, определяющий, это указатель или значение. Код консультируется с этим битом для работы (похоже на 2.2.4).

```

struct value_witness_table {
    size_t size, align;
    void (*copy_init)(opaque *dst, const opaque *src, type *T);
    void (*copy_assign)(opaque *dst, const opaque *src, type *T);
    void (*move_init)(opaque *dst, opaque *src, type *T);
    void (*move_assign)(opaque *dst, opaque *src, type *T);
    void (*destroy)(opaque *val, type *T);
};

```

Рис. 8: Swift value witness table.

Так делают, например, OCaml и Koka. Агрессивный инлайнинг вместе с другими оптимизациями тоже помогают Blanchet [1999]. Java, например, также пытается аккуратно двигаться в сторону возможности мономорфизации³⁹⁴⁰.

2.2.3 Гибридный подход

C# реализует гибридный подход⁴¹. Они различают значения, хранимые в куче — reference types, и значения, хранимые на стеке — value types. Для первых они генерируют одну специализацию, работающую с указателями. Для каждого набора value-типов они генерируют лениво, в рантайме, специализации.

То есть следы дженериков в таком подходе есть и в промежуточном представлении CIL, и в рантайме.

- + value-типы хранятся и передаются as-is без боксинга;
- + Доступна рефлексия по дженерикам;
- + Небольшое время компиляции;
- Инстанциация в рантайме замедляет исполнение;
- Variance работает только для reference types (что странно — есть “правильная” подтипизация, а есть “неправильная”).

2.2.4 Использование виртуальной таблицы свойств типов

Swift⁴² вместе с каждым типовым параметром передаёт value witness table (рис. 8). Это таблица со всей необходимой информацией о типе: размер и выравнивание, что нужно сделать при копировании и перемещении объекта (например, инкрементировать счётчик ссылок). Таким образом, скомпилированный код постоянно обращается к этой таблице и делает виртуальные вызовы функций из неё (рис. 9).

- + Небольшое время компиляции;
- + Предсказуемая эффективность (не приводит к неожиданным паузам в рантайме);

³⁹Type Specialization of Java Generics - What If Casts Have Teeth?

⁴⁰<https://cr.openjdk.org/~jrose/values/parametric-vm.html>

⁴¹Generics in the runtime (C# programming guide).

⁴²(youtube) 2017 LLVM Developers' Meeting: “Implementing Swift Generics”

```

Example:
func f<T>(_ t: T) -> T {
    let copy = t
    return copy
}

Implementation:
void f(opaque *result, opaque *t, type *T) {
    opaque *copy = alloca(T->vwt->size);
    T->vwt->copy_init(copy, t, T);
    T->vwt->move_init(result, copy, T);
    T->vwt->destroy(t, T);
}

```

Рис. 9: Код полиморфной функции, порождаемый компилятором Swift.

- + Эффективная работа с value-значениями;
- + Высокая гибкость;
- + Информация о типах не стирается;
- Серьёзный константный оверхед на динамические вызовы через таблицу, эффективность очень сильно зависит от компиляторных оптимизаций.

Это своего рода реализация параметрического полиморфизма через специальный. Вместе с типом в функцию неявно передаётся таблица операций, которые умеет этот тип: узнать размер, скопировать, переместить. В следующей главе мы увидим, что ровно так же, передачей словарей функций, реализуются классы типов (3.1.1), только набор операций там задаёт программист, а не компилятор.

2.2.5 Сравнение подходов

Сведём четыре подхода в таблицу 10. Ни один из них не выигрывает по всем строкам: каждый язык выбирает, чем платить.

Упражнение 21 В Kotlin (JVM, стирание типов) проверка `x is T` для типового параметра `T` разрешена только внутри `inline`-функций, где параметр помечен как `reified`. Объясните это ограничение. Какой из рассмотренных подходов к реализации полиморфизма здесь локально применяется?

Упражнение 22 В Rust (мономорфизация) трейт с обобщённым методом нельзя использовать как `trait object` (`dyn Trait`). Каким из перечисленных недостатков мономорфизации это объясняется?

Упражнение 23 Приложение подгружает плагин, скомпилированный отдельно, и хочет вызвать из него обобщённую функцию с типом, о котором автор плагина не знал. Для каждого из четырёх подходов определите, возможно ли это, и почему.

	Мономорфизация (2.2.1)	Стирание типа (2.2.2)	Гибридный подход (2.2.3)	Таблица свойств типа (2.2.4)
Языки	C++ , Rust	JVM, Haskell	C#	Swift
Скорость кода	+ лучшая	— боксинг, индирекции	+ для value-типов	— вызовы через таблицу
Время компиляции, размер бинарного файла	—	+	+	+
Динамическая загрузка полиморфного кода	—	+	+	+
Полиморфные методы в интерфейсах, higher-rank	—	+	+	+
Тип известен во время исполнения	— только статически	— стёрт	+	+

Рис. 10: Сравнение реализаций параметрического полиморфизма.

2.3 Полиморфизм по конвенции вызова

Как мы уже обсуждали выше 2.2.2, параметрический полиморфизм в Haskell реализуется следующим образом: все значения хранятся в куче и передаются в полиморфные функции по указателю. Однако, если для вычислительного кода важна производительность, такой подход не годится ввиду большой нагрузки на подсистему управления памятью и множества индирекций. Поэтому Haskell позволяет также писать код с использованием `unboxed` значений. А если конвенция вызова не принципиальна, можно по ней абстрагироваться и писать один код для `boxed` и `unboxed` значений Eisenberg and Peyton Jones [2017].

2.3.1 Разновидности runtime представлений в Haskell

На рисунке 11 можно увидеть классификацию значений в Haskell с примерами типов. **Unboxed типы** — их значения удерживаются и передаются по значению. **Boxed**, соответственно, наоборот, передаются по указателю и хранятся в куче. Обычный `Int` является просто декларацией следующего вида, где `I#` — это обычный конструктор с необычным именем, содержащий `unboxed` значение.

```
1 data Int = I# Int#
```

	Boxed	Unboxed
Lifted	<i>Int</i> <i>Bool</i>	
Unlifted	<i>ByteArray</i> _#	<i>Int</i> _# <i>Char</i> _#

Рис. 11: Виды значений в Haskell с примерами Eisenberg and Peyton Jones [2017].

Lifted типы — содержат $\perp$ в качестве значения. Иначе говоря, могут содержать отложенные вычисления (это для них специальным образом обеспечивают компилятор и рантайм). **Unlifted типы** — наоборот, не могут быть отложенными. Операции, производящие значения unlifted типов всегда энергичные. Свойство lifted/unlifted называют **levity**. Чтобы распространить дальнейшее изложение на энергичные языки, можно levity заменить на boxity и всё останется справедливым.

в именах типов и функций — это конвенция, показывающая, что где-то рядом происходит работа с unlifted значениями⁴³.

Также в Haskell есть unboxed кортежи, которых не существует на этапе исполнения. Например, следующая функция как бы возвращает пару значений, но в действительности компилятор может их разместить, например, в паре регистров. Соответственно, паттерн-матчинг по таким кортежам, просто позволяет сослаться на каждое из этих значений.

```
1 divMod# :: Int -> Int -> (# Int, Int #)
2 case divMod# n k of (# quot, rem #) -> ...
```

Соответственно, во время исполнения нет никакого различия между по-разному вложенными unboxed кортежами, все они представлены одним и тем же набором значений:

```
1 (# A, (# B, C #) #) ≡ (# (# A, B #), C #) ≡ (# A, B, C #)
```

2.3.2 Классификация значений по runtime представлению

Значения различных типов могут быть на этапе исполнения устроены по-разному. То есть нам нужна некоторая система классификации типов. Но такая система в Haskell уж есть — кайнды. Опишем в виде структур данных предметную область, а потом продвинем на нужный уровень (2.1.4).

Стандартная библиотека Haskell предоставляет следующие типы данных⁴⁴:

```
1 TYPE :: RuntimeRep -> Type
```

⁴³Нужно подключить расширение MagicHash, чтобы пользоваться *#* в идентификаторах.

⁴⁴https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/representation_polymorphism.html

```

2 data Levity = Lifted | Unlifted

3 data RuntimeRep = BoxedRep Levity
4                 | IntRep | DoubleRep
5                 | TupleRep [RuntimeRep]
6                 | SumRep [RuntimeRep]
7                 | ...

8 type LiftedRep = BoxedRep Lifted

9 type Type = TYPE LiftedRep

```

`TYPE` — это магический тип, определённый в компиляторе. Он параметризован runtime-представлением значений. Теперь привычный `Type` — это частный случай с boxed lifted значениями.

- `Int :: TYPE (BoxedRep Lifted)` или `:: Type`
 - `IntRep` и `DoubleRep` соответствуют представлению численных констант (в зависимости от архитектуры процессора, целые числа и числа с плавающей запятой может быть необходимо располагать в различных специальных регистрах)
`Int# :: TYPE IntRep`
 - `Maybe Int :: Type`
 - `Maybe :: Type -> Type`
 - `TupleRep` и `SumRep` — unboxed алгебраические типы, представления параметризованы представлениями хранимых значений
`(# Int, Bool #) :: TYPE (TupleRep '[LiftedRep, LiftedRep])`
 - Хотя во время исполнения вложенность кортежей неразличима, на уровне кайндов она для простоты сохраняется, и типы по-разному вложенных кортежей не унифицируются
- ```

1 (# Int#, (# Int, Double# #) #)
2 :: TYPE (TupleRep '[IntRep, TupleRep '[LiftedRep, DoubleRep]])

```

**Упражнение 24** Какой кайнд у типа `(# Int#, Bool #)`? Проверьте себя в `GHCi`, включив расширения `MagicHash` и `UnboxedTuples`.

### 2.3.3 Representation polymorphism

Выставив runtime-представление в структуре кайндов, мы теперь можем параметризоваться по ним. Например, кайнд функциональной стрелки выглядит следующим образом<sup>45</sup>:

```

1 ghci> :k (->)
2 (->) :: forall {q :: RuntimeRep} {r :: RuntimeRep}. TYPE q -> TYPE r -> Type

```

<sup>45</sup>Выключить упрощения: `ghci> :set -fprint-explicit-foralls -fprint-explicit-runtime-reps`

**Упражнение 25** Подумайте, почему функция имеет *boxed* тип. Может ли быть иначе? Может ли это быть полезным?

К сожалению, Haskell выставляет довольно строгое ограничение: связыватели не могут иметь тип, полиморфный по runtime представлению. Можно легко предположить, почему, — нельзя сгенерировать код функции для работы с параметром произвольного рантайм-представления. Это можно решить только мономорфизацией 2.2.1, но Haskell избегает этого подхода<sup>46</sup>. Сообщество также пытается найти другие решения<sup>47</sup> (что-то вроде 2.2.4).

Например, изначально оператор аппликации был обобщён только по возвращаемому типу. Это не порождает проблем, так как вызывающий код сможет вывести представление и сгенерировать подходящий код:

```
1 ($) :: forall r a (b :: TYPE r). (a -> b) -> a -> b
2 f $ x = f x
```

Однако, было замечено, что для оператора аппликации можно получить другую реализацию, не использующую levity-полиморфное связывание<sup>48</sup>:

```
1 ($) :: forall ra rb (a :: TYPE ra) (b :: TYPE rb). (a -> b) -> a -> b
2 ($) f = f
```

**Упражнение 26** Функция *error* полиморфна по представлению результата:

```
1 error :: forall (r :: RuntimeRep) (a :: TYPE r). String -> a
```

Почему это не нарушает ограничение на связыватели?

Таким образом, в Haskell полиморфизм по представлениям несколько вырожден и помогает лишь в небольшом количестве случаев, однако немаловажных. Если позволить мономорфизацию по *RuntimeRep* параметрам, получится система аналогичная гибридной реализации параметрического полиморфизма 2.2.3, только с большим контролем со стороны программиста над мономорфизацией.

---

<sup>46</sup><https://gitlab.haskell.org/ghc/ghc/-/issues/14917>

<sup>47</sup><https://mail.haskell.org/pipermail/haskell-cafe/2023-January/135770.html>

<sup>48</sup>[https://gitlab.haskell.org/ghc/ghc/-/merge\\_requests/10131](https://gitlab.haskell.org/ghc/ghc/-/merge_requests/10131)

### 3 Специальный (ad-hoc) полиморфизм

Как-то Joe Fasel в разговоре с Philip Wadler высказал идею того, что перегрузка функций (overloading) должна находить своё отражение в типах. Wadler понял его неправильно Hudak et al. [2007]. Но то, что он понял, — оказалось классами типов Wadler and Blott [1989].

Christopher Strachey ввёл классификацию полиморфизма на две категории Strachey [2000]. Параметрический — один и тот же код работает с данными различных типов. **Специальный (ad-hoc) полиморфизм** — код выбирается в зависимости от типа. Например, один и тот же символ умножения по-разному действует на целые числа и на числа с плавающей точкой.

Перегрузка в языках обозначает возможность назвать несколько функций с различными наборами входных параметров одинаково. В месте вызова компилятор статически определяет по типам аргументов, какую из них действительно следует вызвать.

```
1 std::string toString(int x) { ... }
2 std::string toString(std::string fmt, double d) { ... }
```

Классы типов обязуют сначала задекларировать именованную сущность (собственно, класс типов), включающую в себя пачку деклараций функций, которые могут быть перегружены для различных типов.

```
1 class Show a where
2 show :: a -> String

3 instance Show Int where
4 show :: Int -> String
5 show = ...
```

Необходимо заметить, что декларация класса типов содержит формальный типовой параметр, по вхождению которого в тип функции, собственно, выбирается перегрузка. Таких параметров может быть много<sup>49</sup>, они могут иметь стрелочные кайнды. Например, в случае класса типов `Applicative`, выбор реализации операции `pure` будет происходить по типовому конструктору результата, то есть даже не по полноценному типу.

```
1 class Functor f => Applicative (f :: Type -> Type) where
2 pure :: a -> f a
3 ...

4 instance Applicative Maybe where
5 pure :: a -> Maybe a
6 ...
```

Также, в отличие от перегрузки, классы типов совместимы с параметрическим полиморфизмом. Так, в типе полиморфной функции нельзя указать, что для типа должна присутствовать определённая перегрузка. Классы типов же позволяют ограничить набор возможных типовых аргументов теми, для которых реализован инстанс нужного класса типов:

---

<sup>49</sup>Расширение `MultiParamTypeClasses`.

```
1 showPrefixed :: Show a => a -> String -> String
```

Если сравнивать классы типов с переопределением (overriding) в ООП языках, то разрешение вызова виртуальной функции происходит с использованием таблицы, хранящейся в объекте первого параметра (получателя вызова, receiver). Классы типов же опираются исключительно на тип, поэтому, например, возможно определение констант в классах типов<sup>50</sup>:

```
1 class Enum a => Bounded a where
2 minBound :: a
3 maxBound :: a
```

В то же время, классы типов не являются типами, а, скорее, предикатами на типах. Тип удовлетворяет такому предикату, или свойству, если для него есть соответствующий инстанс. Поэтому, в частности, привычный способ в ООП создать гетерогенную коллекцию элементов, имеющих общий интерфейс, напрямую не работает с классами типов. Например, такой тип не будет корректным: `[Show]`. Мы вернёмся к этой проблеме в 3.4.2.

В этой главе мы сначала разберём, как классы типов устроены внутри: это словари функций, которые компилятор передаёт неявно и конструирует автоматически. Затем посмотрим, как та же идея выбора по типу переносится с кода на сами типы (семейства) и на ограничения (каинд Constraint). В завершение рассмотрим, какие возможности языка и полезные техники выражают через классы типов.

## 3.1 Классы типов в языке

Несмотря на поразительное могущество, идея реализации классов типов крайне проста. Она была уже во всей полноте представлена в первой работе Wadler and Blott [1989]. В дальнейших работах уточнялся механизм вывода типов в виде сведения к классической системе типов в стиле Hindley-Milner Hall et al. [1996]. Остальные работы, в основном, предлагают огромное разнообразие различных расширений и приложений Jones et al. [1997].

### 3.1.1 Словари

Рассмотрим идею реализации классов типов на примере полиморфной сортировки. Сортировка для списка элементов конкретного типа пишется тривиально:

```
1 sort :: [Int] -> [Int]
2 sort = \case [] -> []; x:xs -> insert x (sort xs)
3 where
4 insert x xs = let (l, r) = List.partition (< x) xs in l ++ x : r
```

---

<sup>50</sup>Современные ООП языки, тем не менее, стремятся поддержать статические члены в интерфейсах, что делает их ближе к классам типов и позволяет делать похожие вещи. Например, статические требования в протоколах Swift (<https://docs.swift.org/swift-book/documentation/the-swift-programming-language/protocols/>) или статические абстрактные члены интерфейсов в C# (<https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/tutorials/static-virtual-interface-members>).

В реализации единственная информация о типе, которой мы пользуемся — порядок на его обитателях. Таким образом, при переходе к полиморфной сортировке, нам нужно принять словарь с предикатами, задающими нужный порядок для данного типа.

```
1 data OrdDict a = OrdDict { less :: a -> a -> Bool }

2 sort :: OrdDict a -> [a] -> [a]
3 sort d@OrdDict{ less } = \case [] -> []; x:xs -> insert x (sort d xs)
4 where
5 insert x xs = let (l, r) = List.partition (less x) xs in l ++ x : r
```

Теперь, чтобы воспользоваться сортировкой на списке чисел, нужно сконструировать нужный рекорд и вызвать с ним функцию на списке конкретных типов:

```
1 intOrd :: OrdDict Int
2 intOrd = OrdDict { less = (<) }

3 ghci> sort intOrd [3, 2, 1]
4 [1,2,3]
```

Возможна ситуация, когда инстанс для одного типа зависит от инстанса для другого. Например, порядок на списках можно получить автоматически, зная порядок на элементах. В случае словарей мы это моделируем функцией между словарями:

```
1 listOrd :: OrdDict a -> OrdDict [a]
2 listOrd d = OrdDict { less = ... less d ... }
```

Теперь мы можем сортировать список списков, конструируя нужный словарь:

```
1 ghci> sort (listOrd intOrd) [[3, 2], [2, 1], [0]]
2 [[0],[2,1],[3,2]]
```

**Упражнение 27** *Допишите `listOrd` так, чтобы списки сравнивались лексикографически.*

Сравнение явной передачи словарей и классов типов можно увидеть в следующей таблице:

1. Определение словаря функций

```
1 data MyOrd a = MyOrd
2 { less :: a -> a -> Bool }
```

2. Экземпляр словаря для конкретного типа

- Именованное значение

```
1 intMyOrd :: MyOrd Int
2 intMyOrd = MyOrd { less = (<) }
```

3. Явный параметр функции

```
1 sort :: MyOrd a -> [a] -> [a]
```

4. Передаётся пользователем

```
1 test = sort intMyOrd [3, 2, 1]
```

1. Определение класса типов

```
1 class MyOrd a where
2 less :: a -> a -> Bool
```

2. Объявление типа представителем класса типов

- Не имеет имени

```
1 instance MyOrd Int where
2 less = (<)
```

3. Неявный параметр функции

```
1 sort :: MyOrd a => [a] -> [a]
```

4. Передаётся компилятором

```
1 test = sort [3, 2, 1]
```

Таким образом, словарь — это **свидетель (witness)** или доказательство того, что тип удовлетворяет ограничению.

**Упражнение 28** Какой словарь будет соответствовать *higher-kinded* классу типов **Functor**?

### 3.1.2 Неявные аргументы

Можно думать так, что слева от `=>` передаются неявные аргументы функций, выводимые компилятором из контекста. То есть, например, не стоит удивляться вхождению `=>` в типе аргумента, это просто функция с неявным аргументом. Так, следующий код не скомпилируется, потому что в месте использования переменной `x` нет значения типа **Show** `b`:

```
1 f :: (Show b => b) -> b
2 f x = x -- ошибка
```

Можно это значение принять в функции `f`, тогда оно автоматически пропагируется в `x`:

```
1 f :: Show b => (Show b => b) -> b
2 f x = x
```

Haskell даёт возможность делать неявными и обычные аргументы функции<sup>51</sup>. Фактически, это реализация динамического связывания в статическом языке Lewis et al. [2000] (см. далее 5.3.1). Неявные аргументы берутся из скоупа по имени и подставляются автоматически:

```
1 sortBy :: (a -> a -> Bool) -> [a] -> [a]

2 sort :: (?cmp :: a -> a -> Bool) => [a] -> [a]
3 sort = sortBy ?cmp
```

---

<sup>51</sup>Расширение `ImplicitParams`.

Haskell также предоставляет возможность сохранять словари в структуры данных:

```
1 data ShowDict a where
2 ShowDict :: Show a => ShowDict a

3 f :: ShowDict b -> (Show b => b) -> b
4 f d x = case d of ShowDict -> x -- в области видимости доступен инстанс Show b
```

**Упражнение 29** Аргумент  $x$  имеет тип `Show b => b`, то есть ждёт словарь. В какой момент он его получает? Возможна ли именно такая семантика в энергичном языке? Почему?

### 3.1.3 Вывод инстансов

Чтобы вызвать ограниченно-полиморфную функцию, GHC производит вывод инстансов или, иначе говоря, автоматически конструирует свидетелей. Вывод инстансов тесно интегрирован с общей системой вывода типов Haskell Peyton Jones [2019].

В действительности вывод инстансов — это ни что иное, как *задача населения типа*. Действительно, после трансляции в Core (промежуточное представление в GHC), классы типов представляют собой словари функций. У нас в контексте имеются конкретные словари и функции, позволяющие из одних словарей получать другие. Требуется найти терм, конструирующий словарь нужного типа.

Пусть, например, внутри функции `f :: Show a => ..` происходит вызов ограниченно-полиморфной функции `g :: Show [a] => ..`. То есть у нас имеется словарь `d1 :: ShowDict a`, а также функция `d2 :: ShowDict a -> ShowDict [a]`, пришедшая из импортов<sup>52</sup>. Необходимо сконструировать терм типа `ShowDict [a]`. Очевидно, это будет просто аппликация одного к другому: `d2 d1`.

Вывод инстансов происходит рекурсивно. Чтобы вывести `ShowDict [a]`, выводится сначала посылка `ShowDict a`. То есть получается рекурсия по структуре типа. Иначе говоря, вывод инстансов можно эксплуатировать как вычислительный примитив уровня типов. Так, например, мы можем опускать информацию из типов в термы (аналогично `GHC.TypeLits`)<sup>53</sup>:

```
1 type data Nat = Zero | Suc Nat

2 class KnownNat (n :: Nat) where
3 natVal :: Int

4 instance KnownNat Zero where
5 natVal = 0

6 instance KnownNat n => KnownNat (Suc n) where
```

<sup>52</sup>Инстансы можно импортировать пустым импортом: `import Module ()`.

<sup>53</sup>Тип метода `natVal` не упоминает `n`, поэтому вызвать его можно только с явной типовой аппликацией. Такие сигнатуры разрешает расширение `AllowAmbiguousTypes`.

```

7 natVal = 1 + natVal @n

8 ghci> natVal @(Suc (Suc Zero))
9 -- выведется natVal {knownSuc (knownSuc knownZero)}

```

В общем случае процесс населения типа, как можно предположить по вычислительной аналогии, неразрешим. Поэтому GHC накладывает большое количество ограничений на вид инстансов, которые гарантируют тотальность вывода. Подробно эти ограничения описаны в Sulzmann et al. [2007a]. Также GHC предоставляет различные расширения, ослабляющие эти ограничения и перекалывающие часть ответственности на плечи программиста<sup>54</sup>. Если отказаться от гарантии завершимости вывода<sup>55</sup>, можно легко написать разворот списка типов на этапе компиляции, как и любую другую функцию. Пусть класс `ShowT` (`tys :: [Type]`) с методом `showTypes :: String` умеет печатать список типов (его реализацию оставим упражнением, см. 3.4.6):

```

1 class Reverse (acc :: [Type]) (tys :: [Type]) where
2 showReverse :: String

3 instance ShowT acc => Reverse acc '[] where
4 showReverse = showTypes @acc

5 instance Reverse (ty : acc) tys => Reverse acc (ty : tys) where
6 showReverse = showReverse @(ty : acc) @tys

7 ghci> showReverse @'[] @'[Char, Int, Double]
8 "Double Int Char"

```

Можно заметить, что процесс вывода классов типов очень похож на вычисление логических программ, например, на Prolog, только без backtracking'a (перебора различных вариантов решений в поисках подходящего). Как, впрочем, и вывод типов в Haskell Peyton Jones [2019] в целом: собранные по программе эквивалентности можно рассматривать как логическую программу, решение этой системы типовых уравнений — как исполнение этой программы.

Между классами типов и выводом типов существует интересная синергия (рис. 12)<sup>56</sup>. Исходя из термов, выводятся типы. Затем, исходя из типов, выводятся инстансы классов типов. То есть мы пишем какой-то интересный интеллектуальный код, а параллельно с нами компилятор выписывает неинтересный код.

Вывод инстансов опирается только на вид “головы” декларации — справа от `=>`, а ограничения слева применяются постфактум. Это можно использовать, чтобы писать более об-

<sup>54</sup>[https://downloads.haskell.org/ghc/latest/docs/users\\_guide/exts/instances.html](https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/instances.html)

<sup>55</sup>Расширение `UndecidableInstances`.

<sup>56</sup>(youtube) Hackett: a metaprogrammable Haskell.

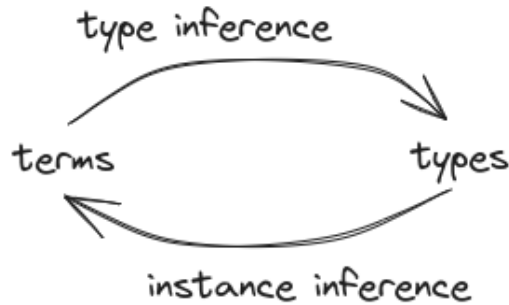


Рис. 12: Классы типов + вывод инстансов = кодогенерация.

щие инстансы. Так, например, работает **constraint trick**<sup>57</sup>, позволяющий разрешать ad-hoc-полиморфные функции в параметрически-полиморфном контексте.

### 3.1.4 Построение типа по значению

После того как мы научились опускать значения из типов, закономерно научиться обратному — поднимать значения в типы. Воспользуемся техникой, описанной в Kiselyov and Shan [2004]. Существует соответствующая библиотека `Data.Reflection` (см. далее 3.4.7).

В действительности мы, конечно, не можем честно получить синтаксически тип нужного размера, просто потому, что типы существуют строго до стадии исполнения. Однако, как мы знаем, словари классов типов имеют воплощение в рантайме (случай полиморфной рекурсии — как раз пример, когда этого нельзя полностью избежать). Поэтому воспользуемся continuation-passing style, который будет подробно рассмотрен далее в главе 6: вместо того чтобы вернуть результат, примем продолжение, умеющее работать с любым типом с `KnownNat` (пользуемся типовыми абстракциями и аппликациями, см. 2.1)<sup>58</sup>:

```

1 reify :: Int -> (forall n. KnownNat n => a) -> a
2 reify n k
3 | n <= 0 = k @Zero
4 | otherwise = reify (n - 1) \ @n' -> k @(Suc n')

```

Продолжение, передаваемое в рекурсивный вызов, захватывает словарь для типа `n` и конструирует словарь для `Suc n`.

Наконец, можем написать следующую удивительную тождественную функцию, поднимающую сначала значение в тип, а потом опускающее тип обратно в термы:

```

1 wonderId :: Int -> Int
2 wonderId n = reify n \ @t -> natVal @t

```

**Упражнение 30** Что вернёт `wonderId (-3)`? Является ли `wonderId` тождественной функцией?

<sup>57</sup><https://chrisdone.com/posts/haskell-constraint-trick/>

<sup>58</sup>В не самых свежих версиях GHC потребуется воспользоваться техникой `Proxy` из 2.1.5.

### 3.1.5 Имплиситы и когерентность

Классы типов можно не делать специальным языковым механизмом, но вместо этого предоставлять на языковом уровне неявные параметры и население, достаточные для реализации классов типов.

Так, в Scala существует механизм имплицитов (implicits) Křikava et al. [2019]<sup>59</sup>. Параметры функций могут быть помечены ключевым словом `implicit`, тогда Scala попытается их вывести самостоятельно с помощью доступных в области видимости `implicit` деклараций. Объявления переменных, функций и конструкторов объектов также могут быть помечены `implicit`, тогда они будут использоваться при населении. Теперь мы можем смоделировать словарь функций, например, с помощью интерфейсов (которые в Scala называются `trait`) и ООП синглтонов, чтобы получить классы типов Oliveira et al. [2010]:

```
1 // Пачка функций.
2 trait Show[T] {
3 def show(x: T): String
4 }

5 // Обёртка для удобства вызова.
6 def show[T](x: T)(implicit ev: Show[T]): String = ev.show(x)

7 // Объект-синглтон, значение для пачки функций.
8 implicit object intShow extends Show[Int] {
9 def show(x: Int): String = x.toString
10 }

11 def showAll[T](xs: List[T])(implicit ev: Show[T]): String =
12 xs.map(show(_)).mkString(", ")
```

Как мы говорили ранее (2.1), работа с типовыми параметрами похожа на работу с обычными. Также, вывод типов можно рассматривать как процесс восстановления пропущенных типовых аппликаций. Заметим, что примерно этим же занимается и механизм вывода имплицитов. Таким образом, при попытке сделать функции с имплиситами функциями первого класса, будут возникать сложности схожие со сложностями first-class полиморфизма (2.1.1).

В языках с зависимыми типами неявные параметры<sup>60</sup> особенно нужны, потому что, например, типы — это ровно такие же параметры функции, как и все остальные. Поэтому вывод типов — это фактически вывод неявных аргументов функций. Более того, зависимые функции, вместе с аргументами часто принимают доказательства каких-то свойств этих аргументов, которые тоже хочется по возможности выводить из контекста автоматически. Такой механизм вывода можно переиспользовать для эмулирования классов типов<sup>61</sup> Devriese and

<sup>59</sup>Дизайн неявных параметров в Scala3 изменился (youtube) Scala Implicits Revisited, Martin Odersky.

<sup>60</sup><https://agda.readthedocs.io/en/v2.7.0.1/language/implicit-arguments.html>

<sup>61</sup><https://agda.readthedocs.io/en/v2.7.0.1/language/instance-arguments.html>

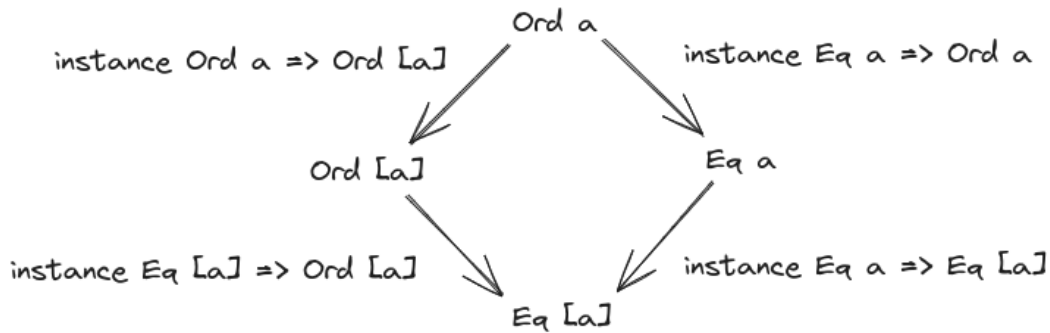


Рис. 13: Когерентность инстансов — диаграмма коммутрует.

Piessens [2011]. В обратную сторону тоже работает — можно механизмы зависимой типизации эмулировать классами типов McBride [2002].

Как мы увидим далее, неявные параметры сами по себе тоже нужны как статическая аппроксимация динамических свободных переменных для реализации системы эффектов (см. далее 8). Однако, иметь классы типов отдельной языковой возможностью всё же полезно, несмотря на то, что они, вроде бы, представляют собой те же неявные параметры (пусть и с рекурсивным механизмом населения). Так, можно поддержать важное свойство при соблюдении всех ограничений, т.е. при отсутствии **orphan instances**<sup>62</sup>. **Когерентность инстансов (coherence)** — для одного типа все инстансы данного класса типов, полученные разными способами, неотличимы (рис. 13). Соответственно, не имеет значения происхождение того или иного инстанса. Иначе говоря, об этом можно не думать, это снимает существенное количество когнитивной нагрузки и упрощает рефакторинг<sup>63</sup>. В то время как остальные подходы требуют трепетного отношения к контексту вызова, потому что из него может прийти неожиданная реализация.

**Упражнение 31** Множество **Set** *a* хранит элементы упорядоченными и при вставке и поиске пользуется порядком **Ord** *a*. Что произойдёт, если множество построить с одним порядком на элементах, а искать в нём с другим? Почему в *Haskell* такая ситуация невозможна, а в языке с имплицитами возможна?

### 3.1.6 Правила (rules) и специализация

GHC позволяет прямо в коде, с помощью специально прагмы, указывать оптимизирующие правила переписывания для компилятора<sup>64</sup> Jones et al. [2001]. Например:

```
1 {-# RULES
2 "map/map" forall f g xs. map f (map g xs) = map (f . g) xs
```

<sup>62</sup><https://stackoverflow.com/questions/3079537/orphaned-instances-in-haskell>

<sup>63</sup>Edward Kmett - Type Classes vs. the World.

<sup>64</sup>[https://downloads.haskell.org/ghc/latest/docs/users\\_guide/xts/rewrite\\_rules.html](https://downloads.haskell.org/ghc/latest/docs/users_guide/xts/rewrite_rules.html)

```

3 "map/append" forall f xs ys. map f (xs ++ ys) = map f xs ++ map f ys
4 #-}

```

Первый закон представляет собой не что иное, как закон функторов. В идеале, мы формулируем законы на этапе дизайна Maguire [b], проверяем их выполнение с помощью property-based testing<sup>65</sup>, а потом используем их для оптимизаций.

Можно переписать полиморфную версию функции на специализированную, если типы подходят. Для этого нужно реализовать специализированную версию (совпадение семантики — полностью ответственность программиста) и задать соответствующее правило переписывания:

```

1 genericLookup :: Ord a => Table a b -> a -> b
2 intLookup :: Table Int b -> Int -> b

3 {-# RULES "genericLookup/Int" genericLookup = intLookup #-}

```

Основной эффект такой оптимизации — гарантированное превращение динамических вызовов функций классов типов в статические (потому что тип известен, следовательно, — и соответствующий ему словарь).

### 3.1.7 Отступление: дефункционализация

**Дефункционализация (defunctionalization)** — техника избавления от функций высших порядков в программе<sup>66</sup> Xia. Впервые предложена в Reynolds [1972, 1998].

Идея заключается в том, чтобы заменить каждое создание лямбда-функции вызовом конструктора некоторого алгебраического типа данных. А каждый call-site функции заменить на вызов специальной first-order функции apply, интерпретирующей данный алгебраический тип.

Рассмотрим пример из функции высших порядков map и двух колсайтов, создающих лямбда-функции:

```

1 map :: (Int -> Int) -> [Int] -> [Int]
2 map f = \case [] -> []; x:xs -> f x : map f xs

3 example1 xs = map (\x -> x + 1) xs
4 example2 y xs = map (\x -> x * y) xs

```

Для каждого лямбда-литерала заводим по конструктору, хранящему замыкание. Аппликацию заменяем на вызов apply. Таким образом получили функцию map первого порядка.

```

1 data Fun = F1 | F2 Int
2 apply :: Fun -> Int -> Int
3 apply df x = case df of F1 -> x + 1; F2 y -> x * y

```

<sup>65</sup>(youtube) John Hughes - Keynote: How to specify it!

<sup>66</sup><https://en.wikipedia.org/wiki/Defunctionalization>

```

4 map :: Fun -> [Int] -> [Int]
5 map df = \case [] -> []; x:xs -> apply df x : map df xs

6 example1 xs = map F1 xs
7 example2 y xs = map (F2 y) xs

```

**Упражнение 32** Добавим третий колсайт, в котором функция приходит снаружи:

```

1 example3 f xs = map (\x -> f x + 1) xs

```

Как его дефункционализировать? Что для этого нужно знать об остальной программе?

### 3.1.8 Эмуляция полиморфизма высших порядков

Далеко не во всех языках есть полиморфизм высших порядков (2.1), но иногда он бывает полезен. Самое распространённое его применение — эмуляция классов типов стрелочных кайндов вроде `Monad`.

Заметим, что типовый конструктор кайнда `Type -> Type` — это функция на типах, принимающая один тип, и возвращающая другой. Применим дефункционализацию, чтобы избежать необходимости параметризовать один типовый конструктор другим Xia, Yallor and White [2014].

Поставим в соответствие типовому конструктору `List` тип-“символ” `ListSym` (для примера используем Kotlin):

```

1 class ListSym

```

Заведём тип, соответствующий аппликации символа к типу, хранящий оригинальное значение со стёртым типом:

```

1 class Apply<Sym, T>(val value: Any)

```

Установим изоморфизм между изначальным типом, полученным типовой аппликацией конструктора, и новой аппликацией символа:<sup>67</sup>

```

1 fun <T> List<T>.to(): Apply<ListSym, T> = Apply(this)
2 fun <T> Apply<ListSym, T>.from(): List<T> = this.value as List<T>

```

Теперь мы можем объявить интерфейс монад и задать реализацию для списка с помощью объекта-синглтона:

---

<sup>67</sup> Слева от точки в декларации указывается дополнительный аргумент функции с синтаксисом передачи совпадающим с вызовом метода на объекте. Из тела функции на него можно ссылаться с помощью `this`.

```

1 interface Monad<M> {
2 fun <T> pure(x: T): Apply<M, T>
3 infix fun <T, R> Apply<M, T>.bind(k: (T) -> Apply<M, R>): Apply<M, R>
4 }

5 object ListMonad : Monad<ListSym> {
6 override fun <T> pure(x: T): Apply<ListSym, T> = listOf(x).to()
7 override fun <T, R> Apply<ListSym, T>.
8 bind(k: (T) -> Apply<ListSym, R>): Apply<ListSym, R> =
9 this.from().flatMap { k(it).from() }.to()
10 }

```

И наконец мы можем писать функции над произвольными монадами:<sup>68</sup>

```

1 fun <M> Monad<M>.go(x: Apply<M, Int>): Apply<M, Int> =
2 x bind { it -> pure(it + 1) } bind { it -> pure(it + 2) }

3 fun test(xs: List<Int>): List<Int> = ListMonad.go(xs.to()).from()

```

Не лишним будет отметить, что результирующий код выглядит несколько чудовищно. Скорее всего, использование этой техники не окупает себя и нужно выбирать другой стиль программирования.

## 3.2 Семейства

Идея ad-hoc полиморфизма в том, чтобы в зависимости от типа получать различный код. Семейства начинались как продолжение этой идеи в плоскость данных и типов. Так, ассоциированные синонимы типов — различные типы для различных индексов (типовых параметров) Chakravarty et al. [2005a]. Ассоциированные **data** — различные представления для различных индексов Chakravarty et al. [2005b]. В конце концов эти идеи были обобщены до открытых семейств Schrijvers et al. [2008], потом были введены закрытые Eisenberg et al. [2014].

Можно считать, что семейства<sup>6970</sup> — это типовые конструкторы, задающие множество типов. Конкретный тип из множества можно выбрать, передав типовой параметр, называемый **индексом**. Сравните с обычными полиморфными конструкторами типа, которые ведут себя одинаково вне зависимости от типовых параметров.

Большое количество интересных примеров использования можно найти, например, в Kiselyov et al. [2010].

<sup>68</sup>Аргументы слева от точки умеют самостоятельно запрыгивать в последующие вызовы. Собственно говоря, они являются неявными параметрами.

<sup>69</sup>[https://ghc.gitlab.haskell.org/ghc/doc/users\\_guide/exts/type\\_families.html](https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/type_families.html)

<sup>70</sup><https://serokell.io/blog/type-families-haskell>

### 3.2.1 Data families

Data families позволяют выбирать декларацию алгебраического типа в зависимости от типового индекса. Например, для более эффективной реализации структур данных. Это напоминает специализацию шаблонов в C++.

```
1 data family XList elem
2 data instance XList () = UnitList Int -- достаточно хранить длину
3 data instance XList Bool = BoolList ByteArray -- по биту на элемент
```

Единственный способ работать с data family — разместить реализацию в классе типов, чтобы она сопровождала значение с того момента, когда типовой индекс конкретный.

```
1 class XListOp elem where
2 xelem :: elem -> XList elem -> Bool

3 instance XListOp () where
4 xelem () (UnitList size) = size > 0

5 instance XListOp Bool where
6 xelem key (BoolList bs) = ...

7 xelemAll :: XListOp elem => XList elem -> [elem] -> Bool
8 xelemAll xs = all (`xelem` xs)
```

К сожалению, в отличие от специализации шаблонов, нельзя задать определение по умолчанию вроде `data instance XList a = AList [a]`.

### 3.2.2 Synonym families

Synonym families (семейства типов) фактически являются функциями на типах. Они бывают открытыми, закрытыми и ассоциированными. Открытые семейства открыты в том смысле, что инстансы можно писать отдельно от декларации, подобно классам типов. Закрытые, наоборот, полностью описываются в одном месте, кейсы упорядочены сверху вниз, что несколько ослабляет ограничения на паттерны.

```
1 type family Plus (n :: Nat) (m :: Nat) :: Nat where
2 Plus Zero m = m
3 Plus (Suc n) m = Suc (Plus n m)
```

Чтобы посмотреть, во что вычисляется нечто на уровне типов, можно воспользоваться следующей командой в ghci:

```

1 ghci> :k! Plus (Suc Zero) (Suc (Suc Zero))
2 Plus (Suc Zero) (Suc (Suc Zero)) :: Nat
3 = Suc (Suc (Suc Zero))

```

**Упражнение 33** Реализуйте закрытое семейство `Length :: [Type] -> Nat`, вычисляющее длину списка типов.

Ассоциированные семейства работают аналогично, только объявляются в рамках некоторого класса типов, являясь некоторой функциональной альтернативой функциональным зависимостям между параметрами класса<sup>71</sup> Jones [2000] (которые выглядят скорее реляционно). Иначе говоря, позволяют поставить в соответствие типу, для которого написан инстанс, другой тип. Например, коллекции — тип её элементов:

```

1 class Container c where
2 type Elem c
3 elements :: c -> [Elem c]

4 instance Container [a] where
5 type Elem [a] = a
6 elements = id

7 instance Container ByteString where
8 type Elem ByteString = Word8
9 elements = ByteString.unpack

```

В современных языках часто встречаются ассоциированные семейства под видом ассоциированных типов<sup>72</sup>. Так, Swift сильно полагается на ассоциированные типы: типовых параметров у протоколов (интерфейсов) в нём нет<sup>73</sup>. В то же время Scala пытается отслеживать, в присутствии экзистенциальных типов (интерфейсов), из какого именно значения пришёл тот или иной ассоциированный тип с помощью *path-dependent types* Amin et al. [2014].

### 3.2.3 Инъективные семейства

Семейства типов отличаются от типовых конструкторов примерно так же, как функции от конструкторов данных. Конструкторы пассивны и не редуцируются (`Maybe Int`), в то время как функции вычисляются в какой-то результат (e.g. `F Int ~ Bool`). В частности, как и функции, семейства не обязательно инъективны.

Для типовых конструкторов, зная, что сконструированные ими типы эквивалентны, можно вывести, что типовые аргументы эквивалентны тоже. Например:

<sup>71</sup>Расширение `FunctionalDependencies`.

<sup>72</sup>Ассоциированные типы являются фактически экзистенциальными типами, их связывает с ассоциированными семействами логический процесс сколемизации.

<sup>73</sup>(youtube) 2017 LLVM Developers' Meeting: "Implementing Swift Generics"

```
1 Maybe a ~ Maybe b ⇒ a ~ b
```

Очевидно, что для семейств типов это свойство по умолчанию не выполняется:

```
1 type family NonInjective a where
2 NonInjective Int = Double
3 NonInjective Char = Double
```

Haskell предоставляет явный синтаксис для объявления инъективных семейств типов, напоминающий функциональные зависимости в классах типов<sup>74</sup> Stolarek et al. [2015]. Конечно, компилятор проверит, что реализация инъективна. Синтаксис требует связать результат именем через равенство и указать, аналогично функциональным зависимостям, что результат определяет какие-то из типовых индексов семейства:

```
1 type family InjectiveB a b = r | r -> b
2 ...
```

### 3.2.4 Семейства первого класса

Помимо инъективности, семейства типов также не обязательно обладают свойством **generativity**, критически важным для вывода типов, — один и тот же результат не обязательно получен из одного и того же семейства:

```
1 f a ~ g a ⇒ f ~ g
```

Вместе инъективность и **generativity** — **matchability**. Когда Haskell работает с типом стрелочного кайнда, он подразумевает, что этот тип **matchable**. Соответственно, семейства не могут передаваться в качестве параметров, а все их вхождения должны быть полностью применёнными ко всем аргументам (**fully saturated**). Либо нужно явно указать, что семейство возвращает конструктор (в этом конкретном месте семантика зависит от переноса аргументов направо от `::`):

```
1 type family ToCtor (s :: Symbol) :: Type -> Type where
2 ToCtor "maybe" = Maybe
3 ToCtor "identity" = Identity
```

Одним из способов обойти это ограничение является дефункционализация семейств Xia, Eisenberg and Stolarek [2014]. Как мы и обсуждали ранее 3.1.7, вместо функции первого класса заводится некоторый символ, обозначающий её, и функция интерпретации, умеющая сделать действие, соответствующее этому символу. В данном случае функцией интерпретации будет открытое семейство `Eval` [Maguire, a, глава 10].

Символ функции, вычисляющей результат кайнда `a`, будем представлять типовым конструктором, результат которого имеет кайнд `Exp a`. Реализацию функции зададим инстансом семейства `Eval` для этого символа:

---

<sup>74</sup>Расширение `TypeFamilyDependencies`.

```

1 type Exp a = a -> Type
2 type family Eval (e :: Exp a) :: a

3 data Snd :: (a, b) -> Exp b
4 type instance Eval (Snd '(a, b)) = b

```

Декларация `data` обязана в итоге возвращать кайнд `Type`, поэтому объявить символ с кайндом `(a, b) -> b` нельзя. Вместо этого результирующий кайнд выражения записан фантомным параметром при `Exp`, который никогда не применяется: полный кайнд символа — `Snd :: (a, b) -> b -> Type`. По этому параметру семейство `Eval` и узнаёт, какого кайнда результат вернуть. Таким образом, кайнд `Exp` — это чисто трюк, позволяющий приписать кайнды таким программам на уровне типов.

Символы — обычные типовые конструкторы, поэтому их, в отличие от семейств, можно передавать в другие функции не до конца применёнными. Так, можно написать функцию высшего порядка на типах:

```

1 data Map :: (a -> Exp b) -> [a] -> Exp [b]
2 type instance Eval (Map f '[]) = '[]
3 type instance Eval (Map f (x : xs)) = Eval (f x) : Eval (Map f xs)

4 ghci> :k! Eval (Map Snd '['(Int, Bool), '(Char, Double)])
5 = [Bool, Double]

```

Сравните с дефункционализацией на уровне термов: конструкторам `F1` и `F2` соответствуют декларации `Snd` и `Map`, а функции `apply` — семейство `Eval`. Поскольку семейство открытое, новые функции можно добавлять, не трогая уже написанные.

**Упражнение 34** *Реализуйте в этом стиле свёртку списка типов:*

```

1 data Foldr :: (a -> b -> Exp b) -> b -> [a] -> Exp b

```

В Haskell ведутся работы<sup>75</sup> по устранению saturation ограничения Kiss et al. [2019]. Для этого нужно различать `matchable` и `unmatchable` типовые функции. Это предлагается делать дополнительным индексированием стрелочных кайндов:  $\rightarrow \equiv \rightarrow^M$  и  $\rightarrow \equiv \rightarrow^U$ . И, конечно, эти индексы могут быть полиморфными.

```

1 data Matchability = Matchable | Unmatchable

2 hMap
3 :: forall (m :: Matchability) (c :: Type -> Constraint)
4 . forall (f :: Type ->m Type) (as :: [Type])
5 . All c as => (forall a. c a => a -> f a) -> HList as -> HList (Map f as)

```

---

<sup>75</sup>GHC proposal: Unsaturated Type Families.

### 3.3 Кайнд Constraint

Давно появлялись предложения добавить в GHC поддержку синонимов для констреинтов, семейств констреинтов и т.д Orchard and Schrijvers [2010]. В итоге был предложен<sup>76</sup> и реализован<sup>77</sup> некоторый механизм унификации типов и констреинтов. Таким образом, всё, что работало для типов, стало работать и для констреинтов.

В GHC был добавлен специальный кайнд `Constraint`<sup>78</sup>:

- Класс типов конструирует констреинт: `Monad :: (Type -> Type) -> Constraint`;
- Эквивалентность является констреинтом: `(a ~ b) :: Constraint`;
- Пустой кортеж констреинтов является констреинтом: `() :: Constraint`;
- Кортеж констреинтов является констреинтом: `(Eq a, a ~ b) :: Constraint`.

Теперь, например, мы можем реифицировать словарь как объект языка:<sup>79</sup>

```
1 data Dict (c :: Constraint) where
2 Dict :: c => Dict c
```

Вспомним гетерогенный список `HList`, рассмотренный ранее (2.1.4). Эта структура данных является first-class аналогом variadic generics в C++ или Swift<sup>80</sup> (собственно, смысл вариадиков — не работать `HList`-подобными структурами напрямую). Например, мы можем написать `map` для такой структуры, если все типы удовлетворяют определённому ограничению. Для этого сначала реализуем семейство, генерирующее кортеж констреинтов для каждого типа из списка:

```
1 type family All (c :: k -> Constraint) (tys :: [k]) :: Constraint where
2 All c '[] = ()
3 All c (ty : tys) = (c ty, All c tys)
4
5 -- All Show [Int, Double] ~ (Show Int, (Show Double, ()))
```

Теперь можем реализовать `map`:

```
1 hmap :: forall c res tys . All c tys
2 => (forall ty . c ty => ty -> res) -> HList tys -> [res]
3 hmap f = \case
4 HNil -> []
5 HCons x xs -> f x : hmap @c f xs
6
7 ghci> hmap @Show show (HCons (1 :: Int) $ HCons 'a' HNil)
8 ["1", "a"]
```

<sup>76</sup><https://gitlab.haskell.org/ghc/ghc/-/wikis/kind-fact>

<sup>77</sup><http://blog.omega-prime.co.uk/2011/09/10/constraint-kinds-for-ghc/>

<sup>78</sup>Расширение `ConstraintKinds`.

<sup>79</sup><https://hackage.haskell.org/package/constraints-0.14.2/docs/src/Data.Constraint.html#Dict>

<sup>80</sup><https://github.com/swiftlang/swift-evolution/blob/main/proposals/0398-variadic-types.md>

### Упражнение 35 *Реализуйте сравнение гетерогенных списков:*

```
1 heq :: All Eq tys => HList tys -> HList tys -> Bool
```

Откуда в рекурсивном вызове берётся словарь для хвоста списка?

Больше такого рода упражнений в гетерогенных конструкциях можно найти в de Vries and Löh [2014].

Констрейнты также могут быть параметрически-полиморфными<sup>81</sup> Bottu et al. [2017]:

```
1 data Rose f x = Rose x (f (Rose f x))

2 instance (Eq a, forall b. Eq b => Eq (f b)) => Eq (Rose f a) where
3 Rose x1 rs1 == Rose x2 rs2 = x1 == x2 && rs1 == rs2
```

Типовые переменные, обозначающие констрейнты, компилятор не выводит. Обычную типовую переменную он восстанавливает по типам аргументов, а переменная-констрейнт встречается только слева от `=>`, и подсказать её значение нечему: компилятор ищет свидетельство для уже известного констрейнта, но не подбирает констрейнт под имеющиеся свидетельства. Поэтому, если в типе функции есть `forall` (`c :: Constraint`) . `c => ...`, значение `c` приходится указывать вручную типовой аппликацией. Именно так мы вызывали `hmap`: без `@Show` вызов не типизируется, хотя первым аргументом и передана функция `show`.

Словари существуют во время исполнения, а значит, у констрейнтов, как и у типов, есть рантайм-представление (2.3)<sup>82</sup>:

```
1 CONSTRAINT :: RuntimeRep -> Type
2 type Constraint = CONSTRAINT LiftedRep
```

Обычный словарь представлен указателем на структуру в куче, отсюда `LiftedRep`. Параметр позволяет свидетельству иметь и другое представление, как `TYPE` позволяет это обычным значениям. Задуманные применения — неявные параметры неупакованных типов, передаваемые без упаковки, и строгие классы, словари которых не бывают отложенными вычислениями<sup>83</sup>.

## 3.4 Использование ad-hoc полиморфизма

Часто языки, имеющие что-то напоминающее классы типов, стремятся выразить через них как можно больше других языковых возможностей и полезных техник. Оказывается, это на удивление мощный механизм. Рассмотрим в этом параграфе некоторые избранные примеры.

<sup>81</sup>[https://downloads.haskell.org/ghc/latest/docs/users\\_guide/exts/quantified\\_constraints.html](https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/quantified_constraints.html)

<sup>82</sup><https://hackage.haskell.org/package/base-4.21.0.0/docs/GHC-Exts.html#t:CONSTRAINT>

<sup>83</sup>GHC proposal: Generalising Type and Constraint. На момент GHC 9.10 эти возможности ещё не реализованы.

### 3.4.1 Сериализация

Классическим примером использования классов типов является сериализация. Проблема в том, что десериализация производится, когда самого объекта ещё нет (ущербный Java подход заполнения объекта дефолтными значениями с последующей мутацией мы не рассматриваем). Поэтому нет возможности написать ООП интерфейс `Serializable`.

Стандартная библиотека сериализации в Kotlin<sup>84</sup> предоставляет сущность `KSerializer`, которая является интерфейсом для отдельного объекта-сериализатора нашего типа (для эффективности тут используется CPS в виде потоков событий `encoder` и `decoder`, вернёмся к этому подходу далее 4.4.1):

```
1 interface KSerializer<T> {
2 fun serialize(encoder: Encoder, value: T)
3 fun deserialize(decoder: Decoder): T
4 }
```

Очевидно, чтобы сконструировать сериализатор полиморфного типа, нужны сериализаторы типов-параметров. Что уже отчётливо напоминает классы типов.

```
1 class PairSerializer<K, V>(
2 keySerializer: KSerializer<K>,
3 valueSerializer: KSerializer<V>,
4): KSerializer<Pair<K, V>> { ... }
```

Однако в Kotlin нет классов типов, а значит создавать сериализаторы придётся вручную. С этим, однако несколько помогает пачка технологий: `inline fun`, `reified` дженерики, рефлексия и компиляторный плагин библиотеки, который, однако не безопасен с точки зрения типов.

### 3.4.2 Экзистенциальные типы

Квантор существования в типах является механизмом инкапсуляции [Pierce, 2002, глава 24]. Так, мы можем в типе  $\tau$  заменить вхождения подтипа  $\sigma$  на переменную  $\alpha$  и получить  $\exists\alpha. \tau'$ , где  $[\alpha \mapsto \sigma] \tau' \equiv \tau$ . Таким образом, мы скрыли от пользователя часть типа.

Например, можно с помощью экзистенциальных типов привести разные типы к одной форме и сложить их значения в один список. В Haskell экзистенциальный тип выражают через `data` декларацию, в которой полиморфный конструктор данных и мономорфный тип<sup>85</sup>. Так, тривиальный экзистенциальный тип  $\exists\alpha. \alpha$  можно получить в Haskell в виде типа `Any`, заданного следующим образом:

```
1 data Any where
2 Any :: forall a . a -> Any -- логически эквивалентно (exists a . a) -> Any
```

---

<sup>84</sup><https://github.com/Kotlin/kotlinx.serialization>

<sup>85</sup>Подобно идиоме type erasure в C++.

```

3 list :: [Any]
4 list = [Any 42, Any "Hello", Any (Just Nothing)]

```

В месте деконструирования `Any`, будет доступно значение некоторого неизвестного типа. Очевидно, с таким значением ничего сделать нельзя. Однако, помимо значения, можно положить в конструктор свидетельство о том, что этот неизвестный тип удовлетворяет некоторому классу типов<sup>86</sup>.

```

1 data Has (c :: Type -> Constraint) where
2 Has :: c a => a -> Has c

```

Значение типа `Has` свидетельствует о том, что существует некоторый населённый тип `a`, который принадлежит определённому классу типов. Например, рассмотрим `Show`:

```

1 showAll :: [Has Show] -> String
2 showAll = List.intercalate ", " . map \(Has x) -> show x

```

В общем случае, чтобы элиминировать такой тип данных нужны типы высших рангов [Pierce, 2002, глава 24.3]:

```

1 foldHas :: Has c -> (forall a . c a => a -> b) -> b
2 foldHas (Has x) k = k x

```

Подробнее можно посмотреть в [Maguire, а, глава 7] и [Pierce, 2002, глава 24].

**Упражнение 36** Почему нельзя написать функцию `unHas :: Has c -> a`, достающую значение из обёртки? Чем от неё отличается `foldHas`?

### 3.4.3 Разрешение имён

Процесс разрешения имён (name resolution) в языках программирования определяет, с какой программной сущностью связать то или иное употребление имени. Разрешение имён рассматривает импорты, иерархию пространств имён и скоупов, типы выражений. . . Как правило, это сложный процесс, неотделимый от вывода типов.

Однако, в GHC стадия разрешения имён довольно простая и отрабатывает до вывода типов. На её тривиальную суть намекает её название — Renamer — она просто переписывает имена в программе на fully-qualified имена, опираясь на импорты.

С одной стороны, простота — это хорошо. С другой — строгое отделение от вывода типов накладывает неприятное ограничение: типы не могут участвовать в разрешении имён. Наиболее остро эта проблема стоит с метками полей в рекордах. Приходится называть все поля в модуле по-разному, чтобы избежать клешей.

Чтобы заставить разрешение имён зависеть от типов, Haskell снова прибегает к классам типов. А именно, определяется класс типов `IsLabel`, который зависит от символа и ожидаемого типа:

<sup>86</sup>Подобно Rust fat pointers (пара из указателя на данные и указателя на таблицу виртуальных функций) и ООП объектам.

```

1 class IsLabel (s :: Symbol) a where
2 fromLabel :: a

```

Для вызова `fromLabel` есть синтаксический сахар<sup>87</sup>:

```

1 #name ≡ fromLabel @"name"

```

Теперь разрешение имени `name` будет учитывать тип<sup>88</sup>:

```

1 data Pet = Pet { name :: String }
2 instance (r ~ String) => IsLabel "name" (Pet -> r) where
3 fromLabel Pet{ name } = name

4 data Person = Person { name :: String, pets :: [Pet] }
5 instance (r ~ String) => IsLabel "name" (Person -> r) where
6 fromLabel Person{ name } = name

7 ghci> #name pet
8 "Rex"

```

Обратите внимание на вид инстансов. Если написать инстанс сразу для типа `Pet -> String`, то вызов `#name pet` не типизируется, пока тип результата не известен из контекста: вывод инстансов смотрит только на голову, а она не совпадает с `Pet -> r`. Поэтому голова оставлена общей, а равенство `r ~ String` вынесено в ограничение и применяется постфактум. Это и есть *constraint trick* из 3.1.3.

### 3.4.4 Несинтаксические типовые эквивалентности, System FC

Современный Haskell является синтаксически богатым языком, который, однако, несмотря на многообразие конструкций, транслируется в маленький типизированный внутренний язык. Это язык *System F<sub>C</sub>* Sulzmann et al. [2007b], расширяющий *System F* (2.1) несинтаксическими эквивалентностями типов. Оказывается, этого достаточно, чтобы поддержать такие возможности Haskell как обобщённые алгебраические типы, ассоциированные семейства типов, функциональные зависимости и т.д.

А именно, вводится встроенный констреинт  $\sim$ , свидетельствующий об эквивалентности двух типов<sup>89</sup>. Например, тип функции `id` может быть записан таким странным образом:

```

1 f :: forall a b . a ~ b => a -> b
2 f = id

```

---

<sup>87</sup>Расширение `OverloadedLabels`.

<sup>88</sup>Чтобы избавиться от ошибки переопределения, нужно включить `NoFieldSelectors`.

<sup>89</sup>[https://ghc.gitlab.haskell.org/ghc/doc/users\\_guide/exts/equality\\_constraints.html](https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/equality_constraints.html)

На самом деле это функция от четырёх параметров: двух типовых параметров, коерции и аргумента. Коерция — это значение размера 0, автоматически выводимое компилятором, которое является свидетельством того, что два соответствующих типа эквивалентны.

Например, GADT из 2.1.3 рассахаривается следующим образом:

```
1 data Expr ty where
2 Const :: Int -> Expr Int
3 IsZero :: Expr Int -> Expr Bool
4 If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty
5 -- транслируется в
6 data Expr ty where
7 Const :: forall ty . ty ~ Int => Int -> Expr ty
8 IsZero :: forall ty . ty ~ Bool => Expr Int -> Expr ty
9 If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty
```

И после паттерн-матчинга по конструкторам, констрейнт эквивалентности попадёт в ветку и позволит системе вывода типов сделать необходимые переписывания.

Про вывод типов при наличии локальных предположений можно почитать в классической статье *OutsideIn(X)* Vytiniotis et al. [2011].

Очевидно, что Haskell может населить констрейнт эквивалентности следуя рефлексивности, симметричности и транзитивности. Также, компилятор может генерировать новые аксиомы (пользователь напрямую свои аксиомы записать не может). Например, по семейству типов компилятор генерирует аксиомы равенства апплицированного конструктора семейства результирующим типам:

```
1 type family Plus (n :: Nat) (m :: Nat) :: Nat where
2 Plus Zero m = m
3 Plus (Suc n) m = Suc (Plus n m)
4 -- раскроется в
5 axiom Plus Zero m ~ m
6 axiom Plus (Suc n) m ~ Suc (Plus n m)
```

**Упражнение 37** *Рассахарьте в равенства типов вектор `Vec` из 2.1.4. Какие равенства окажутся в области видимости в каждой из веток функции `vzip`?*

### 3.4.5 Коерции и роли

Haskell имеет поддержку **generative type abstractions** в виде `newtype` деклараций. Эта техника позволяет задавать доменно-специфичные типы, которые во время исполнения не отличимы от оборачиваемых типов, но позволяют различать их. Так, мы можем ввести обёртки для чисел, которые в предметной области представляют собой идентификаторы различных сущностей. Теперь система типов не даст их перепутать.

```
1 newtype ModuleId = ModuleId Int64
2 newtype CourseId = CourseId Int64
```

Существует крайне недооценённая практика программирования<sup>90</sup>, когда у нас в программе есть чёткая граница, на которой происходит парсинг данных из внешнего мира. После неё сырые неструктурированные данные обогащаются структурой и принимают смысл внутри предметной области. Либо же мы отвергаем эти данные как некорректные. В оставшейся же части программы мы уже пользуемся типизированными данными, свойства которых уже установлены и гарантированы. Например, мы можем быть уверены, что число `ModuleId` строго больше нуля.

Однако, если у нас есть коллекция обёрнутых данных, а мы хотим с ней поработать как с коллекцией сырых, то нам придётся трансформировать коллекцию, несмотря на то, что эта трансформация ничего не делает. В общем случае оптимизатор Haskell не справится её элиминировать, потому что работает с типизированным промежуточным представлением и не сможет избавиться от обхода, меняющего тип<sup>91</sup>.

```
1 newtype Csv = Csv { unCsv :: String }
2 concatC :: [Csv] -> Csv
3 concatC = Csv . concat . map unCsv
```

Поэтому в Haskell есть механизм безопасных коерций между типами, у которых одинаковое представление во время исполнения. Это реализовано с помощью магического класса типов `Coercible`. Его имплементирует компилятор автоматически (см. рис. 14).

```
1 class Coercible from to where -- упрощённо
2 coerce :: from -> to
```

Теперь, можем избавиться от лишней трансформации списка:

```
1 concatC :: [Csv] -> Csv
2 concatC = coerce (concat :: [String] -> String)
```

Безопасность коерций обеспечивает **система ролей**. Каждый типовой параметр имеет специальное свойство — роль.

Роль `phantom` имеют фантомные типовые параметры. Их можно свободно коерсить (нет пререквизитов для инстансов `Coercible`):

```
1 data Phantom h = Phantom
2 data NestedPhantom b = MkNP [Phantom b] | SomethingElse
3 instance Coercible (Phantom a) (Phantom b)
4 instance Coercible (NestedPhantom a) (NestedPhantom b)
```

---

<sup>90</sup><https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/>

<sup>91</sup>Для списков в стандартной библиотеке есть специальное правило переписывания (3.1.6), убирающее такой `map`. Для пользовательской структуры данных обход останется.

The most important rules that GHC uses to solve Coercible constraints are as follows (the full rules are given in Figure 5):

(1) The *unwrapping rule*:

- For every **newtype** `NT = MkNT t`, we have `Coercible t NT` if and only if the constructor `MkNT` is in scope.

(2) The *lifting rule*:

- For every type constructor `TC r p n`, where
  - `r` stands for `TC`'s parameters at a representational role,
  - `p` for those at a phantom role and
  - `n` for those at a nominal role,if `Coercible r1 r2`, then `Coercible (TC r1 p1 n) (TC r2 p2 n)`.

(3) Coercible is an equivalence relation:

- The *reflexivity rule*: `Coercible a a`.
- The *symmetry rule*: If `Coercible a b` then `Coercible b a`.
- The *transitivity rule*: If `Coercible a b` and `Coercible b c` then `Coercible a c`.

Рис. 14: Принципы построения инстансов `Coercible` Breitner et al. [2014].

Типовой параметр имеет роль `representational`, если типовой конструктор можно коерсить только при условии, что можно коерсить аргументы:

```
1 data Maybe a = Nothing | Just a
2 instance Coercible a b => Coercible (Maybe a) (Maybe b)
```

Роль `nominal` имеет типовой параметр, если типовой конструктор можно коерсить только при условии, что аргументы эквивалентны. Это требуется, если типовой аргумент индексирует семейство или констреинт.

```
1 type family F a
2 data Applied a = Applied (F a)
3 instance (a ~ b) => Coercible (Applied a) (Applied b)

4 data ShowDict a where
5 ShowDict :: Show a => a -> ShowDict a
6 instance (a ~ b) => Coercible (ShowDict a) (ShowDict b)
```

Иногда компилятор выводит неправильную роль типовому параметру. Например, если инварианты структуры зависят на конкретную имплементацию какого-то класса типов для типового аргумента, что совершенно не видно в декларации самого типа. В таком случае, роли можно указать явно:

```

1 type role Map nominal representational
2 data Map k v = ...

```

**Упражнение 38** Почему у первого параметра `Map` `k` роль *nominal*? Приведите пример обёртки над ключом, для которой приведение `Map k v` к `Map k' v` сломало бы словарь.

Подробнее можно прочитать в Breitner et al. [2014] и [Maguire, а, глава 8].

### 3.4.6 Type reflection

Рефлексия — это языковой механизм получения информации о типах во время исполнения (на уровне термов). Звучит знакомо, и действительно, Haskell реализует этот механизм через классы типов Peyton Jones et al. [2016].

Библиотека предоставляет магический класс типов `Typeable`, который компилятор реализует автоматически для каждого конкретного типа. Чтобы получить информацию о типе, в области видимости должен быть инстанс `Typeable` для этого типа. Структура типа представлена типом-суммы `TypeRep`, который предоставляет возможность дополнительного типового контроля с помощью обобщённых алгебраических типов данных и типовых тегов.

```

1 class Typeable a where
2 typeRep# :: TypeRep a

```

Например, следующим образом можно получить имя конструктора типа:

```

1 typeName :: forall a. Typeable a => String
2 typeName = tyConName $ typeRepTyCon $ typeRep $ Proxy @a

3 ghci> typeName @Int

```

**Упражнение 39** Реализуйте класс `ShowT` (`tys :: [Type]`) из 3.1.3, который позволяет распечатать список типов.

С помощью структуры представления типа и экзистенциальных типов в Haskell можно эмулировать динамическую типизацию. А именно: любой тип может быть преобразован в `Dynamic`, а потом безопасно преобразован обратно.

```

1 data Dynamic where
2 Dynamic :: Typeable a => a -> Dynamic

3 fromDynamic :: Typeable a => Dynamic -> Maybe a

```

Это может быть полезно, например, для определения гетерогенного хранилища ключ-значение:

```

1 type Store = Map Key Dynamic
2 data Ref ty = Ref Key
3 get :: Typeable ty => Store -> Ref ty -> Maybe ty

```

### 3.4.7 Data reflection

Как мы обсуждали ранее, свойство когерентности гарантирует, что каждому типу в Haskell соответствует ровно один инстанс определённого класса типов. И единственный способ объявить инстанс в Haskell — декларацией на верхнем уровне, то есть он не может зависеть ни от каких локальных данных. Однако в Haskell есть библиотека `Data.Reflection`<sup>92</sup>, которая позволяет создавать локальные инстансы для свежих, чёрной магией сгенерированных<sup>93</sup>, типов.

Она пользуется идеей “поднятия значений в типы”, обсуждённой нами ранее (см. 3.1.4), но в несколько более общем виде. Вместо заведения классов типов вида `Known_`, вводится один класс типов, индексированный типом термов `terms`, которые спускаются из типов:

```
1 class Reifies ty terms | ty -> terms where
2 reflect :: Proxy ty -> terms
```

Также, с помощью следующей функции, библиотека позволяет сгенерировать свежий тип и инстанс `Reifies`, который по этому свежему типу возвращает данное значение типа `a` (переданное первым аргументом). Поскольку он передаётся в функцию высшего ранга, свежий тип не может утечь из скоупа 2.1.1:

```
1 reify :: a -> (forall fresh . Reifies fresh a => Proxy fresh -> res) -> res
```

Чтобы воспользоваться нестандартным инстансом класса типов для некоторого типа `a`, нужно объявить новый тип (например, с помощью `newtype`), содержащий данный, и написать для него нужный инстанс (см., например, `Down`). Мы не хотим объявлять по новой декларации для каждого случая, поэтому заведём обёртку, похожую на `Data.Tagged`, которая позволяет добавлять фантомный типовой тег к типу значения. Варьируя тег, можно получить сколь угодно много типов, оборачивающих данный.

```
1 newtype Wrapped tag a = Wrapped { unwrap :: a }
```

Объявим тип обёртки `Wrapped tag a` представителем нужного класса типов. Код для реализации будем с помощью `reflect` получать по типу тега в виде честного словаря.

```
1 data ReifiedOrd a = ReifiedOrd { compare :: a -> a -> Ordering }
2 instance Reifies tag (ReifiedOrd a) => Ord (Wrapped tag a) where
3 compare = coerce $ compare $ reflect $ Proxy @tag
```

Наконец, можем вызвать функцию сортировки, подменив локально порядок на обратный:

```
1 sort :: Ord a => [a] -> [a]
2 sortReverse :: forall a . Ord a => [a] -> [a]
```

---

<sup>92</sup><https://www.tweag.io/blog/2017-12-21-reflection-tutorial/>

<sup>93</sup><https://www.schoolofhaskell.com/user/thoughtpolice/using-reflection>

```

3 sortReverse xs =
4 let dict = ReifiedOrd { compare = flip compare } in
5 reify dict \(Proxy :: Proxy fresh) ->
6 coerce $ sort @(Wrapped fresh a) $ coerce xs

```

### 3.4.8 Открытые структуры

В динамических языках можно создавать объекты на ходу, последовательно дописывая в них содержимое, и не вводя предварительно декларацию. В Haskell тоже так можно, используя пары для произведений и `Either` для сумм. Например, можно добавить новое поле, создав новую пару: `(oldObj, newField)`.

Однако, такая реализация не оптимальна как с точки зрения эффективности (о более эффективных реализациях можно почитать в [Maguire, а, глава 11]), так и с точки зрения удобства использования. А именно — порядок полей имеет значение и на типах в Haskell нет отношения подтипизации (например, нельзя передать значение с меньшим количеством полей или вариантов). Но можно заметить, что констреинты лишены этих недостатков. Поэтому можно организовывать тип структуры данных, например, таким образом:

```

1 (Int, Double) заменяем на (Member Int d, Member Double d) => Prod d

```

### 3.4.9 Исключения и открытая иерархия

Важный аспект работы с ошибками заключается в том, что многие из них обрабатываются единообразно. Таким образом, ошибки должны образовывать иерархию наподобие той, которая в ООП языках получается с помощью наследования, чтобы иметь возможность реагировать сразу на группу ошибок одним кодом. Так, возникает задача моделирования подобной иерархии в Haskell.

Более того, статически типизированные ошибки это активная область исследований, будем говорить об этом в рамках систем эффектов (см. далее 8). Классические исключения же динамически типизированные. Особенно хорошо этот вариант подходит для ошибок программиста, которые по-хорошему не должны обрабатываться в программе кроме как закрытием ресурсов<sup>9495</sup>.

Поддержка исключений присутствует в системе исполнения Haskell как простого и привычного способа обработки исключительных ситуаций: ошибок программиста, исполнения неполного паттерн-матчинга, асинхронных системных сигналов Marlow et al. [2001]. . . Исключения динамически типизированные и образуют иерархию Marlow [2006]. Если породить исключение может и чистый код, так как  $\perp$ , по семантике Haskell, населяет любой тип, то поймать исключение можно только<sup>96</sup> в `IO` Jones [2001], используя специальные примитивы языка.

Чтобы сделать тип исключением, нужно объявить инстанс `Exception` для него:

<sup>94</sup>Joe Duffy — The Error Model: ошибки программиста отделяются от ожидаемых ошибок и приводят к аварийному завершению.

<sup>95</sup>[https://wiki.haskell.org/Error\\_vs.\\_Exception](https://wiki.haskell.org/Error_vs._Exception)

<sup>96</sup>(stackoverflow) Why can Haskell exceptions only be caught inside the IO monad?

```

1 class (Typeable a, Show a) => Exception a where
2 toException :: a -> SomeException
3 toException = SomeException

4 fromException :: SomeException -> Maybe a
5 fromException (SomeException e) = cast e

```

Где `SomeException` — это экзистенциальная обёртка наподобие `Dynamic` (см. 3.4.6), в которую заворачивается конкретный тип исключения. Ловя `SomeException`, можно поймать любое исключение (`cast` всегда сработает).

```

1 data SomeException where
2 SomeException :: Exception a => a -> SomeException

3 instance Exception SomeException

```

Система исполнения Haskell предоставляет интринсики для кидания и ловли исключений, обернём их для поддержки любого `Exception` типа:

```

1 throw :: Exception e => e -> a
2 throw e = primThrow (toException e)

3 catch :: Exception e => IO a -> (e -> IO a) -> IO a
4 catch io handler = io `primCatch` \e -> case fromException e of
5 Nothing -> throw e
6 Just e' -> handler e'

```

В простейшем случае свой тип исключения можно реализовать в две строчки. Чтобы его поймать, нужно либо ловить сам этот тип, либо `SomeException`, потому что для них обоих `fromException` на объекте вида `SomeException MyError` вернёт `Just`<sup>97</sup>.

```

1 data MyError = MyError deriving Show
2 instance Exception MyError

3 ghci> throw MyError `catch` \(e :: MyError) -> putStrLn "caught"
4 caught

```

Добавим исключение `DivisionByZero` и ещё один более общий тип исключений между ним и `SomeException` — `SomeArithException` (таким образом, будет три способа поймать `DivisionByZero`). Для этого сделаем `SomeArithException` экзистенциальной обёрткой, а каждое арифметическое исключение будем автоматически оборачивать в неё. В `fromException` на каждом уровне вложенности будем пытаться получить оборачивающий конструктор рекурсивным вызовом.

---

<sup>97</sup>`instance Exception SomeException where fromException = Just`

```

1 data SomeArithException where
2 SomeArithException :: Exception a => a -> SomeArithException
3 instance Show SomeArithException where
4 show (SomeArithException e) = show e

5 -- родитель --- SomeException (реализация по умолчанию)
6 instance Exception SomeArithException

7 data DivisionByZero = DivisionByZero deriving Show
8 instance Exception DivisionByZero where
9 toException = toException . SomeArithException
10 fromException e = do
11 SomeArithException e' <- fromException e
12 cast e'

```

Так, во время бросания `DivisionByZero` будет конструироваться объект вида:

```

1 SomeException (SomeArithException DivisionByZero)

```

Реализация `fromException` для конкретного типа умеет убедиться в наличии соответствующего конструктора в результирующем объекте исключения.

**Упражнение 40** Добавьте в иерархию исключение `Overflow`, тоже арифметическое. Какими обработчиками оно ловится, а какими нет?

### 3.4.10 Легковесные частичные стек-трейсы

Забавной эксплуатацией классов типов в Haskell являются легковесные частичные стек-трейсы<sup>98</sup>. Вообще для сбора трейсов нужна поддержка рантайма, что в случае Haskell усложняется ещё и тем, что модель вычислений, редукция графов, реальных трейсов не содержит и их приходится эмулировать. Мы же получим трейсы без поддержки рантайма.

В стандартной библиотеке определён констрейнт `GHC.Stack.HasCallStack`, позволяющий получить информацию о месте вызова функции. Эту информацию фактически размещает компилятор в процессе вывода инстансов. Если в месте вызова доступна информация с уровня выше, компилятор распространяет её дальше. Таким образом, доступна информация только на определённую глубину стека вызовов.

```

1 myHead :: HasCallStack => [a] -> a
2 myHead [] = error "empty"
3 myHead (x:xs) = x

4 bad :: Int

```

<sup>98</sup>[https://downloads.haskell.org/ghc/latest/docs/users\\_guide/exts/callstack.html](https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/callstack.html)

```

5 bad = myHead []

6 ghci> bad
7 *** Exception: empty
8 CallStack (from HasCallStack):
9 error, called at Bad.hs:8:15 in main:Bad
10 myHead, called at Bad.hs:12:7 in main:Bad
11 -- no information about bad call site here

```

`HasCallStack` — это просто имплицит (см. 3.1.2), про который знает компилятор:

```

1 type HasCallStack = (?callStack :: CallStack)

```

### 3.4.11 Кастомизируемые ошибки типизации

При программировании сложных с точки зрения типов библиотек, желательно предоставлять пользователям более информативные ошибки типизации, чем ошибки по умолчанию. Для этого в GHC есть механизм в `GHC.TypeLits`, позволяющий сконструировать специальный тип, информация из которого попадёт в сообщение об ошибке. Например, этот тип можно вернуть из `synonym family` при некорректном наборе аргументов. Или же можно воспользоваться `constraint trick` (см. 3.1.3) и разместить такой тип в качестве посылки в инстансе. Если инстанс подошел и компилятор начал обрабатывать ограничения слева, значит, что-то пошло не так [Maguire, а, глава 12].

```

1 instance (TypeError
2 (Text "Attempting to show a function of type "
3 :<>: Text "" :<>: ShowType (a -> b) :<>: Text ""
4 :$$: Text "Did you forget to apply an argument?"
5)) => Show (a -> b) where
6 show = undefined -- реализация не важна, до исполнения дело не дойдёт

```

### 3.4.12 Вычисление до нормальной формы

Haskell ленив: значение вычисляется только тогда, когда оно понадобилось, и только настолько, насколько понадобилось. Встроенная функция `seq` вычисляет свой первый аргумент до **слабой головной нормальной формы (WHNF)**, то есть до первого конструктора, и не трогает его поля. Иногда этого мало и значение нужно вычислить целиком, до **нормальной формы**:

- чтобы измерить время самого вычисления, а не создания отложенного вычисления;
- чтобы работа была сделана в нужном потоке, а не в том, который первым обратится к результату;
- чтобы исключение возникло там, где стоит его обработчик;
- чтобы не держать в памяти цепочку отложенных вычислений.

Параметрически-полиморфная функция обойти значение целиком не может: способ обхода зависит от типа. Значит, это работа для класса типов<sup>99</sup> [Marlow, 2011, глава 2]:

```
1 class NFData a where
2 rnf :: a -> () -- reduce to normal form

3 instance NFData Int where
4 rnf x = x `seq` ()

5 instance NFData a => NFData [a] where
6 rnf [] = ()
7 rnf (x : xs) = rnf x `seq` rnf xs

8 deepseq :: NFData a => a -> b -> b
9 deepseq x y = rnf x `seq` y

10 force :: NFData a => a -> a
11 force x = x `deepseq` x
```

Разницу между `seq` и `deepseq` легко увидеть на списке, внутри которого спрятано невычислимое значение:

```
1 ghci> xs = [1, undefined, 3] :: [Int]
2 ghci> xs `seq` "ok"
3 "ok"
4 ghci> length xs
5 3
6 ghci> xs `deepseq` "ok"
7 *** Exception: Prelude.undefined
```

Вернёмся к исключениям. Чистый код порождает исключение не там, где он написан, а там, где его результат вычисляется. Поэтому обработчик ловит исключение, только если вычисление произошло внутри него. Функция `evaluate` вычисляет значение лишь до первого конструктора, и исключение, спрятанное глубже, уйдёт мимо обработчика:

```
1 safe, safe' :: [Int] -> IO (Either SomeException [Int])
2 safe xs = try (evaluate xs) -- не поймает
3 safe' xs = try (evaluate (force xs)) -- поймает
```

Цена: `rnf` обходит значение целиком, даже если оно уже вычислено, и каждый следующий вызов обходит его заново. Для функций же инстанс вычисляет только до WHNF: заглянуть в замыкание нельзя. Альтернатива — не допускать отложенных вычислений в структуре изначально, объявив её поля строгими<sup>100</sup>.

---

<sup>99</sup><https://hackage.haskell.org/package/deepseq>

<sup>100</sup>Расширение `StrictData` делает строгими все поля в модуле.

Можно заметить, что инстансы `NFData` полностью определяются формой типа: нужно обойти все поля всех конструкторов. Писать такой код руками не приходится, его порождает компилятор (см. 9).

**Упражнение 41** Реализуйте инстанс `NFData` для дерева:

```
1 data Tree a = Leaf | Node (Tree a) a (Tree a)
```

**Упражнение 42** Почему `length xs `seq` ()` не заменяет `rnf xs`? Что именно вычислит каждое из выражений?

## 4 Типы данных

В этой главе собраны некоторые общие знания о типах. Также, мы получим различные эквивалентные представления рекурсивных типов данных (иначе говоря, коллекций). Многие концепции являются частными случаями этого многообразия.

Разделы 4.1, 4.2 в основном следуют [Maguire, а, глава 1].

### 4.1 Вариантность

В этом параграфе мы будем рассматривать тему с точки зрения программирования [Maguire, а, глава 3], не отдавая должного теории категорий. Восполнить пробел можно с помощью замечательной статьи, написанной в жанре пьесы Hinze et al. [2012].

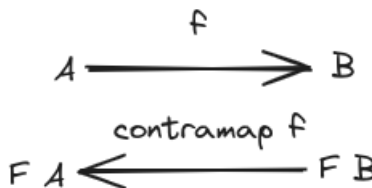
**Ковариантный функтор** — пара из некоторого типового конструктора  $F$  и операции на функциях  $fmap :: (a \rightarrow b) \rightarrow (F\ a \rightarrow F\ b)$ . Плюс законы о том, что  $fmap$  уважает  $id$  и композицию.

```
1 class Functor f where
2 fmap :: (a -> b) -> (f a -> f b)
```



**Контравариантный функтор** — пара из типового конструктора и операции на функциях, разворачивающей стрелку. Плюс соответствующие законы.

```
1 class Contravariant f where
2 contrafmap :: (a -> b) -> (f b -> f a)
```



Типовой конструктор можно объявить ковариантным или контравариантным функтором (или никаким из них) относительно некоторого типового параметра в зависимости от вида

декларации соответствующих конструкторов данных. А именно, от знака позиций, в которых входит этот типовой параметр в тип.

Разовьём интуитивное понимание знаков позиций. Тип **A** входит в положительной позиции в **B** если его значение можно извлечь из **B**. И наоборот, тип **A** входит в отрицательной позиции, если его значение нужно, наоборот, предоставить. Рассмотрим знаки позиций типов в базовых типовых конструкторах:

| Тип               | знак позиции <b>A</b> | знак позиции <b>B</b> |
|-------------------|-----------------------|-----------------------|
| <b>Either A B</b> | +                     | +                     |
| <b>(A, B)</b>     | +                     | +                     |
| <b>A -&gt; B</b>  | —                     | +                     |

Действительно, из суммы и произведения можно извлечь компоненты с помощью паттерн-матчинга, а из стрелки можно получить правый тип апплицируя её к аргументу. В то же время значение типа слева от стрелки нужно предоставить.

На плюс и минус действуют интуитивные алгебраические законы при рассмотрении более сложных типов. Рассмотрим на примере `f :: ((A, B) -> C) -> (D, E)`.

- Плюс на плюс даёт плюс. Действительно, нужно лишь применить две элиминации вместо одной, чтобы получить заветный тип. В нашем примере, чтобы получить **D**, нужно сначала апплицировать функцию, а потом разобрать пару.
- Плюс на минус (и наоборот) даёт минус. Действительно, **C** нам нужно предоставить: `f (\ab -> provideC)`.
- Минус на минус даёт плюс. Пару **(A, B)** нам предоставляют: `f (\ab -> ...)`.

**Упражнение 43** Убедитесь что плюс на минус даёт минус.

Возвращаясь к функторам, если типовой параметр входит в декларацию только в положительных позициях, типовой конструктор можно объявить ковариантным функтором относительно этого параметра. Если в только в отрицательных — контравариантным функтором. Если в обоих, то никаким функтором объявить нельзя. Соответственно, будем называть типовые параметры ковариантными, контравариантными и инвариантными.

**Упражнение 44** Объявите `instance Contravariant F` для `data F a = L (a -> ()) | R Int`.

Таким образом, можно понимать ковариантный функтор как вычисление, результат которого можно пост-обработать, а контравариантный функтор — как вычисление, аргументы которого можно пред-обработать.

Тип от двух положительных параметров можно объявить **бифунктором**:

```
1 class Bifunctor f where
2 bimap :: (a -> c) -> (b -> d) -> f a b -> f c d
```

Тип от двух параметров, положительного и отрицательного, — **профунктором**:

```

1 class Profunctor p where
2 dimap :: (c -> a) -> (b -> d) -> p a b -> p c d

```

Профункторы являются некоторыми обобщениями функциональной стрелки. Например, если у нас есть SQL запрос, который по данным возвращает результат, его можно объявить профунктором с семантикой — добавить пред-обработку входных данных и пост-обработку выходных:

```

1 dimap serialize deserialize (query :: Sql Text Text) :: Sql Age [User]

```

Также понятие вариантности часто встречается в объектно ориентированных языках (да и вообще в теории подтипизации) для обозначения возможности дополнить отношение подтипизации на полиморфные типы.

Действительно, **отношение подтипизации**  $B <: A$  говорит о том, что значение типа  $B$  безопасно использовать в позиции, где ожидается значение типа  $A$ . Иначе говоря, существует функция  $upcast :: B \rightarrow A$ . Если типовой конструктор  $F$  а ковариантен относительно параметра  $a$ , то по  $upcast$  найдётся  $upcast' :: F B \rightarrow F A$ . То есть отношение подтипизации также автоматически включает  $F B <: F A$ . Контравариантный случай аналогично.

**Упражнение 45** Убедитесь в вашем любимом языке с подтипизацией и поддержкой вариантности, что минус на минус даёт плюс.

## 4.2 Изоморфизм

Пусть нам нужно спроектировать функцию или модель данных. Мы начинаем с декларации типа, как её выбрать и из каких вариантов? Для начала поймём, когда два типа взаимозаменяемы, для этого рассмотрим понятия изоморфизма.

Два типа  $A$  и  $B$  называются **изоморфными** (обозначают  $A \cong B$ ) тогда и только тогда, когда существует такая пара функций  $to :: A \rightarrow B$  и  $from :: B \rightarrow A$ , что<sup>101</sup>

```

1 to . from = id
2 from . to = id

```

Иначе говоря, между обитателями таких типов можно установить взаимно-однозначное соответствие. Легко понять, что со смысловой точки зрения не принципиально, какой из изоморфных типов использовать — их можно заменять друг на друга, добавляя вызовы функций перехода. Такие два типа заключают в себе одинаковое “количество информации”. Например, типы `Bool` и `Maybe ()` в этом смысле совершенно взаимозаменяемы. Покажем это, предъявив пару взаимнообратных функций<sup>102</sup>:

<sup>101</sup>Под равенством термов можно понимать разное, например,  $\alpha\beta\gamma$ -эквивалентность. Мы будем пользоваться **экстенциональным равенством** для функций — две функции равны, когда равны их результаты на всех входах. <https://ncatlab.org/nlab/show/function+extensionality>

<sup>102</sup>Нужно не забыть показать взаимнообратность функций, но это делается тривиально перебором входов (может быть с помощью индукции) и редукцией.

```

1 to :: Bool -> Maybe ()
2 to b = if b then Just () else Nothing

3 from :: Maybe () -> Bool
4 from m = case m of Nothing -> False; Just () -> True

```

Несмотря на смысловую взаимозаменяемость, для кодирования информации о том, передал ли пользователь программе определённый флаг, мы, скорее всего, воспользуемся типом `Bool` ввиду нефункциональных соображений о читаемости кода. Аналогично можно рассматривать соображения эффективности.

С категорным взглядом на происходящее можно ознакомиться в Hinze and James [2010]. Мы же придерживаемся теоретико-множественной интерпретации типов.

#### 4.2.1 Кардинальность: суммы, произведения, экспоненты

Типы можно воспринимать как синтаксис для записи множеств, а населяющие их термы — как синтаксические записи элементов этих множеств. Так терм `(True, False)` — запись элемента множества пар, записываемого в синтаксисе типов как `(Bool, Bool)` (вместо математического  $\mathbb{B} \times \mathbb{B}$ ). Или же терм `\x -> x + 1` является записью функции прибавляющей единицу из множества функций над целыми числами, записываемого как `Integer -> Integer` (вместо математического  $\mathbb{Z} \rightarrow \mathbb{Z}$ ).

Заметим, что два типа изоморфны, если соответствующие им множества имеют одинаковое количество элементов. Более того, таких изоморфизмов  $n!$  в случае конечности множеств. Научимся определять количество таких элементов. С помощью  $|\cdot|$  будем записывать **кардинальность** типа — количество элементов в соответствующем множестве.

| Тип и его декларация                         | кардинальность |
|----------------------------------------------|----------------|
| <code>data Void</code>                       | 0              |
| <code>data Unit = Unit</code> <sup>103</sup> | 1              |
| <code>data Bool = False   True</code>        | 2              |

Идея алгебраических типов данных в том, что сложные типы можно строить из простых с помощью операции  $+$  (“или”) и операции  $\times$  (“и”)<sup>104</sup>:

| Тип                                             | кардинальность   |
|-------------------------------------------------|------------------|
| <code>data Either a b = Left a   Right b</code> | $ a  +  b $      |
| <code>data Pair a b = Pair a b</code>           | $ a  \times  b $ |

Посчитаем количество обитателей различных типов (вы можете убедиться в справедливости заключения перебрав все термы вручную):

- $|\text{Either Unit (Either Bool Bool)}| = |\text{Unit}| + (|\text{Bool}| + |\text{Bool}|) = 5.$

<sup>103</sup>`Unit` записывается в Haskell с помощью специального синтаксиса `()`, означающем как бы пустой кортеж.

<sup>104</sup><https://stanford-cs242.github.io/f18/lectures/02-2-algebraic-data-types.html>

- `Pair (Either Bool Unit) (Pair Unit Void) | = 0` — тип `Void` не населён, как и кортеж, его включающий.
- Если `data Example = FirstAlternative Bool | AnotherOne Unit Bool Bool`, то `|Example| = |Bool| + |Unit| * |Bool| * |Bool| = 2 + 1 * 2 * 2 = 6`.

Функциональную стрелку называют экспоненциальным типом. Действительно, комбинаторно количество обитателей `A -> B` вычисляется как

$$|A \rightarrow B| = |B|^{|A|}$$

Так как же проектировать типы? Тому есть несколько соображений:

- В типе должно быть не меньше элементов, чем в предметной области, чтобы все необходимые объекты были представимы.
- В типе должно быть как можно меньше элементов, которых нет в предметной области, чтобы пространство ошибок было минимальным.
- Далее среди изоморфных типов выбирается оптимальный исходя из нефункциональных требований.

Прежде чем работать с некоторым объектом предметной области, информацию о нём, в соответствии со вторым правилом, следует привести в максимально структурное представление, дающее наибольшее количество гарантий<sup>105</sup>.

#### 4.2.2 Алгебраическое представление типа

Как мы увидели выше, чтобы показать наличие изоморфизма между двумя типами можно либо предъявить пару взаимнообратных функций, либо показать, что кардинальности этих двух типов совпадают. В этом разделе мы научимся сопоставлять типу некоторую алгебраическую запись, отражающую его структуру и кардинальность. Так, мы сможем синтаксическими преобразованиями формул получать эквивалентные записи, из которых будем восстанавливать типы, заведомо изоморфные данному<sup>106</sup>.

В основу алгебраического представления положим вычисление кардинальности типов. Фактически мы забываем несущественную для изоморфизма информацию об именах конструкторов данных и конструкторов типов, то есть переходим к структурной типизации.

| Тип                                             | алгебраическая формула  |
|-------------------------------------------------|-------------------------|
| <code>data Void</code>                          | 0                       |
| <code>data Unit = Unit</code>                   | 1                       |
| <code>data Bool = False   True</code>           | 1 + 1 (обозначим как 2) |
| <code>data Maybe a = Nothing   Just a</code>    | 1 + a                   |
| <code>data Either a b = Left a   Right b</code> | a + b                   |
| <code>data Pair a b = Pair a b</code>           | a × b                   |
| <code>a -&gt; b</code>                          | b <sup>a</sup>          |

<sup>105</sup><https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/>

<sup>106</sup><https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types>

$$\begin{array}{c}
\text{sums} \\
X + 0 \cong X \quad X + Y \cong Y + X \\
(X + Y) + Z \cong X + (Y + Z) \\
\text{products} \\
X \times 1 \cong X \quad 0 \times X \cong 0 \quad X \times Y \cong Y \times X \\
(X \times Y) \times Z \cong X \times (Y \times Z) \\
\text{product over sum} \\
(X + Y) \times Z \cong (X \times Z) + (Y \times Z) \\
\text{exponentials} \\
1^X \cong 1 \quad X^0 \cong 1 \quad X^1 \cong X \\
Z^{X+Y} \cong Z^X \times Z^Y \quad (Z^Y)^X \cong Z^{X \times Y} \\
\text{exponential over product} \\
(Y \times Z)^X \cong Y^X \times Z^X
\end{array}$$

Рис. 15: Законы школьной алгебры ностальгии ради Hinze and James [2010].

**Упражнение 46** Запишите в алгебраическом виде следующий тип:

```
1 data T a b = Undefined | Defined a (a -> b)
```

В качестве отношения эквивалентности, будем использовать изоморфизм соответствующих типов. В такой интерпретации, классические свойства алгебраических операций сохраняются (рис. 15). Действительно, например:

```
1 -- (cb)a ≅ ca×b
2 to :: (a -> b -> c) -> (a, b) -> c
3 to = uncurry
4 from :: ((a, b) -> c) -> a -> b -> c
5 from = curry
```

**Упражнение 47** Покажите, что  $(a + b) + c \cong a + (b + c)$ .

**Упражнение 48** Покажите, что  $c^{a+b} \cong c^a \times c^b$ .

Интересным наблюдением может быть то, что функции можно использовать как структуры данных, в соответствие с изоморфизмом  $c^{a+b} \cong c^a \times c^b$ . Действительно, в таком случае аргумент функции выступает индексом (его кардинальность должна совпадать с размером коллекции).

```
1 -- a × a ≅ a2
2 get :: (a, a) -> (Bool -> a)
3 get (x, y) idx = if idx then x else y
4 tabulate :: (Bool -> a) -> (a, a)
5 tabulate f = (f True, f False)
```

**Каноническим представлением типа (canonical representation)** называют сумму произведений типов:

$$\sum_i \prod_j t_{ij}$$

Каноническое представление является своего рода нормальной формой, в которой можно записывать алгебраические типы (любой алгебраический тип можно по правилам привести к ней). Легко узнать в нём вид `data` деклараций в Haskell.

Удивительно, но даже производная алгебраического типа имеет вполне понятную семантику. Это контекст zipper, структуры данных наподобие итератора, позволяющей навигироваться по структуре данных произвольной формы Huet [1997], McBride [2001], Abbott et al. [2003].

### 4.3 Рекурсивные типы

Рекурсивные типы, наподобие рекурсивным термам, могут включать себя в своих определениях. Иначе говоря, рекурсивный тип изоморфен какому-то другому типу, в который он сам входит как подтип.

```
1 fac n = if n <= 1 then 1 else n * fac (n - 1)
2 data Nat = Zero | Suc Nat
```

Мы в основном посмотрим на рекурсивные типы с практической точки зрения. Однако, их формальное теоретико-типовое описание, теоретико-категорная и теоретико-множественная интерпретации представляют отдельный интерес [Pierce, 2002, часть 4].

#### 4.3.1 Просто список

Рассмотрим классический функциональный список. Список это либо коллекция из нуля элементов, либо одного, либо двух. . . Алгебраически это запишется следующим образом:

$$L = 1 + a + a^2 + a^3 + \dots$$

Фактически получили тип с бесконечной записью. Поработаем с ним как с формальным рядом. Вынесем  $a$  за скобки:

$$L = 1 + a \times (1 + a + a^2 + \dots)$$

Заметим, что выражение в скобках представляет собой список, получим такое рекурсивное уравнение<sup>107</sup>:

$$L = 1 + a \times L$$

Легко видеть, что это на самом деле знакомое нам определение списка из Haskell:

---

<sup>107</sup>Либо можно получить то же самое, заметив, что мы имеем дело с рядом Тейлора <https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types>.

```
1 data List a = Nil | Cons a (List a)
```

Получим конечное нерекурсивное представление типа  $L$ . Фактически, нам нужно получить тип изоморфный типу, включающему в себя исходный:

$$L \cong 1 + a \times L$$

Расширим язык типов абстракцией (полиморфизмом) и решим полученное рекурсивное уравнение в стиле  $\lambda$ -исчисления, с помощью некоторого комбинатора неподвижной точки:

$$L = FIX \lambda r. 1 + a \times r$$

Закодируем это на Haskell. В качестве комбинатора рекурсии возьмём `data FixList a`:

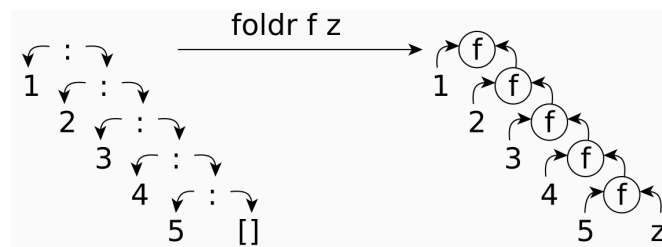
```
1 type Shape a r = Either () (a, r) -- λar.1 + a × r
2 data FixList a = In (Shape a (FixList a))
3 -- FixList a ≅ Shape a (FixList a) ≅ Shape a (Shape a (FixList a)) ≅ ...

4 example :: FixList Int -- [1, 2, 3]
5 example = In (Right (1, In (Right (2, In (Right 3, Left ()))))))
```

Таким образом, мы разделили определение списка на две части: одна отвечает за форму типа, другая — за рекурсию<sup>108</sup>. Форму можно переиспользовать, в определении свёртки:

```
1 foldr :: (Either () (a, r) -> r) -> FixList a -> r
2 foldr phi (FixList shape) = case shape of
3 Left () -> phi (Left ())
4 Right (x, xs) -> phi (Right (x, foldr phi xs))
5 -- сравните с классическим определением
6 foldr :: r -> (a -> r -> r) -> [a] -> r
7 foldr ini f list = case list of
8 [] -> ini
9 x:xs -> f x (foldr ini f xs)
```

Действительно, свёртка в общем смысле позволяет заменить каждый конструктор данных в дереве на некоторую функцию. В результате получается вычисление, имеющее доступ ко всему содержимому структуры данных, и возвращающее результат агрегации:



<sup>108</sup>Техника абстрагирования по “рекурсивной ссылке на себя” называется **открытой рекурсией**. Так, мы предоставляем пользователю больше контроля. Например, на уровне термов комбинатор рекурсии может подсчитывать количество рекурсивных вызовов и оборвать вычисление при превышении какого-то лимита.

То есть количество и типы аргументов свёртки зависят от “формы” сворачиваемого типа. В случае списка требуется нольарная функция вместо `Nil` и бинарная вместо `Cons`. Мы же эти две функции представляем в как одну с помощью типа-формы. Сравните:

```
1 foldr 0 (\e r -> e + r)
2 foldr (\case Left () -> 0; Right (e, r) -> e + r)
```

### 4.3.2 Неподвижная точка функтора

Абстрагируем `FixList` по типу-форме:

```
1 newtype Fix :: (Type -> Type) -> Type
2 newtype Fix f = In { out :: f (Fix f) }

3 data ListF a r = Nil | Cons a r
4 type List a = Fix (ListF a)
```

**Упражнение 49** Какие типы будут у `In` и `out`?

Можно показать, что  $[a] \cong \text{List } a$ :

```
1 to :: [a] -> List a
2 to = \case
3 [] -> In Nil
4 x:xs -> In $ Cons x (to xs)

5 from :: List a -> [a]
6 from (In shape) = case shape of
7 Nil -> []
8 Cons x xs -> x : from xs
```

Тип формы можно сделать функтором по последнему параметру. Это позволит нам в дальнейшем заменять вхождения поддеревьев на что-то полезное.

```
1 instance Functor (ListF a) where
2 fmap :: (rec -> other) -> ListF a rec -> ListF a other
3 fmap f = \case
4 Nil -> Nil
5 Cons x xs -> Cons x (f xs)
```

**Упражнение 50** Выразите следующее дерево как неподвижную точку функтора. Объявите инстанс функтора для типа-формы.

```
1 data Tree a = Leaf a | Node a (Tree a) (Tree a)
```

### 4.3.3 Схемы рекурсии

Подобно тому, как структурное императивное программирование, в сравнении с беспорядочным использованием `goto`, помогает рассуждать о программах, так **схемы рекурсии** позволяют алгебраически описывать свойства рекурсивных функций, в отличие от “неструктурной” рекурсии<sup>109</sup> Meijer et al. [1991], Meijer and Hutton [1995].

Глобальная идея состояла в том, чтобы формулировать требуемые свойства и вычислять нужные программы подобно тому, как математики находят решения дифференциальных уравнений. Однако, эта идея не нашла нужного развития и применения (однако, владеть алгебраическим подходом в целом полезно Maguire [b]). Тем не менее это знание, с одной стороны, даёт более глубокое понимание рекурсии, с другой, пригодится нам для рассмотрения и решения главной проблемы этого курса — *expression problem*.

Универсальная свёртка называется **катаморфизмом**. Катаморфизм сначала рекурсивно сворачивает поддеревья, добираясь к ним с помощью `fmap`, и оставляет результаты свёртки типа `f a`, где `f` — какой-то функтор формы. Далее применяется функция типа `f a -> a`, которая определяет, как свернуть один слой рекурсивной структуры, когда поддеревья уже свёрнуты.

```
1 cata :: Functor f => (f a -> a) -> Fix f -> a
2 cata phi = phi . fmap (cata phi) . out
```

Иначе говоря, `cata` заменяет все конструкторы `In` на функцию `phi`, которая содержит информацию о том, как реагировать на разные варианты конструкторов в дереве:

```
1 example = In (Cons 1 (In (Cons 2 (In (Cons 3 (In Nil)))))
2 cata phi example =β phi (Cons 1 (phi (Cons 2 (phi (Cons 3 (phi Nil)))))
```

Например, сумма списка будет выглядеть следующим образом (рис. 16):

```
1 sum :: List Int -> Int
2 sum = cata \case
3 Nil -> 0
4 Cons x result -> x + result
```

Функцию `f a -> a` называют **f-алгеброй**. Действительно, если в качестве функтора `f` взять сигнатуру алгебры, а в качестве `a` носитель, то `f`-алгебра будет задавать некоторую интерпретацию сигнатуры:

```
1 data MonoidSig carrier = Mempty | Mappend carrier carrier
2 interpretSig :: MonoidSig Int -> Int
3 interpretSig = \case Mempty -> 0; Mappend l r -> l + r
```

**Упражнение 51** С помощью какой алгебры можно скопировать структуру данных?

<sup>109</sup><https://reasonablypolymorphic.com/blog/recursion-schemes/index.html>

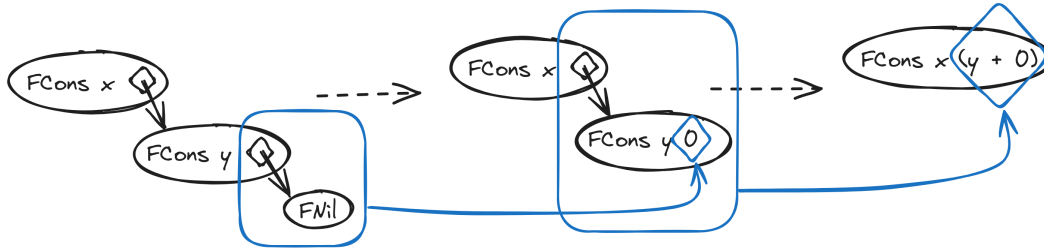


Рис. 16: Катаморфизм переиспользует вхождения рекурсивных поддеревьев, чтобы хранить результаты свёртки соответствующих поддеревьев.

### Упражнение 52 С помощью какой алгебры можно распечатать список в строку?

В противоположность универсальной свёртке, можно построить **анаморфизм** — универсальную развёртку (аналогично `unfold` для списка). Здесь *f*-коалгебра (стрелочка в обратную сторону) показывает, как из некоторого значения-зерна получить один слой структуры данных, где вместо рекурсивных ссылок будут зёрнышки, из которых потом прорастут поддеревья. Аноморфизм как раз сначала разворачивает один слой, а потом рекурсивно разворачивает все поддеревья:

```
1 ana :: Functor f => (s -> f s) -> s -> Fix f
2 ana psi = In . fmap (ana psi) . psi
3 -- сравните с классическим определением развёртки списка
4 unfoldr :: (s -> Maybe (a, s)) -> s -> [a]
```

### Упражнение 53 Реализуйте аноморфизм, строящий список от 0 до заданного *n*.

Также вводят **гиломорфизм (hylomorphism)**, которые позволяют описать произвольное рекурсивное вычисление. Гиломорфизм задаётся как композиция аноморфизма и катаморфизма. Сначала аноморфизм строит явное дерево, представляющее собой дерево вызовов некоторой рекурсивной процедуры, затем катаморфизм сворачивает его в результат.

```
1 hylo :: Functor f => (a -> f a) -> (f b -> b) -> a -> b
2 hylo psi phi = cata phi . ana psi
```

Например, вычисление факториала может быть реализовано следующим образом:

```
1 fac n = hylo
2 (\n -> if n > 0 then Cons n (n - 1) else Nil)
3 (\case Nil -> 1; Cons n acc -> n * acc)
```

Можно ввести ещё много различных рекурсивных схем<sup>110</sup> и описать их свойства. Однако, мы пока остановимся.

Интересный факт: катаморфизм представляет собой теоретико-множественный принцип индукции, а аноморфизм — коиндукции, если в качестве вложения множеств взять функцию [Pierce, 2002, глава 21].

<sup>110</sup>[https://wiki.haskell.org/Zygomorphic\\_prepromorphisms](https://wiki.haskell.org/Zygomorphic_prepromorphisms)

## 4.4 Всё через свёртки

Оказывается, что с помощью катаморфизма можно получить изоморфизм между структурами данных и их свёртками:  $\text{Fix } f \cong \text{forall } a . (f \ a \rightarrow a) \rightarrow a$ .

```
1 to :: Functor f => Fix f -> (forall a . (f a -> a) -> a)
2 to = flip cata

3 from :: (forall a . (f a -> a) -> a) -> Fix f
4 from g = g In
```

Например, следующие два списка эквивалентны (все конструкторы `In` заменяем на данную алгебру):

```
1 data ListF elem rec = Nil | Cons elem rec

2 xs1 :: Fix (ListF Int)
3 xs1 = In (Cons 1 (In (Cons 2 (In (Cons 3 (In Nil))))))

4 xs2 :: (ListF Int a -> a) -> a
5 xs2 = \alg -> alg (Cons 1 (alg (Cons 2 (alg (Cons 3 (alg Nil))))))

6 ghci> xs2 @Int \case Nil -> 0; Cons x acc -> x + acc
7 6
```

Теперь избавимся от функтора формы. Это нерекурсивный тип, который можно представить в канонической форме (см. 4.2.2):

$$f \ a \cong \sum_i \prod_j (t_{ij} \ a)$$

Тогда алгебра может быть записана следующим образом:

$$f \ a \rightarrow a \cong a^{\sum_i \prod_j (t_{ij} \ a)} \cong \prod_i a^{\prod_j (t_{ij} \ a)}$$

Остались произведения, от которых можно избавиться с помощью каррирования, и получить **Church encoded** структуры данных. Например, для списка имеем:

$$\begin{aligned} (\text{ListF } elem \ a \rightarrow a) \rightarrow a &\cong a^{1 + elem \times a} \cong a^a \times a^{elem \times a} \cong (a^a)^{elem} \\ &\cong a \rightarrow (elem \rightarrow a \rightarrow a) \rightarrow a \end{aligned}$$

Мы получили не что иное, как список Чёрча<sup>111112</sup>. Структуру данных, без единого конструктора!<sup>113</sup> Перепишем знакомый нам список ещё раз:

<sup>111</sup>[https://en.wikipedia.org/wiki/Church\\_encoding](https://en.wikipedia.org/wiki/Church_encoding)

<sup>112</sup><https://okmij.org/ftp/tagless-final/course/Boehm-Berarducci.html>

<sup>113</sup>На самом деле в этом нет ничего удивительного, если вспомнить, что функции первого класса представляются как замыкания, содержащие данные. Мы получили тот же односвязный список, только на замыканиях.

```

1 xs3 :: a -> (Int -> a -> a) -> a
2 xs3 = \ini f -> f 1 (f 2 (f 3 ini))

```

**Упражнение 54** *Какая знакомая вам стандартная функция работы со списками по `data` списку возвращает список Чёрча?*

Попробуем интуитивно понять, что это всё значит. Заметим, что список Чёрча принимает функции, соответствующие веткам паттерн-матчинга или аргументам сворачивающей функции. Вместо конструкторов же сразу вызываются соответствующие функции. То есть, вместо того, чтобы создать структуру данных и доставить её к месту деконструирования (паттерн-матчингу), мы *доставляем место деконструирования к месту конструирования* и оказывается, что ничего конструировать по итогу и не надо.

Мы уже работали с обратным процессом, дефункционализацией, когда функции первого класса превращались в вызовы конструкторов, а их тела — в ветки паттерн-матчинга 3.1.7. Можно заметить, что тут мы имеем дело с обратным процессом, **рефункционализацией**, когда вместо вызовов конструкторов сразу вызывается соответствующий интерпретирующий код Danvy and Millikin [2009].

С технической точки зрения, мы поняли, что условное ветвление (if, паттерн-матчинг, конструкция switch) и виртуальные вызовы (вызовы замыканий, методов на интерфейсах) взаимозаменяемы.

Стоит также отметить, что изоморфизм  $\text{Fix } f \simeq \text{forall } d . (f \ d \rightarrow d) \rightarrow d$  является обобщением изоморфизма  $a \simeq \text{forall } r . (a \rightarrow r) \rightarrow r$  (при  $f = \text{Const } a$ ), который лежит в основе CPS-трансформации, рассматриваемой нами далее в главе 6.

#### 4.4.1 Deforestation & list fusion

В функциональном программировании мы строим новые функции путём композирования имеющихся, более простых, функций. Этот подход позволяет переиспользовать реализованную функциональность, снижая сложность кода и вероятность ошибок. Однако, он может приводить к излишним накладным расходам на аллокацию и деконструирование промежуточных структур данных.

Например, сравните следующие две реализации. Первая предпочтительна с точки зрения качества кода, однако она порождает промежуточный список во время работы:

```

1 all p xs = and (map p xs)
2 -- или fused версия
3 all p [] = True
4 all p (x:xs) = p x && all p xs

```

Очевидно, это задача оптимизирующего компилятора превращать хороший код с абстракциями в быстрый код. Оптимизация, избавляющая программы от (промежуточных) структур данных (деревьев) называется **дефорестацией (deforestation)**. В результате две функции, как говорят, “сплавляются” — fuse.

```

 append (append xs ys) zs
 transforms to

 h0 xs ys zs
 where
 h0 xs ys zs = case xs of
 Nil : h1 ys zs
 Cons x xs : Cons x (h0 xs ys zs)
 h1 ys zs = case ys of
 Nil : zs
 Cons y ys : Cons y (h1 ys zs)

```

---

```

 flip (flip zt)
 transforms to

 h0 zt
 where
 h0 zt = case zt of
 Leaf z : Leaf z
 Branch xt yt : Branch (h0 xt) (h0 yt)

```

Рис. 17: Примеры работы дефорестирующего алгоритма Wadler [1988].

Термин и первый дефорестирующий алгоритм был предложен Philip Wadler [1988], он основан на нескольких простых правилах переписывания, нацеленных получить ситуацию `case K args of ...`, и агрессивном инлайнинге (см. примеры работы на рис. 17). Однако, этот алгоритм может приводить к экспоненциальному разбуханию кода и имеет шанс не завершиться при наличии рекурсивных вызовов<sup>114</sup>. Чтобы алгоритм завершался программы должны быть написаны в некоторой строгой форме под названием *treeless*.

Сфокусируемся на списках и рассмотрим более практичные решения<sup>115</sup>. Первым очевидным решением было бы для каждой пары функций, работающих со списками, добавить специальное правило переписывания (см. 3.1.6), дефорестирующее с помощью алгебраических свойств списочных трансформаций. Однако, таких правил будет экспоненциально много.

<sup>1</sup> {-# RULES

<sup>114</sup> Дефорестация является частным случаем суперкомпиляции Романенко, которая в свою очередь является обобщением большого количества компиляторных оптимизаций.

<sup>115</sup> <https://markkarpov.com/tutorial/ghc-optimization-and-fusion.html>

|                                                                                                                                                                                                                                             |                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1  std::variant&lt;Msg1, Msg2&gt; 2      deserialize(bytes bs) { 3          if (...) { 4              return std::variant{Msg1(...)}; 5          } else { 6              return std::variant{Msg2(...)}; 7          } 8      } </pre> | <pre> 1  template&lt;class Impl&gt; 2  auto deserialize(bytes bs) { 3      if (...) { 4          return Impl::processMsg1(...); 5      } else { 6          return Impl::processMsg2(...); 7      } 8  } </pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Рис. 18: Ручная дефорестация в C++.

```

2 "map/map" forall f g xs. map f (map g xs) = map (f . g) xs
3 #-}

```

Современная техника дефорестирования в Haskell — fold/build list fusion Gill et al. [1993] — вместо использования множества алгебраических правил, определяет универсальный способ конструирования и деконструирования списка. Деконструировать список будем с помощью `foldr`. Конструировать будем с помощью функции `build`:

```

1 build :: (forall b . (a -> b -> b) -> b -> b) -> [a]
2 build g = g (:) []

```

Например, список `[1, 2, 3]` и функция `map` теперь запишутся следующим образом<sup>116</sup>:

```

1 list123 :: [Int]
2 list123 = build \s z -> s 1 (s 2 (s 3 z))

3 map :: (a -> b) -> [a] -> [b]
4 map f xs = build \s z -> foldr (\x acc -> s (f x) acc) z xs

```

Можно заметить, что на вход `build` передаётся список Чёрча, отсюда не удивителен закон:

```

1 foldr f ini (build g) ≡ g f ini

```

Таким образом, мы избавились от конструирования списков путём замены вызовов конструкторов на вызовы сворачивающих функций.

Дефорестацию также можно производить вручную (см. 18).

#### 4.4.2 Visitor pattern

Рассмотрим некоторое дерево и его свёртку:

```

1 data Tree a = Leaf | Node a [Tree a]
2 foldTree :: Tree a -> r -> (a -> [r] -> r) -> r

```

<sup>116</sup>В стандартной библиотеке Haskell функции работы со списками написаны нормально, но рядом написаны правила RULES (см. 3.1.6), которые подменяют их реализацию на fold/build.

Перепишем и переименуем:

```
1 data Visitor a r = Visitor { onLeaf :: r, onNode :: a -> [r] -> r }
2 visitTree :: Tree a -> Visitor a r -> r
```

Чтобы это ещё более выглядело в ООП стиле, само дерево должно задаваться свёрткой (как бы интерфейсом с функцией `visit`), а разные вершины — конкретными её реализациями (объектами-наследниками):

```
1 data Tree a = Tree { visit :: forall r . Visitor a r -> r }

2 leaf :: Tree a
3 leaf = Tree { visit = \Visitor{onLeaf} -> onLeaf }

4 node :: a -> [Tree a] -> Tree a
5 node x ts = Tree { visit = \v@Visitor{onNode} -> onNode x (map (`visit` v) ts) }
```

Для наглядности, покажем этот код ещё и на Kotlin:

```
1 interface Visitor<a, r> {
2 fun onLeaf(): r
3 fun onNode(x: a, subtrees: List<r>): r
4 }

5 interface Tree<a> {
6 fun <r> visit(visitor: Visitor<a, r>): r
7 }

8 class Leaf : Tree<Nothing> {
9 override fun <r> visit(visitor: Visitor<Nothing, r>): r = visitor.onLeaf()
10 }

11 class Node<a>(val value: a, val subtrees: List<Tree<a>>) : Tree<a> {
12 override fun <r> visit(visitor: Visitor<a, r>): r =
13 visitor.onNode(value, subtrees.map { it -> it.visit(visitor) })
14 }
```

## 4.5 Всё через развёртку

Вспомним, что существует универсальная развёртка — анаморфизм, которая по генерирующей процедуре позволяет получить целую структуру данных (см. 4.3.3).

```
1 ana :: Functor f => forall s . (s -> f s) -> s -> Fix f
2 ana psi = In . fmap (ana psi) . psi
```

Приведём функцию `ana` к типу вида `A -> B`, чтобы затем проще было исследовать стрелку `B -> A`. Для этого сделаем `uncurry` и перенесём квантор налево от стрелки (он изменится на противоположный):

```
1 ana :: Functor f => (exists s . (s, s -> f s)) -> Fix f
```

Закодируем квантор существования с помощью нового типа (см. 3.4.2) и перепишем анаморфизм на работу с ним:

```
1 data Box f where
2 -- exists s. (s, s -> f s)
3 Box :: forall s . s -> (s -> f s) -> Box

4 ana' :: Functor f => Box f -> Fix f
5 ana' (Box currSeed psi) =
6 In $ (\nextSeed -> ana' (Box nextSeed psi)) <$> psi currSeed
```

Теперь построим изоморфизм между структурами данных и их тривиальными развёртками, возвращающими каждый раз следующий слой данной структуры,  $\text{Fix } f \simeq \text{Box } f$ :

```
1 to :: Fix f -> Box f
2 to x = Box x out

3 from :: Functor f => Box f -> Fix f
4 from = ana'
```

Таким образом, мы получили свидетельство того, что любую рекурсивную структуру данных можно хотя бы тривиальным образом представить как  $\text{Box } f$ . Иногда такое представление называют **co-Church encoding** Gibbons [2008].

Например, бесконечный ленивый список натуральных чисел может быть задан следующим образом. Заметьте, что тут мы *не полагаемся на ленивость Haskell*, а значит можем использовать эту технику и в энергичных языках.

```
1 nats :: Box (ListF Int)
2 nats = Box 0 \curr -> Cons curr (curr + 1)
```

**Упражнение 55** Реализуйте ленивую функцию

$\text{take} :: \text{Int} \rightarrow \text{Box } (\text{ListF Int}) \rightarrow \text{Box } (\text{ListF Int})$ .

Иначе говоря, бесконечные структуры данных представимы как процедуры, лениво генерирующие слой рекурсивной структуры за слоем по запросу.

#### 4.5.1 Абстрактные типы данных

Рассмотрим случай, когда функтор формы представляет собой произведение:

$$s \rightarrow f \ s \cong \left( \prod_i (t_i \ s) \right)^s \cong \prod_i (s \rightarrow t_i \ s)$$

То есть коалгебра эквивалентна кортежу функций. Таким образом, `Box` это не что иное, как *абстрактный тип данных (ADT)*: он включает в себя скрытое состояние неизвестной природы и набор операций для работы с ним Gibbons [2008]. На самом деле мы уже встречали такую конструкцию ранее, когда говорили про экзистенциальные типы 3.4.2 (там мы использовали инстансы классов типов в качестве кортежей функций).

ООП объекты тоже можно рассматривать через коалгебры. Подробнее про соотношение объектов и абстрактных типов данных можно почитать в Cook [2009].

## 4.5.2 Stream fusion

Ранее мы рассматривали `foldr/build list fusion` оптимизацию, элиминирующую промежуточные списки (см. 4.4.1). Однако, эта техника не подходит для многих популярных функций, например, `zip` (нужно корутинить между двумя алгебрами) или `take` (нужно оборвать свёртку в определённый момент).

Позднее была предложена техника использования ко-структуры списка — развёртки или `Stream`<sup>117</sup> Coutts et al. [2007]:

```

1 data ListF a r = Nil | Cons a r
2 type MyStream a = Box (ListF a) -- ∃s . (s -> ListF a s, s)
3 -- ≅
4 data Step a s = Done | Yield a s
5 data Stream a where
6 Stream :: forall s . (s -> Step a s) -> s -> Stream a

7 stream :: [a] -> Stream a
8 stream xs = Stream (\case [] -> Done; x:xs -> Yield x xs) xs

9 unstream :: Stream a -> [a]
10 unstream (Stream next s) = case next s of
11 Done -> []
12 Yield a s' -> a : unstream next s'
```

Идея в том, что теперь *функции работы со стримами не рекурсивны* и легко подвергаются базовым компиляторным дефорестирующим трансформациям в стиле Wadler [1988]. Действительно, вместо рекурсивного вызова мы запоминаем состояние для следующей итерации.

```

1 mapS :: (a -> b) -> Stream a -> Stream b
2 mapS f (Stream next s) = Stream next' s
3 where
4 next' s = case next s of
5 Done -> Done
```

<sup>117</sup><https://markkarpov.com/tutorial/ghc-optimization-and-fusion.html>

```
6 Yield x s' -> Yield (f x) s'
```

Общий пайплайн выглядит следующим образом:

1. Мы пишем все функции работы со списками через стримы:

```
1 map :: (a -> b) -> [a] -> [b]
2 map f = unstream . mapS f . stream
```

2. С помощью RULES (см. 3.1.6) задаём правило переписывания stream/unstream, двойственное foldr/build: stream . unstream = id.

3. Далее помогают обычные компиляторные оптимизации.<sup>118</sup>

Чтобы поддержать нерекурсивную функцию фильтрации, обычно добавляют отдельный вид шагов:

```
1 data Step a s = Done | Skip s | Yield a s

2 filterS :: (a -> Bool) -> Stream a -> Stream a
3 filterS p (Stream next s) = Stream next' s
4 where
5 next' s = case next s of
6 Done -> Done
7 Skip s' -> Skip s'
8 Yield a s' -> if p a then Yield a s' else Skip s'
```

В стандартной библиотеке Haskell всё-таки используется foldr/build fusion, однако существует множество промышленных библиотек стримов [Bragilevsky, глава 14].

Больше про стримы можно почитать у Олега Киселёва<sup>119</sup>.

## 4.6 Вездесущий дуализм

Мы рассмотрели три формы рекурсивных данных:

- **Fix**  $f$  — обычные рекурсивные типы данных, конечные в энергичных языках;
- $\forall a . (f\ a \rightarrow a) \rightarrow a$  — Church encoding, конечные структуры заданные свёртками;
- $\exists s . (s \rightarrow f\ s, s)$  — co-Church encoding, потенциально бесконечные структуры данных, абстракции по данным (работаем через интерфейс над сокрытым представлением).

В этом разделе мы рассмотрим некоторые практические примеры, в которых пространство решений задано дуальными представлениями коллекций.

---

<sup>118</sup>Можно гарантировать полную дефорестацию с помощью staging (см. далее 9) Kiselyov et al. [2017].

<sup>119</sup><https://okmij.org/ftp/Streams.html>

### 4.6.1 Push vs pull streaming

Рассмотрим различные представления списков или, как их иногда называют, потоков событий. Подробнее сравнение можно посмотреть тут [Kiselyov et al., 2017, параграф 3].

Church encoding для списков даёт **push stream**, или **internal iteration**, или **Observer pattern**<sup>120</sup> (в некотором смысле частный случай Visitor pattern 4.4.2), когда мы регистрируем обработчика событий, а коллекция вызывает (push) на нём методы. Примерами push потоков являются Java Stream API<sup>121</sup>, Closure Transducers<sup>122</sup>.

```
1 events.observe(object : Observer<Event> {
2 override fun onComplete() { .. }
3 override fun onNext(elem: Event) { .. }
4 })
```

co-Church encoding для списков даёт **pull stream**, или **external iteration**, или **Iterator pattern**. Чтобы получить следующий элемент, его нужно явно запросить (pull) потребляющему коду. Это представление часто гораздо удобнее с точки зрения использования (см. 4.5.2)<sup>123</sup>, однако сложнее на стороне имплементации, потому что нужно выделить и поддерживать экзистенциальное состояние в коалгебре. Генераторы призваны автоматизировать выделение этого состояния, так как можно заметить, что это continuation вычисления (см. далее 6.4.3).

Нередко работу с потоками называют **реактивным программированием**<sup>124</sup>.

### 4.6.2 Data vs codata

Вспомним функтор-формы для списка:

```
1 data ListF a r = Nil | Cons a r
```

С помощью ключевого слова `data` мы определяем новый алгебраический тип данных через способы его сконструировать. Далее, с помощью pattern-matching мы можем деконструировать значения алгебраического типа данных и воспользоваться хранящейся в нём информацией.

```
1 data List a where
2 List :: ListF a (List a) -> List a

3 fold :: (ListF a r -> r) -> List a -> r
4 fold alg xs = case xs of
5 Fix Nil -> alg Nil
6 Fix (Cons y ys) -> alg (Cons y (fold alg ys))
```

<sup>120</sup><https://learn.microsoft.com/en-us/dotnet/api/system.ioobserver-1?view=net-9.0>

<sup>121</sup><https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html>

<sup>122</sup><https://clojure.org/reference/transducers>

<sup>123</sup><https://github.com/pulldown-cmark/pulldown-cmark/?tab=readme-ov-file#why-a-pull-parser>

<sup>124</sup>(youtube) React 2014 : Erik Meijer - What does it mean to be Reactive?

Однако, также можно определять тип данных через способы его деконструировать с помощью ключевого слова `codata` (в воображаемом языке). Так, у ко-списка можно потребовать показать следующий слой структуры. Сконструировать такой тип данных можно с помощью co-pattern-matching'a, описав как деконструирующая функция должна вести себя на конструирующем терме (`unfold coalg s`)<sup>125</sup>.

```
1 codata CoList a where
2 force :: CoList a -> ListF a (CoList a)

3 unfold :: (s -> ListF a s) -> s -> CoList a
4 force (unfold coalg s) = case coalg s of
5 Nil -> Nil
6 Cons x s' -> Cons x (unfold coalg s')
```

По сути, `codata` представляет собой словарь функций, которые захватывают некоторое состояние, необходимое для генерации следующих слоёв структуры. Так, `codata` можно сравнить с ООП интерфейсами, а co-pattern-matching с анонимной реализацией интерфейса. Предыдущие два примера можно записать, например, в Kotlin следующим образом (форма списка кодируется как `Pair<a, r>?`, убедитесь самостоятельно, что этот тип изоморфен `ListF`).

```
1 class List<a>(val layer: Pair<a, List<a>>?)

2 fun <a, r> fold(alg: (Pair<a, r>?) -> r, xs: List<a>): r =
3 if (xs.layer == null) alg(null)
4 else alg(Pair(xs.layer.first, fold(alg, xs.list.second)))

5 interface CoList<a> { fun force(): Pair<a, CoList<a>>? }

6 fun <a, s> unfold(coalg: (s) -> Pair<a, s>?, ini: s): CoList<a> =
7 object : CoList<a> {
8 override fun force(): Pair<a, CoList<a>>? {
9 val layer = coalg(ini)
10 return if (layer == null) null else
11 Pair(layer.first, unfold(coalg, layer.second))
12 }
13 }
```

Функциональный стиль программирования в основном оперирует напрямую наблюдаемыми данными `data`, а ООП — скрывает данные за интерфейсами `codata`. Однако, обе конструкции имеют свои понятные области применения и ими не стоит пренебрегать вне зависимости от предпочитаемого стиля Downen et al. [2019]<sup>126</sup>.

<sup>125</sup>(youtube) CS410 2017 Lecture 15 Coinduction and Coalgebras

<sup>126</sup><https://reasonablypolymorphic.com/blog/review-codata/>

В языках с проверкой тотальности функций разделяют индуктивные и коиндуктивные определения. Для них используются паттерн-матчинг и ко-паттерн-матчинг, принцип индукции (катаморфизм) и принцип коиндукции (анаморфизм). Индуктивные определения задают конечные структуры данных, проверка тотальности проверяет, что рекурсивные вызовы делаются на структурно меньших подтермах. Коиндуктивные определения задают потенциально бесконечные структуры данных, для них проверяется продуктивность — порождающая функция всегда после рекурсивного вызова породит новый конструктор <sup>127</sup>.

## 4.7 Приложение: категория алгебр

Это факультативный раздел, не являющийся необходимым для понимания курса в дальнейшем. Однако, полезен для понимания часто употребляемой терминологии.

**Категория** — это коллекция объектов и коллекция стрелок. Для каждого объекта  $X$  существует тождественная стрелка, а для каждой пары стрелок существует способ получить их композицию:  $f : Y \rightarrow Z, g : X \rightarrow Y \Rightarrow f \circ g : X \rightarrow Z$ .

Определяют категорию, соответствующую Haskell —  $Hask$ . На самом деле это плохая категория с точки зрения теории, но для наших нестрогих рассуждений подойдёт<sup>128</sup>. Объектами в  $Hask$  являются типы языка Haskell, а морфизмами — термы, задающие функции между соответствующими типами. Тождественный морфизм — `id`, композиция задаётся как `f . g = \x -> f (g x)`.

**Упражнение 56** Как в такой категории представить константы?

**Функтором** называется отображение между категориями, которое объектам одной категории сопоставляет объекты другой, а стрелкам одной — стрелки другой. В Haskell типовые конструкторы задают отображение между объектами, а `fmap` — между стрелками. Функтор должен сохранять тождественный морфизм и композицию.

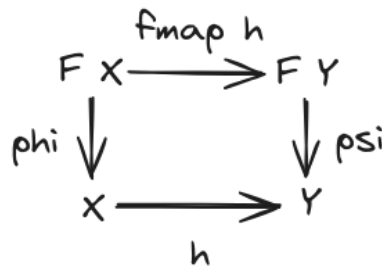
$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ F A & \xrightarrow{\text{fmap } f} & F B \end{array}$$

**Алгеброй** в категории  $C$  называется пара из объекта категории  $X \in \text{Obj}(C)$  и морфизма  $\phi : F X \rightarrow X$ , где  $F$  — функтор. Сам морфизм  $F X \rightarrow X$  называют **f-алгеброй**. Алгебрами в смысле категорий можно описывать алгебры. Так, в качестве объекта  $X$  берём носитель алгебры. В качестве функтора  $F$  — сигнатуру алгебры в виде типа-формы. Тогда морфизмом будет интерпретацией сигнатуры.

<sup>127</sup><https://rocq-prover.org/doc/V8.18.0/refman/language/core/coinductive.html>

<sup>128</sup><https://math.andrej.com/2016/08/06/hask-is-not-a-category/>

**Морфизмом алгебр** называется такой морфизм между носителями  $h : X \rightarrow Y$ , что следующая диаграмма коммутует. Говорят, что **диаграмма коммутует**, если все возможные пути по стрелкам в ней равны.



В морфизме алгебр можно обнаружить знакомые черты гомоморфизмов, то есть операций между носителями, которые “уважают” операции сигнатуры алгебраической теории.

Алгебры над категорией  $\mathcal{C}$  образуют **категорию алгебр**, в которой объектами являются алгебры, а морфизмами — морфизмы алгебр.

**Начальным (инициальным) объектом** категории называется объект, из которого в каждый другой объект существует уникальная стрелка. **Терминальным (финальным) объектом** категории называется объект, в который из каждого другого объекта категории существует уникальная стрелка.

Инициальный и терминальный объекты категории не обязательно присутствуют в единственном экземпляре. Но все инициальные объекты изоморфны друг другу, как и все терминальные.

**Упражнение 57** Приведите начальный и терминальный объекты категории *Hask*.

Рекурсивный тип — это тип, значит ему соответствует объект в категории *Hask*.  $X$  является рекурсивным типом с формой  $F$ , если имеет место следующий изоморфизм:

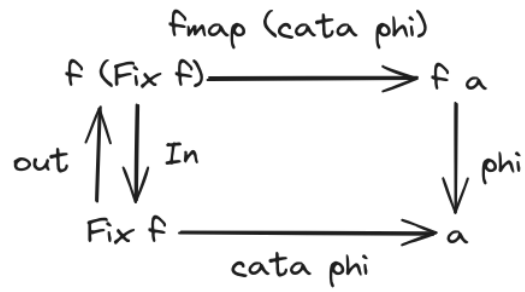
$$X \simeq F X$$

Можно заметить, что свидетель изоморфизма справа налево напоминает  $f$ -алгебру, а слева направо —  $f$ -коалгебру (всё то же самое, только все стрелки в обратную сторону). И действительно, подходящий объект  $X$  должен быть либо начальным объектом категории алгебр, либо терминальным объектом категории коалгебр (с соответствующими морфизмами). Первый вариант соответствует конечным структурам данных, второй — потенциально бесконечным.

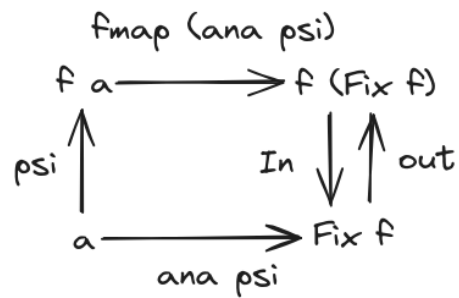
Начальным объектом категории алгебр над *Hask* для функтора  $f$  является следующая алгебра: `(Fix f, In)`<sup>129130</sup>. Действительно, для каждого типа  $a$  и для каждой  $f$ -алгебры  $\text{phi}$  мы можем построить такой морфизм `cata phi :: Fix f -> a`, что следующая диаграмма будет коммутировать:

<sup>129</sup><https://bartoszmilewski.com/2017/02/28/f-algebras/>

<sup>130</sup><https://ncatlab.org/nlab/show/catamorphism>



Покажем, что  $(\text{Fix } f, \text{out})$  является терминальным объектом категории коалгебр (благодаря ленивости Haskell). Аналогично, для каждого объекта  $a$  и  $f$ -коалгебры  $\text{psi}$  найдётся морфизм  $\text{ana psi} :: a \rightarrow \text{Fix } f$ :



## 5 Интерпретаторы

Искусство программирования во многом состоит в умении управлять сложностью, и в рамках данного курса основным инструментом для этого мы будем рассматривать построение языков и интерпретаторов.

### 5.1 Интерпретаторы как основа основ

Мы начнём с обзора роли интерпретаторов в программировании.

#### 5.1.1 Башня интерпретаторов

Самым базовым интерпретатором является процессор, он воплощён физически в железе. Ему на вход подаётся программа на некотором языке, например, x86, он зачитывает команды и превращает их в действия над памятью. Однако, человеку сложно программировать на этом языке, нужен новый язык, инкапсулирующий часть сложности и скрывающий лишние детали.

Чтобы получить новый язык, мы строим программный интерпретатор. **Программный интерпретатор**  $U_M^N$  — это программа на языке  $M^{131}$ , получающая на вход программу на языке  $N$  и вход для неё — данные из  $D$ , и возвращающая результат выполнения этой программы на этих данных:

$$U_M^N : N \times D \rightarrow D$$

Язык реализации интерпретатора  $M$  мы будем называть **мета-языком**, а  $L$  — **определяемым**. Про интерпретатор можно интуитивно думать следующим образом: это понятное мета-языку объяснение того, что значат конструкции определяемого языка. Иными словами, какие инструкции мета-языка нужно исполнить, чтобы получить нужную семантику инструкций определяемого языка.

Например, у нас есть программа  $p_N$  и данные для неё  $d_{in}$ , результат исполнения этой программы  $d_{out}$  можно получить как

$$d_{out} = U_M^N \left( \underbrace{\langle p_N, d_{in} \rangle}_{\in N \times D} \right)$$

Но интерпретатор это тоже программа. Как её запустить? Возьмём наш базовый интерпретатор  $U^{x86}$ , у него нет языка реализации, так как он реализован в железе, а не программно. Возьмём интерпретатор языка ассемблера, реализованный в кодах x86,  $U_{x86}^{Asm}$ , программу на ассемблере  $p_{Asm}$  и вход для неё  $d_{in}$ . Вспомним, что программа — это тоже данные, просто в некотором специальном формате. Тогда результат применения  $p_{Asm}$  на данных мы получим

---

<sup>131</sup>Под языком мы тут понимаем множество программ на этом языке, иначе говоря, множество деревьев определённого вида.

следующим образом:

$$d_{out} = U^{x86} \left( \left\langle \underbrace{U_{x86}^{Asm}}_{\in x86}, \underbrace{\langle p_{Asm}, d_{in} \rangle}_{\in D} \right\rangle \right)$$

Но язык ассемблера, тоже не очень приятен для программирования. Однако, на нем можно уже написать интерпретатор языка посложнее. И так далее. Получаем *башню интерпретаторов*, на вершине находится язык, на котором мы хотим уже решать непосредственно нашу задачу:

$$d_{out} = U^{x86} (\langle U_{x86}^{Asm}, \langle U_{Asm}^C, \langle U_C^{Has}, \langle p_{Has}, d_{in} \rangle \rangle \rangle \rangle)$$

На практике часто язык задают через трансляцию (компиляцию) в другой. Однако, далее мы будем фокусироваться на встроенных языках и понятие интерпретатора будет нам полезнее. Кроме того, существуют универсальные теоретические способы построения компилятора по интерпретатору<sup>132</sup> (см. далее 9), так что для нас они схожи.

## 5.1.2 Интерпретаторы повсюду

Хорошо, мы пришли к языку нашего сердца (Хаскеллу), почему же мы продолжаем говорить об интерпретаторах? Потому что для решения конкретных бизнес-задач прикладные языки всё ещё слишком церемониальны — программисту приходится думать о большом количестве вещей, нерелевантных его предметной области и решаемой задаче. Сложность — главный враг программиста, потому что ресурсы человеческого мозга несопоставимы со сложностью реальности, которую приходится описываться в программах. Таким образом, в работе постоянно приходится описывать новые языки, наиболее подходящие для решения конкретных прикладных задач. А новые языки мы задаём с помощью интерпретаторов.

Как выглядит классический рекурсивный интерпретатор? Он получает программу в виде некоторого дерева и рекурсивно обходит его, считая результаты поддеревьев. Когда он посещает вершину дерева, он определяет её тип и понимает, какие действия нужно исполнить. То есть тип вершины диспатчит, навигирует, исполнение интерпретатора на нужный код. Так, интерпретатор простой языка выражений имеет следующий вид:

```
1 eval :: Expr -> Int
2 eval prog = case prog of
3 Const x -> x
4 Plus l r -> eval l + eval r
```

Видно, что это похоже, например, на работу утилиты командной строки — разбираем аргументы, определяем, что и как нужно сделать, делаем. Как ни странно, философия Unix, в частности, заключается в построении маленьких языков (утилит с текстовым API), решающих хорошо одну задачу Bentley [1986]. Ещё это похоже на обработку запроса web-сервером — определяем роут на которую пришел запрос, выполняем соответствующее действие. То есть

<sup>132</sup>[https://en.wikipedia.org/wiki/Partial\\_evaluation](https://en.wikipedia.org/wiki/Partial_evaluation)

не так редко мы в реальной жизни пишем интерпретаторы. Мы просто не видим, что то, что мы пишем — это на самом деле интерпретатор некоторого языка. В общем случае, свёртку структуры данных уже можно рассматривать как интерпретацию Gibbons and Wu [2014].

Более того, как мы убедимся в дальнейшем 5.4, написание любой функции — это уже задание нового языка. Вот был язык, в котором нельзя было добавить пользователя в приложение. Написали функцию `registerUser` — появилась новая команда в языке — добавить пользователя. Далее мы формально покажем, то такой способ эквивалентен добавлению новой ноды в синтаксическое дерево языка (5.4.1). Использование функций является примером встраивания языка, когда мы вместо того, чтобы делать новый отдельный язык, его реализуем как библиотеку для уже существующего языка Gibbons [2013].

Как мы убедимся в течение курса, многие задачи можно рассматривать как придумывание языка и реализацию интерпретатора. Значит, если мы научимся хорошо писать интерпретаторы, мы научимся делать сразу кучу всего! И основные наши усилия будут направлены на изучение средств построения интерпретаторов встроенных языков.

Во время повествования мы часто пользуемся приёмом Hutton’s Razor, который подразумевает рассмотрение до смешного маленького языка для изучения сложных концепций. Утверждается, что для изучения большинства вопросов можно сконструировать такой язык, делающий всё важное максимально наглядным.

### 5.1.3 Интерпретаторы и семантика языков программирования

Семантика языков программирования<sup>133</sup> — это наука, изучающая свойства языков и смысл программ, его свойства и способы описания. Отличным введением может послужить серия книг Software Foundations<sup>134</sup> Pierce et al. [2010]. Существует много различных стилей описания семантики программ, для нас важнейшим будет денотационная семантика.

**Денотационная семантика**<sup>135136137</sup> описывает смысл программ путём сопоставления им объектов некоторого множества, **семантического домена**. Иначе говоря, денотационная семантика языка  $L$  — это тотальная функция из программы на этом языке в элемент домена  $D$ :

$$\llbracket \bullet \rrbracket : L \mapsto D$$

Домен выбирается исходя из языка и информации, которую хочется извлекать из программ. Например, чтобы узнать размер программы (тут, выражения со сложением), в качестве домена можно взять натуральные числа:

$$\begin{aligned} \llbracket n \rrbracket &= 1 \\ \llbracket l + r \rrbracket &= 1 + \max(\llbracket l \rrbracket, \llbracket r \rrbracket) \end{aligned}$$

<sup>133</sup>[https://en.wikipedia.org/wiki/Semantics\\_\(computer\\_science\)](https://en.wikipedia.org/wiki/Semantics_(computer_science))

<sup>134</sup><https://coq.vercel.app/ext/sf/>

<sup>135</sup>[https://en.wikipedia.org/wiki/Denotational\\_semantics](https://en.wikipedia.org/wiki/Denotational_semantics)

<sup>136</sup>[https://en.wikibooks.org/wiki/Haskell/Denotational\\_semantics](https://en.wikibooks.org/wiki/Haskell/Denotational_semantics)

<sup>137</sup>(youtube) The Lost Art of Denotational Semantics — Eric Meyer.

Если нас интересует конечный результат, можно посчитать его:

$$\begin{aligned}\llbracket n \rrbracket &= n \\ \llbracket l + r \rrbracket &= \llbracket l \rrbracket + \llbracket r \rrbracket\end{aligned}$$

Если у программы есть вход, доменом будет функция  $\mathbb{N} \rightarrow \mathbb{N}$ :

$$\begin{aligned}\llbracket n \rrbracket(m) &= n \\ \llbracket l + r \rrbracket(m) &= \llbracket l \rrbracket(m) + \llbracket r \rrbracket(m) \\ \llbracket \text{input} \rrbracket(m) &= m\end{aligned}$$

Таким образом, *программа является лишь синтаксической записью для некоторого элемента семантического домена*. Вариантов доменов много, это могут быть даже игры<sup>138</sup>.

**Упражнение 58** В какой домен разумно проинтерпретировать программы на языке с целочисленными мутабельными переменными? А на недетерминированном языке?

Легко заметить, что денотационная семантика языка — это просто интерпретатор, только написанный на языке математики. Такие интерпретаторы ставят в основу башни интерпретаторов, когда цель исследовать свойства языков и программ, а не исполнять их.

Также, можем реализовывать интерпретатор на каком-нибудь реальном языке и он тоже будет задавать семантику определяемого языка. Однако формальность такого определения будет зависеть от формальности описания семантики мета-языка. Такие интерпретаторы называют **определяющими**, они задают семантику языка, как правило, жертвуя эффективностью ради наглядности. Взаимоотношения определяемого языка и мета-языка изучаются в классических статьях Reynolds [1972, 1998]<sup>139</sup>.

Мы будем использовать определяющие интерпретаторы для задания семантики новых языков и в качестве мета-языка будем использовать Haskell. А в качестве доменов будем брать типы Haskell. И интерпретировать программу не в множество функций между натуральными числами, а, скажем, в тип `Nat -> Nat` в языке Haskell<sup>140</sup>. Так, денотационная семантика языка сумм с входом будет записываться следующим образом:

```
1 eval :: Prog -> (Nat -> Nat)
2 eval = \case
3 Val n -> _ -> n
4 Plus l r -> \m -> eval l m + eval r m
5 Input -> \m -> m
```

Семантика называется **композиционной (compositional)**, если смысл конструкций зависит только от смысла подконструкций. Иначе говоря, если денотационная семантика представляет собой свёртку программы (рис. 19) и может быть записана в терминах катаморфизма 4.3.3.

<sup>138</sup>[https://en.wikipedia.org/wiki/Game\\_semantics](https://en.wikipedia.org/wiki/Game_semantics)

<sup>139</sup>Активно используемое автором понятие продолжения будет рассмотрено далее в этом курсе (раздел 6).

<sup>140</sup><https://okmij.org/ftp/Denotational.html>

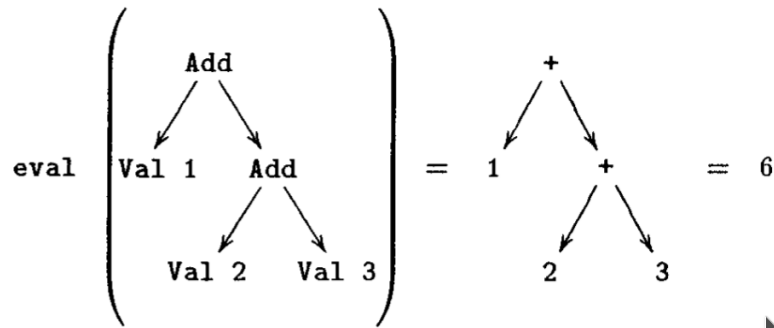


Рис. 19: Денотационная семантика определяет смысл синтаксических конструкций через операции над доменом Hutton [1998].

Другим популярным стилем описания семантики является **операционная семантика**, которая представляет смысл программы в виде последовательности шагов вычислений. Это может быть как последовательное переписывание самого выражения, так и переписывание состояния некоторой абстрактной машины. Операционная семантика, в свою очередь, задаётся как развёртка (или анаморфизм 4.3.3) последовательности шагов вычисления из программы. Тут отчётливо видна некоторая двойственность между денотационной и операционной семантиками Hutton [1998].

#### 5.1.4 Встроенные доменно-специфичные языки (eDSL)

Под **доменно-специфичными языками (domain specific languages, DSL)**<sup>141</sup> часто понимают специализированные языки для конкретных предметных областей, например, запросов к БД или форматирования документов. Как правило, такие языки не являются полными по Тьюрингу.

В этом курсе, однако, мы будем считать доменно-специфичным языком любую доменно-специфичную специализацию языка общего назначения<sup>142</sup>. Это следует из того соображения, что код должен читаться как грамотная проза с уместным словоупотреблением, предоставляющая читателю только необходимое количество подробностей, скрывая несущественное за умолчаниями и терминологией.

**Самостоятельные доменно-специфичные языки (standalone domain specific languages)** — языки, имеющие свой собственный конкретный синтаксис, а также инструменты программирования (IDE, исполняющую среду...). Примеры: SQL, AWK, Antlr...<sup>143</sup>

**Встроенные доменно-специфичные языки (embedded domain specific languages, eDSL)**

<sup>141</sup>[https://en.wikipedia.org/wiki/Domain-specific\\_language](https://en.wikipedia.org/wiki/Domain-specific_language)

<sup>142</sup>[https://en.wikipedia.org/wiki/Language-oriented\\_programming](https://en.wikipedia.org/wiki/Language-oriented_programming), на русскоязычную страницу тоже следует заглянуть.

<sup>143</sup>Есть отличная прикладная книга Nystrom, рассказывающая о построении интерпретаторов и простых виртуальных машин. В то же время она покрывает создание полноценного языка во всех его аспектах, от синтаксического анализа до управления памятью.

— языки, пользующиеся поддержкой инфраструктуры других языков. Обычно реализуются как библиотеки для программ на уже существующем языке общего назначения. Не имеют полностью собственного конкретного синтаксиса. Примеры: ORM, функции обработки строк, библиотека парсер-комбинаторов...

**Deep eDSL** — термы на таком языке строят дерево абстрактного синтаксиса для дальнейшей интерпретации:

```
1 f :: Int -> Int
2 f = eval $ Const 1 `Plus` Input
```

Интерпретаторы, которые принимают деревья на вход и интерпретируют их в семантический домен, называют **инициальными**<sup>144</sup>, мы их также будем называть “классическими”. С них мы и будем начинать. Классические интерпретаторы полезны, например, как для реализации “последнего языка” — интерфейса программы во внешний мир, и как фундамент для наших дальнейших построений.

Однако, можно заметить, что промежуточное дерево, которое получается, нас, как правило, не интересует. Нам важно только получить элемент домена, которым мы уже умеем пользоваться непосредственно. **Shallow eDSL** минуют стадию построения дерева и сразу строят значение в семантическом домене. Такие интерпретаторы мы будем называть **финальными**.

```
1 cnst :: Int -> (Int -> Int)
2 cnst x _ = x

3 input :: Int -> Int
4 input env = env

5 plus :: (Int -> Int) -> (Int -> Int) -> (Int -> Int)
6 plus l r env = l env + r env

7 f :: Int -> Int
8 f = cnst 1 `plus` input
```

Интерпретаторы часто называют **наблюдателями (observers)**, которые анализируют термы и дают им некоторый смысл Gibbons [2013]. Можно заметить, что для deep eDSL можно написать сколь угодно много различных наблюдателей. Однако в случае shallow embedding наблюдатели всегда id. Мы будем обсуждать возможные решения этой проблемы далее в разделе 5.4.1.

Обсуждение терминологии и сравнение подходов к построению DSL можно найти в Gibbons [2013]. Краткое описание терминов — в конспекте курса Language Engineering Hutchinson.

Введём ещё одно важное понятие. **Meta-circular интерпретатор**<sup>145</sup> — это интерпретатор, определяющий конструкции определяемого языка через конструкции мета-языка Reynolds [1972]. Например:

<sup>144</sup>Тип синтаксиса заданный с помощью `data` является начальным объектом категории интерпретаций.

<sup>145</sup>[https://en.wikipedia.org/wiki/Meta-circular\\_evaluator](https://en.wikipedia.org/wiki/Meta-circular_evaluator)

```

1 interpret term = case term of
2 App f t -> (interpret f) (interpret t)
3 If c t e -> if interpret c then interpret t else interpret e
4 ...

```

Свойства мета-языка в таком случае во многом определяют свойства объектного Reynolds [1972, 1998]. Мы будем в этом курсе стремиться как можно более переиспользовать возможности мета-языка.

**Упражнение 59** *Предположите, какие свойства наследует определяемый язык из примера выше.*

### 5.1.5 Пример: библиотека Accelerate

Интересным примером встроенного языка, находящегося где-то между deep и shallow является библиотека Accelerate<sup>146</sup> [Marlow, 2011, глава 6]. Она позволяет на Haskell описать вычисления, которые будут исполняться на GPU<sup>147</sup>.

Чтобы исполнить что-то на GPU, нужно породить и скомпилировать код на Cuda. Таким образом, язык Accelerate должен быть deep embedding, чтобы иметь дерево вычисления для трансляции в Cuda наиболее эффективным образом.

В то же время описывать численные вычисления как дерево крайне неудобно. Неплохо было бы иметь привычные операторы и функции высших порядков для работы с массивами на GPU. Поэтому Accelerate предоставляет на самом деле shallow интерфейс для построения деревьев. Так, для деревьев выражений определена реализация численных классов типов, например, `Num`, где операции просто достраивают дерево:

```

1 example :: Acc (Vector Int) -> Acc (Vector Int) -> Acc (Vector Int)
2 example xs ys = A.zipWith (+) xs ys

```

## 5.2 Типы значений

Рассмотрим, какие есть способы реализации языков, значения в которых могут иметь различные типы.

### 5.2.1 Untyped tagless interpreters

Для начала рассмотрим некоторый тривиальный нетипизированный язык. Под нетипизированностью понимаем отсутствие проверки типов как до исполнения программы, так и во время. Абстрактный синтаксис этого языка зададим следующим образом:

<sup>146</sup><https://hackage.haskell.org/package/accelerate>

<sup>147</sup> Другой подход: Java code reflection, чтобы в рантайме извлекать модель кода. Однако, такой подход не предоставляет статически гарантий программисту и требует глубокого внедрения в мета-язык.

```
1 data Expr = Const Int | IsZero Expr | If Expr Expr Expr
```

Значения, возникающие во время исполнения программ на этом языке будем представлять значениями типов `Bool` и `Int` языка Haskell. Соответственно, семантическим доменом программы на этом языке является либо `Bool`, либо `Int`, в зависимости от самой программы.

```
1 evalUnsafe :: Expr -> forall res . res
2 evalUnsafe = \case
3 Const val -> unsafeCoerce val
4 IsZero cond -> unsafeCoerce $ evalUnsafe @Int cond == 0
5 If c t e -> if evalUnsafe c then evalUnsafe t else evalUnsafe e
```

Здесь `unsafeCoerce` используется, чтобы обмануть статическую систему типов Haskell и просто исполнять программы на нашем нетипизированном языке. Мы имеем право так делать, поскольку `Int` и `Bool` в Haskell имеют одинаковый размер. Неверное написание программы на этом языке или выбор неправильного домена интерпретации приводят к падению.

## 5.2.2 Typed tagged interpreters

Чтобы добиться некоторой безопасности исполнения, будем приписывать значениям теги, которые будут доступны во время исполнения. Заведём следующий алгебраический тип:

```
1 data RtValue = RtBool Bool | RtInt Int
```

Теперь семантическим доменом у нас будет тип `RtValue`, а интерпретатор сможет проверять типы во время исполнения:

```
1 evalRt :: Expr -> RtValue
2 evalRt (IsZero expr) = case evalRt expr of
3 RtBool value -> error "Type error"
4 RtInt value -> RtBool (value == 0)
5 -- ...
```

Ситуация с безопасностью программы определённо стала лучше, однако проверка типов во время исполнения — это уже поздно: требует дополнительных расходов производительности и удорожает тестирование.

Этот подход часто называют **динамической типизацией**, когда мы атрибутируем значения некоторой типовой информацией для использования во время исполнения (см. также 3.4.6).

## 5.2.3 Typed tagless interpreters

Опишем систему типов нашего маленького языка.

$$\frac{n : \text{Int}}{\text{Const } n : \text{int}} \text{ Const} \quad \frac{n : \text{int}}{\text{IsZero } n : \text{bool}} \text{ IsZero} \quad \frac{c : \text{bool} \quad t : \tau \quad e : \tau}{\text{If } c \text{ t e} : \tau} \text{ If}$$

Можем в качестве типовых тегов переиспользовать типы Haskell:

$$int \rightsquigarrow Int, bool \rightsquigarrow Bool$$

Заметим, что с помощью обобщённого алгебраического типа данных `Expr ty` (2.1.3), мы как раз закодировали эти правила вывода. Иначе говоря, мы получили статически типизированный язык программирования, переиспользовав систему типов Haskell.

```
1 data Expr ty where
2 Const :: Int -> Expr Int
3 IsZero :: Expr Int -> Expr Bool
4 If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty

5 eval :: Expr ty -> ty
6 eval = \case
7 Const n -> n
8 IsZero e -> eval e == 0
9 If c t e -> if eval c then eval t else eval e
```

Благодаря статической типизации, мы можем отказаться от тегирования значений во время интерпретации без потери безопасности.

## 5.3 Связывания и функции первого класса

В этом параграфе мы рассмотрим техники и понятия, относящиеся к реализации функций первого класса и связываний в общем. Поскольку эта функциональность уже, как правило, реализована в мета-языке, мы будем стремиться её максимально переиспользовать.

`let`-связывания можно представить через функции первого класса следующим образом:

$$\text{let } x := N \text{ in } M \equiv (\lambda x. M) N$$

Напомним, что от функций первого класса можно избавиться с помощью дефункционализации, рассмотренной ранее 3.1.7.

### 5.3.1 Семантика имён

Существует несколько способов задания семантики идентификаторам.

**Динамическое связывание (dynamic scoping)** — значение свободных переменных функции зависит от области видимости в месте вызова. То есть разрешение имени происходит в момент обращения к переменной. Например, следующий код напечатает 42:

```
1 val x = 4
2 val f = () => x + 1
3 val x = 41
4 println(f())
```

Этот подход проще в реализации и использовался в ранних версиях Lisp'ов, например. Однако в таком случае функции не являются надежным барьером абстракции, по-хорошему все свободные переменные должны являться частью сигнатуры (вернёмся в этому в главе про системы эффектов 8).

**Лексическое/статическое связывание (lexical/static scoping)** — переменные связываются со значениями в момент объявления функции, в момент вызова результат зависит только от параметров (по модулю изменяемого состояния<sup>148</sup>). Слово “лексический” часто употребляется в языках, когда мы что-то можем понять из исходного кода без запуска программы. Так, код из примера выше напечатает 5.

Далее в этом разделе мы будем говорить о различных способах реализации функций первого класса со статическим связыванием переменных, которых, на самом деле, великое множество<sup>149</sup>.

### 5.3.2 Подстановки

Как можно заметить, в классическом лямбда-исчислении подстановки от бета-редукции (вспоминали в разделе 1.1) обеспечивают статическое связывание. Действительно, аргумент немедленно подставляется во все вхождения переменной, соответственно она не остаётся свободной, а просто исчезает.

$$(\lambda x. (\lambda x. \lambda y. x + y) 4) 41 \rightsquigarrow (\lambda x. (\lambda y. 4 + y)) 41$$

Такой подход не является самым эффективным, потому что на каждую аппликацию требуется переписывать код функции целиком (!). В то же время его довольно просто реализовать для некоторых представлений лямбда-термов. Рассмотрим пример такого представления — **locally nameless** Charguéraud [2012].

```
1 data Term var
2 = Var var
3 | App (Term var) (Term var)
4 | Lam (Term (Maybe var))
```

В этом представлении можно выбирать любой тип для именования свободных переменных:

```
1 example :: Term String
2 example = Var "x" `App` Var "y" -- x y
```

Добавление каждой связанной переменной добавляет типу переменных нового обитателя **Nothing** для обращения к ближайшей связанной переменной:

---

<sup>148</sup>Например, в Kotlin в лямбды можно захватывать изменяемые переменные. Изменения снаружи наблюдаемы внутри лямбды, и наоборот. Иногда это может быть очень удобно, однако нередко приводит к очень неочевидному поведению.

<sup>149</sup><https://jesper.cx/posts/1001-syntax-representations.html>

```

1 -- λx.x y
2 example1 = Lam $ Nothing `App` Just "y"
3 -- λx y.x y z
4 example2 = Lam $ Lam $ Just Nothing `App` Nothing `App` Just (Just "z")

```

Монадический bind является реализацией подстановки для таких термов:

```

1 instance Monad Term where
2 (>>=) :: Term var -> (var -> Term var') -> Term var'
3 Var var >>= subst = subst var
4 App l r >>= subst = App (l >>= subst) (r >>= subst)
5 Lam t >>= subst = Lam $ t >>= \case
6 Nothing -> Var Nothing
7 Just var -> Just <$> subst var

```

**Упражнение 60** Подумайте, зачем нужен `fmap Just` в последней строчке.

Соответственно, call-by-name интерпретатор такого лямбда-исчисления будет выглядеть следующим образом:<sup>150</sup>

```

1 eval :: Term var -> Term var
2 eval = \case
3 Var var -> Var var
4 App f arg -> case eval f of
5 Lam body -> eval $ body >>= maybe arg Var
6 t -> App t (eval arg)
7 Lam t -> Lam (eval t)

```

### 5.3.3 Окружение

Можно делать подстановку значений переменных лениво, распространяя окружение, которое ставит в соответствие свободным переменным термы. Лучше, в целом, пока не стало, но мы получили композиционную семантику из некомпозиционной путём эксплицирования контекстных зависимостей (подробнее далее 5.4.3).

```

1 data Term1 = Var1 String | App1 Term1 Term1 | Lam1 String Term1
2 type Env = Map String Term1

3 eval1 :: Term1 -> Env -> Term1
4 eval1 term env = case term of
5 Var1 name -> Map.findWithDefault (Var1 name) name env

```

<sup>150</sup>Поскольку мы рассматриваем классическое λ-исчисление, в качестве результирующего значения мы получаем тоже терм, но в нормальной форме.

```

6 App1 f arg -> case eval1 f env of
7 Lam1 name body -> eval1 body (Map.insert name arg env)
8 t -> App1 t (eval1 arg env)
9 Lam1 name body ->
10 let env' = Map.delete name env in
11 Lam1 name (eval1 body env')

```

**Упражнение 61** Объясните, зачем окружение модифицируется на строчке 10?

Если ветка `Lam1` не будет рекурсивно обходить подтерм и подставлять значения переменных, информация о значениях свободных переменных в нём потеряется и мы получим динамическое связывание вместо статического. Чтобы восстановить статическое связывание, ветка `Lam1` интерпретатора должна конструировать замыкание, включающее текущее окружение (см. далее 5.3.4).

### 5.3.4 Замыкания

Чтобы не делать энергично подстановку в тела функций и сохранить при этом статическое связывание, добавим ещё одну конструкцию, **замыкание (closure)**<sup>151152</sup> [Nystrom, глава 11]. Оно будет хранить контекст, в котором должен исполняться соответствующий терм.

```

1 data Term1 = Var1 String | App1 Term1 Term1 | Lam1 String Term1
2 | Closure Env String Term1 -- ТОЛЬКО ДЛЯ ВЫЧИСЛЕНИЙ
3 type Env = Map String Term1

4 eval1 :: Term1 -> Env -> Term1
5 eval1 term env = case term of
6 Var1 name -> Map.findWithDefault (Var1 name) name env
7 App1 f arg -> case eval1 f env of
8 Closure env' name body ->
9 let arg' = eval1 arg env in
10 eval1 body (Map.insert name arg' env')
11 t -> App1 t (eval1 arg env)
12 Lam1 name body -> Closure env name body

```

Замыкания обычно и используют в промышленных языках как представление времени исполнения функций высших порядков. Во время компиляции сначала производят **closure conversion** — функции высших порядков представляют как пару из окружения и указателя на функцию, принимающую окружение дополнительным аргументом. Теперь, когда функция не содержит свободных переменных, делают **lambda lifting**<sup>153</sup> — поднимают её на верхний уровень. Подробные примеры можно посмотреть в гарвардских слайдах Chong.

<sup>151</sup>[https://en.wikipedia.org/wiki/Closure\\_\(computer\\_programming\)](https://en.wikipedia.org/wiki/Closure_(computer_programming))

<sup>152</sup>Термин closure был предложен Piter Landin, вместе с кучей других вещей.

<sup>153</sup>[https://en.wikipedia.org/wiki/Lambda\\_lifting](https://en.wikipedia.org/wiki/Lambda_lifting)

### 5.3.5 Типизированный контекст

Рассмотрим кодирование, описанное, например, в Kiselyov [2012a].

Для начала научимся с помощью системы типов Haskell проверять валидность обращения к окружению. Представим окружение как список типов, закодированный с помощью вложенных пар:

```
1 (4, (4.0, "hello")) :: (Int, (Double, String))
```

Обращение к окружению будем кодировать числом в унарной записи. Тип числа (типизированной ссылки внутрь контекста) пусть задаёт множество окружений `env`, из которых на данной позиции можно извлечь тип `ty`.

```
1 data Ref env ty where
2 Here :: Ref (ty, env) ty
3 There :: Ref env ty -> Ref (ty', env) ty
```

Например, тип числа 1 утверждает, что с его помощью можно извлечь значение типа `ty` из контекста, в котором значение соответствующего типа находится на первой позиции (нумерация с нуля):

```
1 There Here :: Ref (ty', (ty, env)) ty
```

Теперь мы можем закодировать типизированное безопасное обращение к контексту:

```
1 envLookup :: env -> Ref env ty -> ty
2 envLookup env ref = case (ref, env) of
3 (Here, (x, _)) -> x
4 (There ref', (_, env')) -> envLookup env' ref'
```

**Упражнение 62** Можно ли разобрать пару сразу на строчке 2? Поясните.

### 5.3.6 Meta-circular интерпретация

Крайне не хотелось бы для eDSL самостоятельно реализовывать связывания и функции первого класса. Построим meta-circular интерпретатор (см. 5.1.4), который будет переиспользовать функции первого класса мета-языка для реализации их в определяемом языке.

Термы теперь будут не только аннотированы результирующими типами, но и типами необходимых для интерпретации окружений, рассмотренных ранее 5.3.5. Абстрагированному терму доступно большее окружение.

```
1 data Term2 env ty where
2 Var2 :: Ref env ty -> Term2 env ty
3 App2 :: Term2 env (arg -> res) -> Term2 env arg -> Term2 env res
4 Lam2 :: Term2 (arg, env) res -> Term2 env (arg -> res)
```

Теперь абстракцию можем проинтерпретировать в функцию Haskell, а аппликацию — в аппликацию:

```
1 eval2 :: Term2 env ty -> env -> ty
2 eval2 term env = case term of
3 Var2 ref -> env `envLookup` ref
4 App2 f arg -> (eval2 f env) (eval2 arg env)
5 Lam2 t -> \arg -> eval2 t (arg, env)
```

**Упражнение 63** Как так получилось, что в последней строчке нужно принять ещё один аргумент?

**Упражнение 64** Это *call-by-value* интерпретатор или *call-by-name*? От чего это зависит?

**Упражнение 65** Подумайте, какое решение должно быть более производительным, это или предыдущее?

Обратите внимание, что теперь функции определяемого языка во время исполнения — это просто функции мета-языка. А значит, в программах на определяемом языке мы *можем полностью переиспользовать мета-язык*! Добавим для этого конструкцию, позволяющую сохранить произвольное значение мета-языка в дереве:

```
1 data Term2 env ty where
2 Val2 :: ty -> Term2 env ty
3 -- ...
4
5 eval2 :: Term2 env ty -> env -> ty
6 eval2 term env = case term of
7 Val2 x -> x
8 -- ...
9
10 example :: Term2 env (Int -> Int)
11 example = Lam (Val2 (+) `App2` Val2 1 `App2` Var2 Here)
```

### 5.3.7 Синтаксис высшего порядка

Ещё чем мы ещё занимаемся вручную — определяем связыватели (да ещё и в унарной записи). Хотим переиспользовать их из мета-языка. Для это мы будем прямо в дереве синтаксиса хранить функции мета-языка — использовать **синтаксис высшего порядка (higher order abstract syntax)**<sup>154</sup><sup>155</sup> Pfenning and Elliott [1988]:

<sup>154</sup>[https://en.wikipedia.org/wiki/Higher-order\\_abstract\\_syntax](https://en.wikipedia.org/wiki/Higher-order_abstract_syntax)

<sup>155</sup>What is higher-order in higher-order abstract syntax?

```

1 data Term3 ty where
2 Val3 :: ty -> Term3 ty
3 Plus :: Term3 Int -> Term3 Int -> Term3 Int
4 App3 :: Term3 (arg -> res) -> Term3 arg -> Term3 res
5 Lam3 :: (Term3 arg -> Term3 res) -> Term3 (arg -> res)

6 example3 :: Term3 Int
7 example3 = (Lam3 \x -> x `Plus` Val3 41) `App3` Val3 1

```

Интерпретация очень простая и абсолютно meta-circular:

```

1 eval3 :: Term3 ty -> ty
2 eval3 term = case term of
3 Val3 x -> x
4 Plus l r -> eval3 l + eval3 r
5 App3 f arg -> (eval3 f) (eval3 arg)
6 Lam3 f -> \arg -> eval3 (f (Val3 arg))

```

**Упражнение 66** Можно ли было объявить `Lam3` следующим образом?

```

1 Lam3 :: (arg -> Term3 res) -> Term3 (arg -> res)

```

### 5.3.8 Сериализация функций

В этом разделе мы говорили о возможных реализациях функций первого класса, то есть функций, которые можно использовать так же гибко, как и данные. Возникает закономерный вопрос: можем ли мы сериализовать функцию первого класса и послать исполняться на другую машину?

Функция состоит из кода и захваченных свободных переменных в случае статического связывания. Соответственно, если код представлен в сериализуемом виде (например, позиционно-независимый байт-код), то его в принципе можно переслать по сети и исполнить на другом экземпляре виртуальной машины. Так, например, делает Erlang. Однако, такой подход неэффективный, так как байт-код нужно интерпретировать или предварительно компилировать. Таким образом, Erlang жертвует скоростью исполнения ради горизонтальной масштабируемости.

Если мы гарантируем, что на различных узлах кластера исполняется один и тот же код, как обычно и бывает на практике, можно добиться более эффективной реализации. Например, используя дефункционализацию (см. 3.1.7), мы можем сериализовать только объекты алгебраического типа, кодирующие функции. Поскольку на другом узле кластера исполняется такой же код, мы там можем десериализовать объект и исполнить его с помощью `apply`. Однако этот подход не очень поддерживает модульность (сложно один алгебраический тип разбить на много), а также `apply` каждый раз производит декодирование перед исполнением кода (чем, впрочем, можно пренебречь, учитывая работу с сетью).

Подход, реализованный в Haskell<sup>156157</sup> позволяет наделить каждую функцию без свободных переменных некоторым статически известным адресом, одинаковым для всех инстансов приложения. Далее можно сконструировать сериализуемое замыкание путём последовательности частичных применений:

```
1 data Closure a where
2 StaticPtr :: StaticPtr b -> Closure b
3 Encoded :: ByteString -> Closure ByteString
4 Ap :: Closure (b -> c) -> Closure b -> Closure c

5 main = send "some-node" $
6 closure (static factorial) `closureAp` closurePure 10
```

Подробнее можно прочитать в основополагающей статье про облачный Haskell Epstein et al. [2011]. С практической точки зрения — в книжке [Marlow, 2011, глава 16].

**Упражнение 67** Нужно ли явно добавлять в замыкание свободную переменную (\*) (оператор умножения) в реализации факториала?

## 5.4 Tagless final интерпретаторы

Как мы убедились ранее (5.1.2), программирование состоит из написания новых и новых интерпретаторов поверх друг друга. Интерпретаторы задают семантику новых языков (5.1.3). В классическом виде язык задаётся как множество деревьев, а интерпретатор управляет деревьями в объекте мета-языка. Если язык встроенный, то такой подход называют *deep embedding* (см. 5.1.4), а соответствующий интерпретатор инициальным.

$$\llbracket \bullet \rrbracket : L \rightarrow D$$

Можно заметить, что в конечном итоге мы используем только элемент домена, в который интерпретатор отображает программу. Сама программа же представляет собой лишь удобную синтаксическую запись элемента домена и является промежуточным шагом, а не самоцелью. В то же время доменом в случае встроенных языков, заданных интерпретаторами, являются объекты мета-языка. Можем ли мы миновать стадию интерпретации собственного синтаксиса и сразу строить объект домена в синтаксисе мета-языка? Да, такое встраивание называется *shallow embedding* (5.1.4), о нём эта глава.

### 5.4.1 Разные интерпретации для *shallow embedding*

Как мы узнали ранее (4.4), любую структуру данных можно представить свёрткой. И, более того, в итоге можно обойтись без единого конструктора данных (как в списке Чёрча, например): алгебра представляется набором функций, каждая из которых отвечает за сворачивание определённого конструктора.

<sup>156</sup><https://blog.ocharles.org.uk/blog/guest-posts/2014-12-23-static-pointers.html>

<sup>157</sup><https://hackage.haskell.org/package/distributed-closure>

```

1 Fix f $\cong$ forall a . (f a -> a) -> a
2 List e $\cong$ forall a . (e -> a -> a) -> a -> a

```

Таким образом, вместо того, чтобы конструировать дерево языка, а затем его интерпретировать (сворачивать), мы можем сразу сконструировать терм типа  $\forall a . (f\ a \rightarrow a) \rightarrow a$ . Предоставив ему тип домена  $a$  и алгебру  $f\ a \rightarrow a$  (либо в виде пачки функций), мы немедленно получим элемент нужного домена. Чтобы задать другую интерпретацию, нужно передать другой тип домена и алгебру:

```

1 example :: (Int -> a) -> (a -> a -> a) -> a
2 example cnst plus = cnst 1 `plus` cnst 41

3 ghci> example show (\l r -> l ++ " + " ++ r)
4 "1 + 41"

```

Если зафиксировать интерпретацию, то функции-аргументы можно реализовать статически и просто ссылаться на них в терме. Таким образом, про объявление функций можно думать как про расширение некоторого встроенного предметного языка. Общие рассуждения про shallow embeddings, свёртки и библиотеки можно почитать в Gibbons [2013], Gibbons and Wu [2014]. Сравните:

| Deep                                                                                                                                | Shallow                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Синтаксис языка задаётся набором допустимых нод дерева                                                                              | Декларация функции задаёт новую ноду дерева: вызов этой функции                      |
| Интерпретатор при виде каждой ноды выполняет соответствующий код на метаязыке (ветку паттерн-матчинга) после вычисления поддеревьев | Интерпретатор при виде вызова выполняет код тела функции после вычисления аргументов |

### 5.4.2 Дойти до конца

Вернёмся к некаррированной версии свёрток: теперь это тип, принимающий кортеж функций. А кортеж функций можно заменить на класс типов. Тогда декларация класса будет задавать синтаксис встроенного языка, а инстансы для доменов — реализацию. Этот подход называется **tagless final encoding**<sup>158</sup> и фактически это кодирование данных по Чёрчу с классами типов и набором трюков Carrette et al. [2007], Kiselyov [2012a].

Снова рассмотрим язык со сложением:

```

1 data Expr = Const Int | Plus Expr Expr

2 eval :: Expr -> Int
3 eval = \case Const x -> x; Plus l r -> eval l + eval r

```

<sup>158</sup><https://okmij.org/ftp/tagless-final/>

Или через катаморфизм:

```
1 data ExprF rec = Const Int | Plus rec rec
2 eval :: Fix ExprF -> Int
3 eval = cata \case Const x -> x; Plus l r -> l + r
```

Соответствующее tagless final кодирование будет выглядеть следующим образом:

```
1 class Expr domain where
2 cnst :: Int -> domain
3 plus :: domain -> domain -> domain
4 instance Expr Int where
5 cnst x = x
6 plus l r = l + r
```

Теперь мы можем сконструировать обычный терм Haskell, задать домен, и машинерия классов типов подставит нужную алгебру самостоятельно:

```
1 example :: forall domain . Expr domain => domain
2 example = cnst 1 `plus` cnst 41
3 ghci> example :: Int
4 42
```

Чтобы добавить ещё интерпретацию, реализуем инстанс для другого домена:

```
1 instance Expr String where
2 cnst x = show x
3 plus l r = l <> " + " <> r
4 ghci> example :: String
5 "1 + 41"
```

### 5.4.3 Восстановление композиционности семантики

**Семантика** называется **композиционной**, если семантика конструкций зависит только от семантик подконструкций (см. 5.1.3). Иначе говоря, она может быть задана катаморфизмом и, соответственно, инстансом класса типов в tagless final. То есть, чтобы уметь для любой семантики построить tagless final реализацию, нужно уметь универсальным образом превращать некомпозиционные семантики в композиционные.

Всякие преобразования кода, как правило, не композиционные. Для примера рассмотрим протаскивание унарных отрицаний:

```

1 data Expr1 = Lit Int | Add Expr1 Expr1 | Neg Expr1

2 transform1 :: Expr1 -> Expr1
3 transform1 = \case
4 Lit x -> Lit x
5 Add l r -> Add (transform1 l) (transform1 r)
6 Neg (Lit x) -> Lit (-x) -- проблема
7 Neg (Neg e) -> transform1 e -- проблема
8 Neg (Add l r) -> Add (transform1 (Neg l)) (transform1 (Neg r)) -- проблема

```

Чтобы восстановить композиционность семантики, нужно эксплицировать контекстные зависимости с помощью стрелочного домена Kiselyov [2012a]:

```

1 data Ctx = CtxPos | CtxNeg
2 flipCtx = \case CtxPos -> CtxNeg; CtxNeg -> CtxPos

3 transform1' :: Expr1 -> (Ctx -> Expr1)
4 transform1' expr = case expr of
5 Lit x -> \case CtxNeg -> Lit (-x); CtxPos -> Lit x
6 Neg e -> \ctx -> transform1' e (flipCtx ctx)
7 Add l r -> \ctx -> Add (transform1' l ctx) (transform1' r ctx)

```

Отсюда можно получить tagless final версию:

```

1 class Expr2 d where
2 lit :: Int -> d
3 add :: d -> d -> d
4 neg :: d -> d

5 instance Expr2 d => Expr2 (Ctx -> d) where
6 lit x = \case CtxNeg -> lit (-x); CtxPos -> lit x
7 neg e = \ctx -> neg e (flipCtx ctx)
8 add l r ctx = add (l ctx) (r ctx)

```

#### 5.4.4 Typed tagless final interpreter

Рассмотрим наш пример tagless initial encoding 5.2.3:

```

1 data Expr ty where
2 Const :: Int -> Expr Int
3 IsZero :: Expr Int -> Expr Bool
4 If :: forall ty . Expr Bool -> Expr ty -> Expr ty -> Expr ty

5 eval :: Expr ty -> ty

```

```

6 eval = \case
7 Const x -> x
8 IsZero t -> eval == 0
9 If c t e -> if eval c then eval t else eval e

```

Чтобы получить final encoding, параметризуем домен результирующим типом выражения:

```

1 class Expr (domain :: Type -> Type) where
2 cnst :: Int -> domain Int
3 isZero :: domain Int -> domain Bool
4 if' :: forall ty . domain Bool -> domain ty -> domain ty -> domain ty

5 instance Expr Identity where
6 cnst x = Identity x
7 isZero (Identity x) = Identity (x == 0)
8 if' (Identity c) t e = if c then t else e

```

**Упражнение 68** Какой домен подойдёт для печати выражения?

#### 5.4.5 Встречаем старых друзей: `Applicative`, `Monad`

Рассмотрим следующий язык в initial encoding с higher order abstract syntax (см. 5.3.6, 5.3.7). Справа перепишем в tagless final, выбирая подходящие имена для кусков синтаксиса.

|                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 data Expr s ty where 2   Val  :: ty -&gt; Expr s ty 3   App  :: Expr s (arg -&gt; res) 4         -&gt; Expr s arg 5         -&gt; Expr s res 6 7 8   LetIn :: Expr s ty 9         -&gt; (ty -&gt; Expr s ty') 10        -&gt; Expr s ty' 11 12 13   Get  :: Expr s s 14   Put  :: s -&gt; Expr s () </pre> | <pre> 1 class Applicative domain where 2   pure  :: ty -&gt; domain ty 3   (&lt;*&gt;) :: domain (arg -&gt; res) 4         -&gt; domain arg 5         -&gt; domain res 6 7 class Monad domain where 8   (&gt;=) :: domain ty 9         -&gt; (ty -&gt; domain ty') 10        -&gt; domain ty' 11 12 class MonadState s domain where 13   get :: domain s 14   put :: s -&gt; domain () </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Реализуем функцию, модифицирующую значение в нашем новом языке:

```

1 --
2 modify :: (s -> s) -> Expr s s
3 modify f =
4 Get `LetIn` \x ->
5 Put (f x) `LetIn` \() ->
6 Val x

```

```

1 -- s -> (s, s)
2 modify :: (s -> s) -> State s s
3 modify f =
4 get >>= \x ->
5 put (f x) >>= \() ->
6 pure x

```

И до боли знакомую интерпретацию:

```

1 eval :: Expr s ty -> s -> (s, ty)
2 eval = \case
3
4
5 Val x -> \s -> (s, x)
6 App fs xs -> \s1 ->
7 let (s2, f) = eval fs s1 in
8 let (s3, x) = eval xs s2 in
9 (s3, f x)
10
11
12 LetIn comp k -> \s ->
13 let (s', x) = eval comp s in
14 eval (k x) s'
15
16
17 Get -> \s -> (s, s)
18 Put s -> _ -> (s, ())

```

```

1 newtype State s a = State
2 { runState :: s -> (s, a) }
3
4 instance Applicative (State s) where
5 pure x = State \s -> (s, x)
6 fs <*> xs = State \s1 ->
7 let (s2, f) = runState fs s1 in
8 let (s3, x) = runState xs s2 in
9 (s3, f x)
10
11 instance Monad (State s) where
12 comp >>= k = State \s ->
13 let (s', x) = runState comp s in
14 runState (k x) s'
15
16 instance MonadState s (State s) where
17 get = State \s -> (s, s)
18 put s' = State \s -> (s', ())

```

Таким образом, *аппликативные функторы — meta-circular язык с аппликацией*, а *монадический bind — это фактически let-in в higher-order синтаксисе*.

Если вспомнить, что  $\text{let } x = M \text{ in } N \equiv (\lambda x. N) M \equiv [x \mapsto M] N$ , то мы поймём, что монадическое связывание — это подстановка  $a \rightarrow m$   $b$  вычисления  $m$   $b$  вместо результатов типа  $a$  в вычислении  $m$   $a$ . Либо, если в качестве  $m$  взять дерево выражения, то связывание будет подстановкой поддеревьев вместо переменных:

```

1 data Expr var = Var var | Empty | Append (Expr var) (Expr var)
2 instance Monad Expr where
3 (>>=) :: Expr var -> (var -> Expr var') -> Expr var'
4 Var name >>= subst = subst name
5 Empty >>= _ = Empty
6 Append l r >>= subst = Append (l >>= subst) (r >>= subst)

```

Изначально использовать теоретико-категорное понятие монады<sup>159</sup> в  $\lambda$ -исчислении было предложено в Moggi [1988], чтобы удобнее записывать денотационную семантику<sup>160</sup>. Это оказалось настолько удобно, что монады стали использовать повсеместно в функциональном программировании, чтобы расширять простые функциональные языки различными выразительными возможностями без изменения самих языков Wadler [1990, 1992]. Был сформулирован Moggi's principle:

«Computations of type  $\alpha$  correspond to values of type  $f \alpha$ »

Использовать аппликативные функторы предложили существенно позже, чтобы избежать именования промежуточных шагов вычислений, когда в этом нет необходимости McBride and Paterson [2008]. Таким образом, аппликативы дают встроенный язык выражений, а монады — язык стейтментов.

## 5.5 Expression problem

**Expression problem**<sup>161</sup> или **проблема выразительности** — это некоторый критерий выразительности языка программирования, сформулированный Wadler'ом в 1998<sup>162</sup>. Ставится вопрос: насколько легко расширять синтаксис встроенного языка и добавлять новые интерпретации? Иначе говоря, насколько легко добавлять новые разновидности данных и методы обработки.

Под “легкостью” подразумевается локальность: нужно ли править различные куски кода для этого. Например, если синтаксис языка задан обычным алгебраическим типом данных, то добавить новую интерпретацию “легко” — просто добавить новую рекурсивную функцию, а добавить новую синтаксическую конструкцию — “сложно” — нужно изменить все интерпретаторы:

```
1 data Expr
2 = Const Int
3 | Plus Expr Expr -- добавляем

4 eval :: Expr -> Int
5 eval = \case Const x -> x; Plus l r -> eval l + eval r

6 show :: Expr -> String
7 show = \case Const x -> show x; Plus l r -> show l ++ " + " ++ show r
```

Если язык задан, например, с помощью наследования, то, наоборот, расширить синтаксис легко — добавить новый класс, а добавить интерпретацию сложно — добавить реализацию метода в каждом классе:

---

<sup>159</sup>[https://ncatlab.org/nlab/show/monad+\(in+computer+science\)](https://ncatlab.org/nlab/show/monad+(in+computer+science))

<sup>160</sup>Часто сопутствующее монадам понятие эффекта мы рассмотрим далее 7.

<sup>161</sup>[https://en.wikipedia.org/wiki/Expression\\_problem](https://en.wikipedia.org/wiki/Expression_problem)

<sup>162</sup><https://homepages.inf.ed.ac.uk/wadler/papers/expression/expression.txt>

```

1 interface Lang {
2 fun eval(): Int
3 fun show(): String
4 }
5 class Const(val x: Int) : Lang {
6 override fun eval() = x
7 override fun show() = x.toString()
8 }
9 class Plus(val l: Lang, val r: Lang) : Lang {
10 override fun eval() = l.eval() + r.eval()
11 override fun show() = "$l + $r"
12 }

```

Оказывается, существуют подходы, позволяющие добиться “лёгкости” по обоим измерениям. Мы уделим им много внимания в этом курсе.

Действительно, как мы обсуждали ранее 5.1.1, программы представляют собой серию интерпретаторов. Для всё той же борьбы со сложностью, важно уметь описывать эти интерпретаторы модульно — задавать части языков отдельно друг от друга, и собирать нужные языки по месту из готовых блоков. Это помогает составлять программы из простых переиспользуемых компонент, каждая из которых имеет чёткую зону ответственности.

Expression problem возникала и решалась много раз: expression problem, stable denotations, extensible (modular) interpreters. Прошло немало времени, пока не возникло понимание, что всё это об одном и том же<sup>163</sup>.

### 5.5.1 Копроизведение функторов

Воспользуемся представлением данных как неподвижной точки функтора (см. 4.3.2). В качестве модельного языка возьмём язык выражений со сложением:

```

1 data Basic rec = Const Int | Plus rec rec
2
3 algBasic :: Basic Int -> Int
4 algBasic = \case Const x -> x; Plus l r -> l + r
5
6 evalBasic :: Fix Basic -> Int
7 evalBasic = cata algBasic

```

Заметим, что сумма (копроизведение) функторов формы даёт функтор формы, алгебра для которого получается из алгебр компонент<sup>164</sup>:

```

1 data (l :+: r) rec = L (l rec) | R (r rec)

```

<sup>163</sup><https://okmij.org/ftp/Computation/having-effect.html>

<sup>164</sup>Пользуемся расширением TypeOperators.

```

2 (\/) :: (l a -> a) -> (r a -> a) -> ((l :+: r) a -> a)
3 phi \/ psi = \case L l -> phi l; R r -> psi r

```

Расширим наш язык чтением числа из окружения. Следуя рассмотренной ранее денотационной семантике 5.1.3, выберем функцию `Int -> Int` в качестве домена:

```

1 data Basic rec = Const Int | Plus rec rec
2 data Input rec = Input

3 algBasic' :: Basic (Int -> Int) -> Int -> Int
4 algBasic' = \case Const x -> _env -> x; Plus l r -> \env -> l env + r env

5 algInput :: Input (Int -> Int) -> Int -> Int
6 algInput = \case Input -> \env -> env

```

Таким образом, мы добились возможности отдельно определять куски синтаксиса и семантики языка, и собирать нужный язык по месту. Напишем программу на нашем языке с неявным иммутабельным состоянием:

```

1 f :: Int -> Int
2 f = cata (algBasic' \/ algInput) $
3 In (L (Plus (In (L (Const 1))) (In (R Input)))) -- 1 + input

```

Однако заметим, что пока мы не решили проблему полностью, так как интерпретация новой конструкции `Input` потребовала более сложный домен и нам пришлось переписывать интерпретацию старой для него: из `algBasic` в `algBasic'`. Теперь мы понимаем, почему *stable denotations* — это ещё одно название для *expression problem* 5.5. Далее мы дополним это решение до полноценного 7.

## 5.5.2 Произведение алгебр

Построим *tagless final* представление для интерпретатора из предыдущего параграфа. Теперь синтаксис задаётся классами типов:

```

1 class Basic domain where
2 cst :: Int -> domain
3 plus :: domain -> domain -> domain

4 class Input domain where
5 input :: domain

```

В качестве домена, как и ранее возьмём функции `Int -> Int` и реализуем интерпретацию:

```

1 instance Basic (Int -> Int) where
2 cnst value = _ -> value
3 plus l r = \env -> l env + r env

4 instance Input (Int -> Int) where
5 input = \env -> env

```

Теперь, чтобы собрать язык по месту, можно просто воспользоваться композируемостью контекстов классов типов, всю остальную работу:

```

1 example :: forall domain . (Basic domain, Input domain) => domain
2 example = cnst 1 `plus` input

3 ghci> (example :: Int -> Int) 41
4 42

```

## 6 Продолжения (continuations)

Продолжения с начала 60х не один раз возникали в литературе в различных формах и разнообразных приложениях Reynolds [1993], Landin [1997], пока в 70х Wadsworth не придумал общий термин и единую концепцию — **continuation**<sup>165</sup> — “the meaning of the rest of the program”.

Начальным толчком к размышлениям стал язык Algol 60, имевший нетривиальный механизм меток и прыжков. Проблемой была как имплементация семантики, так и её денотационное описание вместе с трансляцией в лямбда-исчисление. Действительно, как математически описать `goto`? В каком домене искать семантику таких программ? Как написать определяющий интерпретатор, отправляющий программу в этот домен? Решением стала возможность сослаться на семантику остатка программы, продолжение, в определённой точке (например, на метке).

О продолжениях всегда должен думать автор языка, ведь вычислителю в каждой точке программы нужно знать, что исполнять дальше. А как мы уже поняли, любой программист является автором множества языков, приближающих его к решению задачи. Более того, многие повседневные языковые конструкции явно или неявно непосредственно манипулируют продолжениями.

### 6.1 Концепция продолжений

Рассмотрим семантику выражения с операционной точки зрения, как последовательность шагов переписывания (см. 5.1.3). Если внимательно рассмотреть каждый шаг, то мы обнаружим, что он состоит из двух этапов: поиска подвыражения (редекса), в котором можно сделать элементарный шаг вычислений, выполнение этого шага, и так далее<sup>166</sup>. Во время поиска редекса, выражение разбивается на две части (см. 20):

- Фокус — подвыражение в котором ищем редекс;
- Продолжение — остаток выражения с “дыркой”, обозначающий место, куда нужно подставить результат шага вычислений.

Как правило, продолжения существуют вне пользовательского кода как состояние интерпретатора, которому в каждый следующий момент времени нужно помнить, какой код и в каком состоянии исполнять дальше. Однако, языки предоставляют пользователям множество конструкций, позволяющих управлять продолжениями (см. 21):

- Функция `exit` выбрасывает продолжение программы целиком;
- Конструкция `try-catch` позволяет выбросить часть продолжения до места поимки исключения и исполнить оставшееся;

---

<sup>165</sup><https://en.wikipedia.org/wiki/Continuation>

<sup>166</sup>На самом деле вычислениям с продолжениями учат в начальных классах, когда рассказывают про вычисление выражений “по действиям”.

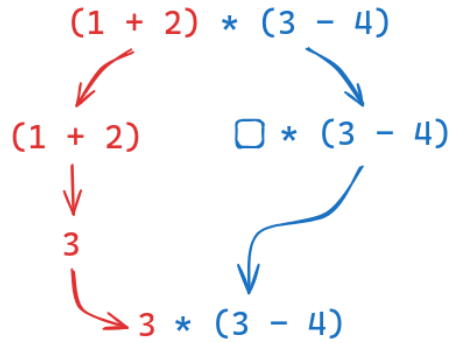


Рис. 20: Выражение разделяется на фокус (красный) и продолжение (синее), когда в фокусе удалось сделать шаг, они объединяются в новое выражение, для которого процесс повторяется.

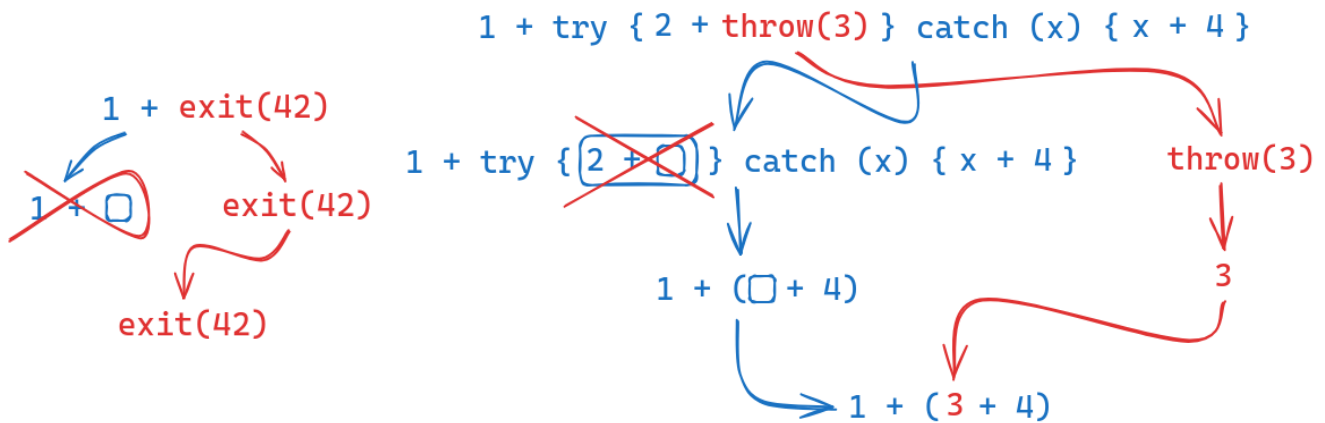


Рис. 21: Конструкция `exit` выбрасывает продолжение и программа останавливается, в то время как `throw` выбрасывает лишь часть продолжения до ближайшего `try-catch`.

- Конструкция `return` позволяет восстановить исполнение в месте, где функция была вызвана;
- Конструкции `break` и `continue` восстанавливают продолжение после цикла и до...

### 6.1.1 Reduction semantics

Стиль задания операционной семантики через описание поиска примитивных (“головных”) редексов и их редукций называют **семантикой редукционных контекстов (reduction semantics)**<sup>167</sup>. Как обычно, следуя Hutton’s Razor, рассмотрим интерпретацию маленького простого язычка с вычитанием (несимметричная операция для проверки реализации):

```
1 data Expr = Const Int | Diff Expr Expr
```

<sup>167</sup>[https://en.wikipedia.org/wiki/Operational\\_semantics](https://en.wikipedia.org/wiki/Operational_semantics)

Для начала нам нужно определить синтаксис продолжений, “выражений с дыркой”. То есть с технической точки зрения продолжение — это структура данных, содержащая всю необходимую информацию, чтобы продолжить исполнение (здесь — только остаток выражения):

```
1 data K
2 = Hole -- дырка □
3 | LDiff K Expr -- фокус пошел в левое подвыражение, запомнили правое
4 | RDiff Int K -- посчитали левое, фокус пошел вправо
```

Зададим операцию разбиения выражения на фокус и продолжение:

```
1 split :: Expr -> (Expr, K)
2 split e = case e of
3 Const _ -> (e, Hole)
4 Diff (Const _) (Const _) -> (e, Hole) -- примитивный редекс
5 Diff (Const l) r -> -- левое подвыражение уже посчитано
6 let (focus, k) = split r in -- ищем редекс в правом
7 (focus, RDiff l k)
8 Diff l r -> -- ничего ещё не посчитано
9 let (focus, k) = split l in -- начинаю с поиска редекса слева
10 (focus, LDiff k r)

11 ghci> split (Diff (Diff (Const 1) (Const 2)) (Const 3)) -- (1 - 2) - 3
12 (Diff (Const 1) (Const 2), LDiff Hole (Const 3)) -- (1 - 2, □ - 3)
```

Шаг примитивной редукции умеет только вычитать числа:

```
1 headReduction :: Expr -> Expr
2 headReduction = \case
3 Diff (Const l) (Const r) -> Const (l - r)
4 e -> e
```

После шага примитивной редукции нам понадобится подставить результат обратно в продолжение, чтобы из “выражения с дыркой” получить полноценное выражение, которое можно продолжить редуцировать:

```
1 plugIn :: Expr -> K -> Expr
2 plugIn e k = case k of
3 Hole -> e
4 LDiff k' r -> Diff (e `plugIn` k') r
5 RDiff l k' -> Diff (Const l) (e `plugIn` k')
```

Как правило, подстановку терма  $t$  в продолжение  $E$  обозначают как  $E[t]$ .

Теперь мы можем определить полноценный шаг:

```

1 transition :: Expr -> Expr
2 transition e =
3 let (focus, k) = split e in -- разбиваем на фокус и контекст
4 headReduction focus `plugIn` k -- делаем вычисление в фокусе и подставляем

```

Тогда операционная семантика это развёртка списка промежуточных выражений:

```

1 eval :: Expr -> [Expr]
2 eval = List.unfoldr \prev ->
3 let next = transition prev in
4 if prev == next then Nothing else Just (next, next)

```

Можно заметить, что `split` и `plugIn` однозначно определяется по синтаксису продолжений и виду главных редукций, а `transition` и `eval` одинаковы для произвольного языка. Поэтому семантику редукционных контекстов обычно задают как синтаксис продолжений, перечень главных редукций и единственное правило вывода — шаг в контексте:<sup>168</sup>

|                    |                                                            |
|--------------------|------------------------------------------------------------|
| Values             | $v ::= \mathbb{Z}$                                         |
| Terms              | $t ::= v \mid t \ominus t$                                 |
| Evaluation context | $E ::= \square \mid E \ominus t \mid \mathbb{Z} \ominus E$ |
| (diff)             | $v_1 \ominus v_2 \longrightarrow v_1 - v_2$                |
|                    | $\frac{t \longrightarrow t'}{E[t] \longrightarrow E[t']}$  |
| step               |                                                            |

Существует стандартный инструмент PLT Redex<sup>169</sup> для описания и тестирования семантики в стиле редукционных контекстов.

### 6.1.2 Continuation semantics

Запишем денотационную семантику нашего языка:

```

1 evalDirect :: Expr -> Int
2 evalDirect = \case
3 Const n -> n
4 Diff l r -> evalDirect l - evalDirect r

```

Перепишем денотационную семантику в стиле с явными продолжениями. Но сначала поработаем с типом `K` и упростим работу с ним:

```

1 data K = Hole | LDiff K Expr | RDiff Int K
2 -- K = 1 + (K × Expr) + (Int × K) = 1 + (Expr + Int) × K
3 data Frame = LDiff Expr | RDiff Int
4 type K = [Frame]

```

<sup>168</sup> $\ominus$  тут обозначает синтаксический минус (ноду в дереве).

<sup>169</sup><https://redex.racket-lang.org/>

Получили представление продолжения как стека фреймов.

Денотационную семантику будем записывать для домена  $K \rightarrow \text{Int}$ . Такая разновидность денотационной семантики с явным представлением смысла остатка программы (в виде  $K$ ) иногда называют **continuation semantics**. А соответствующий стиль программирования с передачей продолжений — **continuation passing style (CPS)**.

В нашем continuation semantics будет выглядеть как пара взаимно-рекурсивных функций:

```
1 evalK :: Expr -> K -> Int
2 evalK e k = case e of
3 Const n -> k `appK` n -- выполняем остаток программы
4 Diff l r -> evalK l (LDiff r : k) -- запоминаем дальше вычислить правое

5 appK :: K -> Int -> Int
6 appK k result = case k of
7 [] -> result -- дальше делать нечего
8 LDiff r : k' -> evalK r (RDiff result : k') -- идем вычислять вправо
9 RDiff l' : k' -> k' `appK` (l' - result) -- продолжаем на результате
```

Тут мы снова спускаемся по выражению в поисках примитивного редекса (тут — константы), попутно запоминая, что нужно будет сделать после.

Первая рекурсивная реализация `evalDirect` не заботилась о продолжениях. Однако, продолжения — это неотъемлемая часть процесса вычисления, вычислителю нужно знать, что делать дальше в каждый момент. На самом деле `evalDirect` лишь делегирует работу с продолжениями определяемого языка мета-языку, но каким образом? Заметьте, что полученная в итоге реализация `evalK` хвостово-рекурсивна, а значит, может быть скомпилирована в цикл, не потребляющий стек вызовов. При этом  $K$  и есть стек. Таким образом, в первом случае мы делали рекурсивные вызовы и продолжение аллоцировалось на (аппаратном) стеке мета-языка, а во втором случае мы самостоятельно аллоцируем стек в куче.

В случае, если стек мета-языка реализован поверх аппаратного, есть риск ошибки его переполнения. Чтобы этого избежать, используется техника **trampolining**, которая как раз состоит в ручной аллокации продолжения в куче Ganz et al. [1999], Bjarnarson [2012].

Подобно тому как тип контекста зиппера можно вычислить как производную алгебраического представления соответствующего типа Huet [1997], McBride [2001], Abbott et al. [2003], так можно вычислить тип продолжения свёртки McBride [2008].

### 6.1.3 Продолжения первого класса

В примерах выше конструкции языка управляют продолжениями неявно (см. рис. 21). Однако, иногда вводят операторы, позволяющие явно оперировать продолжениями. С их помощью можно реализовать как возможности манипуляции потоком управления вроде генераторов и корутин<sup>170171</sup>, так и все остальные эффекты вроде состояния (см. далее 7).

<sup>170</sup>[https://en.wikibooks.org/wiki/Haskell/Continuation\\_passing\\_style#Example:\\_coroutines](https://en.wikibooks.org/wiki/Haskell/Continuation_passing_style#Example:_coroutines)

<sup>171</sup><https://kotlinlang.org/api/core/kotlin-stdlib/kotlin.coroutines/suspend-coroutine.html>

**Продолжения первого класса (first-class continuations)** — продолжения, которые представимы в программе в виде значений. Учитывая, что продолжение имеет вакантное место ещё не вычисленного подвыражения, продолжения первого класса представляются функциями первого класса.

Чтобы получить в коде продолжение первого класса, нужно либо написать код в CPS, либо воспользоваться встроенным в язык оператором; таких операторов придумано великое множество [Hillerström, 2022, приложение A]. Например, *J*, *escape* Reynolds [1972], *call/cc*...

Для примера реализуем в языке операцию *Cont*, позволяющую захватить текущее продолжение. Он будет принимать пользовательскую функцию и передавать в неё текущее продолжение от себя и до конца программы:

$$E[\text{cont } f] \longrightarrow f (\lambda x. E[x])$$

Сначала расширим язык лямбда-исчислением:

```

1 data Expr
2 = Const Int | Diff Expr Expr
3 | Var String | Lam String Expr | App Expr Expr

4 data Frame
5 = LDiff Expr | RDiff Value
6 | LApp Expr | RApp Value
7 | SetEnv Env -- после исполнения замыкания возвращаем текущее окружение

```

Реализация интерпретатора довольно прямолинейна. Единственное, сейчас остаток программы представлен в виде линейного списка, который фактически обходится в цикле последовательно. Нужно не забыть после исполнения тела замыкания восстановить изначальное окружение для исполнения продолжения.<sup>172</sup>

```

1 evalK :: (?env :: Env) => Expr -> K -> Value
2 evalK e k = case e of
3 Const n -> k `appK` Number n
4 Diff l r -> evalK l (LDiff r : k)
5 Var name -> k `appK` (?env ! name)
6 Lam name body -> k `appK` Closure name ?env body
7 App f arg -> evalK f (LApp arg : k)

8 appK :: (?env :: Env) => K -> Value -> Value
9 appK k result = case k of
10 [] -> result
11 LDiff r : k' -> evalK r (RDiff result : k')
12 RDiff l' : k' -> k' `appK` Number (unwrapNumber l' - unwrapNumber result)

```

<sup>172</sup>Для удобства будем передавать окружение неявным параметром.

```

13 LApp arg : k' -> evalK arg (RApp result : k')
14 RApp f : k' -> case f of
15 Closure name env body ->
16 let currEnv = ?env in
17 let ?env = Map.insert name result env in
18 evalK body (SetEnv currEnv : k')
19 K env k' ->
20 let currEnv = ?env in
21 let ?env = env in
22 (k' ++ SetEnv currEnv : k') `appK` result
23 other -> error $ "Expected callable, got " <> show other
24 SetEnv env : k' -> let ?env = env in k' `appK` result
25 where
26 unwrapNumber = \case
27 Number n -> n
28 other -> error $ "Expected number, got " <> show other

```

Наконец, реализуем оператор cont:

```

1 data Expr = ... | Cont Expr
2 data Frame = ... | ContFrame

3 evalK e k = case e of
4 ...
5 Cont f -> evalK f (ContFrame : k)

6 appK k result = case k of
7 ...
8 ContFrame : k' -> [RApp result] `appK` K ?env k' -- переиспользуем ветку RApp

```

Рассмотрим несколько примеров (обозначим `contLam name body = Cont (Lam name body)`, `(-.) = Diff`, `c = Const`, `v = Var` и `(@) = App`):

- `c 10 -.` `contLam "k" (c 1)` — выбросить продолжение и вернуть 1 ( $k = 10 - \square$ );
- `c 10 -.` `contLam "k" (v "k" @ c 1)` —  $\rightsquigarrow k\ 1 \rightsquigarrow 10 - 1 \rightsquigarrow 9$ ;
- `c 10 -.` `contLam "k" (v "k" @ c 1 -.` `c 2)` —  $\rightsquigarrow k\ 1 - 2 \rightsquigarrow 9 - 2 \rightsquigarrow 7$ ;
- `c 10 -.` `contLam "k" (v "k" @ c 1 -.` `v "k" @ c 2)` —  $\rightsquigarrow k\ 1 - k\ 2 \rightsquigarrow 9 - 8 \rightsquigarrow 1$ .

Таким образом, мы получили язык, который позволяет пользователю в произвольном месте программы получить текущее продолжение в виде функции.

Возможность получить продолжение первого класса почти не предоставляется промышленными языками, так как это довольно опасный инструмент. Действительно, если пользователь не вызовет продолжение, может не произойти закрытия ресурсов. Если вызовет несколько раз, снова может произойти как некорректная работа с ресурсами, так и порча

изменяемой памяти (продолжения, которые можно безопасно вызывать много раз называют **multi-shot**). Подобные поведения можно исключать специальной обработкой таких ситуаций Mihcs et al. [2025], проверками времени компиляции (например, с помощью линейных типов) или времени исполнения. Сейчас с активными исследованиями о внедрении хендлеров эффектов (см. далее 7) продолжения первого класса могут получить новый шанс.

## 6.2 Продолжения своими руками

Рассмотрим, как выглядит CPS в обычном коде, а не в контексте deep embedding. Для этого мы, во-первых, перейдём к shallow embedding и будем сразу строить элементы целевого домена. А во-вторых, рефункционализируем продолжения  $K$  и обобщим: вместо  $K \rightarrow \text{Int}$  будем использовать  $\text{forall } r . (a \rightarrow r) \rightarrow r$ , где  $a \rightarrow r$  — функциональное представление продолжения.

Такой CPS эксплуатирует следующий изоморфизм:

```
1 to :: a -> (forall r . (a -> r) -> r)
2 to x k = k x

3 from :: (forall r . (a -> r) -> r) -> a
4 from comp = comp id
```

Иначе говоря, вместо того, чтобы предоставить значение типа  $a$ , можно спросить у вызывающей стороны, как она собирается с этим значением работать  $a \rightarrow r$ , сделать это самостоятельно, и вернуть вызывающей стороне  $r$ .

Прикладным программистам этот изоморфизм знаком по технике использования callback'ов. Теоретикам же известно, что он является частным случаем леммы Йонеды Hinze and James [2010]. Сравните CPS с интансом этой леммы в теории предпорядка (выше мы считали  $a = \text{Unit}$ ;  $b = a$ ):

$$a \rightarrow b \cong \forall r. (b \rightarrow r) \rightarrow (a \rightarrow r)$$

$$a \leq b \iff \forall r. (b \leq r) \Rightarrow (a \leq r)$$

Например, мы можем переписать факториал в CPS. Заметьте, что код имеет доступ к продолжению первого класса (однако, пока никак нетривиально не использует его), при этом снова является хвостово-рекурсивным.

```
1 facCps :: Int -> (forall r . (Int -> r) -> r)
2 facCps n k
3 | n <= 1 = k 1
4 | otherwise = facCps (n - 1) \res -> k (n * res)

5 facCps 3 id ~> facCps 2 \res -> id (3 * res)
6 ~> facCps 1 \res -> id (3 * (2 * res)) ~> id (3 * (2 * 1)) ~> 6
```

**Упражнение 69** Сколько функция `facCps` потребляет стековой памяти?

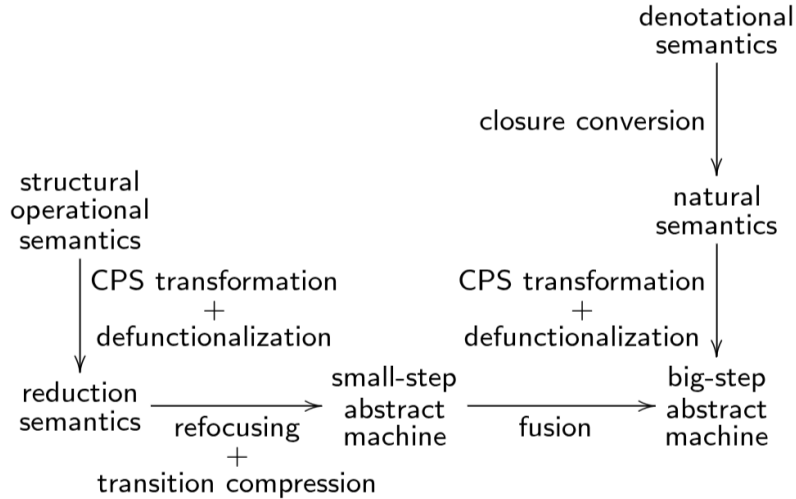


Рис. 22: Связь между семантиками в различных стилях Danvy [2008].

### 6.2.1 Дефункционализация и аккумуляторы

Дефункционализируем продолжения в `facCps`. Мы используем две функции высших порядков, `id` и `\res -> k (n * res)`, они дают нам два конструктора:

```

1 data K = Id | Times Int K
2 -- ≅
3 type K = [Int]

4 runK :: K -> Int
5 runK = product

6 facCps :: Int -> K -> Int
7 facCps n k
8 | n <= 1 = runK k
9 | otherwise = facCps (n - 1) (snoc k n)

10 facCps 3 [] ~> facCps 2 [3] ~> facCps 1 [3, 2] ~> runK [3, 2] ~> 3 * (2 * 1)

```

Мы снова получили представление продолжения в виде стека фреймов. А значит, интерпретатор `evalK` полученный нами ранее (6.1.2) является CPS версией обычного интерпретатора, только с дефункционализированными продолжениями (они удобны для отладки — их можно распечатать, в отличие от функций Haskell). Если расширить тот наш язык и написать на нём факториал, а потом сделать `fusion` или рефункционализацию (4.4.1) дерева программы и дерева продолжения, мы получим текущую реализацию факториала. И в целом между различными стилями реализаций семантик можно построить соответствие (см. рис. 22).

Этой техникой можно пользоваться и в более сложных случаях, чтобы легко получать

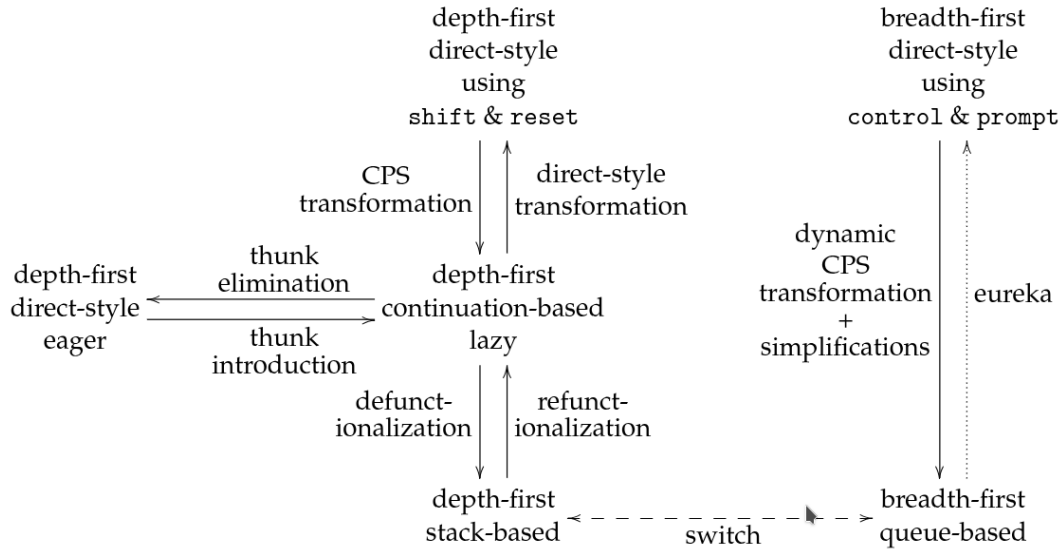


Рис. 23: Трансформации кода, основанные на CPS и де(ре)функционализации Danvy [2006].

хвостово-рекурсивные (итеративные) реализации<sup>173</sup> Gibbons [2021]<sup>174</sup>. В целом CPS и дефункционализация — это богатый источник различных рефакторингов (см. рис. 23).

Теперь заметим, что операция умножения ассоциативна, а значит, можно `знос` заменить на умножение, а продолжение представить одним числом. Получим привычную реализацию факториала с аккумулятором:

```

1 facAcc :: Int -> Int -> Int
2 facAcc n acc
3 | n <= 1 = acc
4 | otherwise = facAcc (n - 1) (acc * n)

5 facAcc 3 1 ~> facAcc 2 (1 * 3) ~> facAcc 1 ((1 * 3) * 2) ~> (1 * 3) * 2 ~> 6

```

Также, можно фреймы представить в виде эндоморфизмов и в качестве ассоциативной операции использовать композицию функций Ploeg and Kiselyov [2014]<sup>175</sup>.

## 6.2.2 Monad Cont

Из-за CPS код потерял привычную структуру, при которой функции напрямую возвращают свои результаты (i.e. **direct style**). При наличии большого количества вызовов трансформированных функций, код становится плохо читаемым (проблема известна как **callback hell**):

```

1 fibCps :: Int -> (forall r . (Int -> r) -> r)

```

<sup>173</sup><https://www.pathsensitive.com/2019/07/the-best-refactoring-youve-never-heard.html>

<sup>174</sup>(youtube) Jeremy Gibbons - Continuation-passing style, defunctionalization, and associativity.

<sup>175</sup>[https://wiki.haskell.org/Difference\\_list](https://wiki.haskell.org/Difference_list)

```

2 fibCps n k = if n <= 2 then k 1 else
3 fibCps (n - 1) \res1 ->
4 fibCps (n - 2) \res2 ->
5 k (res1 + res2)

```

Домен  $(a \rightarrow r) \rightarrow r$  можно сделать монадой и восстановить direct style код внутри `do`-нотации. Заведём `newtype` обёртку для объявления инстансов:

```

1 newtype Cont r a = Cont { runCont :: (a -> r) -> r }

```

Функтор добавляет пост-процессинг результату перед передачей в продолжение:

```

1 instance Functor (Cont r) where
2 -- fmap :: (a -> b) -> ((a -> r) -> r) -> ((b -> r) -> r)
3 fmap f (Cont comp) = Cont \k -> comp (k . f)

```

Аппликатив просто передаёт значение продолжению:

```

1 instance Applicative (Cont r) where
2 pure x = Cont \k -> k x
3 (<*>) = ap

```

Можно заметить, что монадическое связывание вторым аргументом тоже принимает продолжение, но “маленькое”, до конца `do`-блока. Таким образом, смысл реализации монадического связывания для `Cont` — это композиция “маленького” продолжения с “большим” продолжением, передаваемым снаружи:

```

1 instance Monad (Cont r) where
2 (>>=) :: Cont r a -> (a -> Cont r b) -> Cont r b
3 Cont comp >>= k = Cont \k' -> comp \x -> runCont (k x) k'

```

Теперь мы можем писать линейный код, а монадическая машинерия сама конструирует продолжения и подкладывает в предыдущие вычисления:

```

1 fibCont :: Int -> Cont r Int
2 fibCont n = if n <= 2 then pure 1 else do
3 res1 <- fibCont (n - 1)
4 res2 <- fibCont (n - 2)
5 pure (res1 + res2)

```

**Упражнение 70** Оборвите вычисление, если `res1` больше 50.

**Упражнение 71** Оборвите вычисление как только общий результат стал больше 50.

Монада `Cont` даёт реализацию встроенного языка, в котором можно получить продолжение вычисления:

```
1 Cont :: ((a -> r) -> r) -> Cont r a
```

Прикладным программистам такая техника написания CPS кода через монады знакома в виде концепций Future/Promise<sup>176177</sup>.

### 6.2.3 call/cc

Самым известным классическим оператором, который использовали в Scheme для получения продолжений первого класса, является call/cc (**call with current continuation**)<sup>178179</sup>. Предоставляемое продолжение является **неограниченным (undelimited/abortive)**, так как оно содержит “конец программы” — никакой код не будет исполняться после его вызова. Неограниченные продолжения де-факто не совсем являются функциями, так как они не возвращают результата (он уже “beyond the grave”). Следовательно, они также не композируются; точно так же странно композировать abort с exit<sup>180181</sup>. Они скорее являются ко-значениями: пока часть программы выполняется, они ожидают её результата Curien and Herbelin [2000].

Давайте сэмулируем call/cc в монаде Cont. Захватить всё продолжение программы у нас не выйдет, так как оно собирается только в рамках Cont, но мы можем проигнорировать продолжение вызова захваченного продолжения:

```
1 callCC :: ((a -> Cont r b) -> Cont r a) -> Cont r a
2 callCC f = Cont \k -> runCont (f \x -> Cont _ -> k x) k

3 foo :: Int -> Cont r String
4 foo x = callCC \k -> do
5 let y = x ^ 2 + 3
6 when (y > 20) $ k "over twenty" -- throws next line away
7 pure (show $ y - 4)
```

## 6.3 Разграниченные продолжения (delimited continuations)

В современной практике, как правило, используют продолжения не до конца программы, а только до определённой точки. Их называют **ограниченными** или **разграниченными продолжениями**, **delimited** или **composable continuations**, **subcontinuations**<sup>182183</sup>.

Конструкции для работы с такими продолжениями парные: вводится оператор, ограничивающий текущее продолжение (мб имеющий метку); а также оператор захвата фрагмента

<sup>176</sup>[https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Promise](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise)

<sup>177</sup><https://github.com/promises-aplus/promises-spec/issues/94>

<sup>178</sup><https://en.wikipedia.org/wiki/Call-with-current-continuation>

<sup>179</sup>[https://en.wikibooks.org/wiki/Haskell/Continuation\\_passing\\_style#callCC](https://en.wikibooks.org/wiki/Haskell/Continuation_passing_style#callCC)

<sup>180</sup><https://okmij.org/ftp/continuations/undelimited.html>

<sup>181</sup><https://okmij.org/ftp/continuations/against-callcc.html>

<sup>182</sup><https://www.cl.cam.ac.uk/teaching/2324/R277/handout-delimited-continuations.pdf>

<sup>183</sup>(youtube) Keynote: Delimited Continuations, Demystified by Alexis King | Lambda Days 2023.

(Variables)  $x, y, \dots$   
 (Expressions)  $e ::= x \mid \lambda x. e \mid e e$   
 $\quad \quad \quad \mid \text{newPrompt} \mid \text{pushPrompt } e \text{ } p$   
 $\quad \quad \quad \mid \text{withSubCont } e \text{ } f \mid \text{pushSubCont } e \text{ } k$

Рис. 24: Синтаксис  $\lambda$ -исчисления с примитивами для работы с продолжениями.

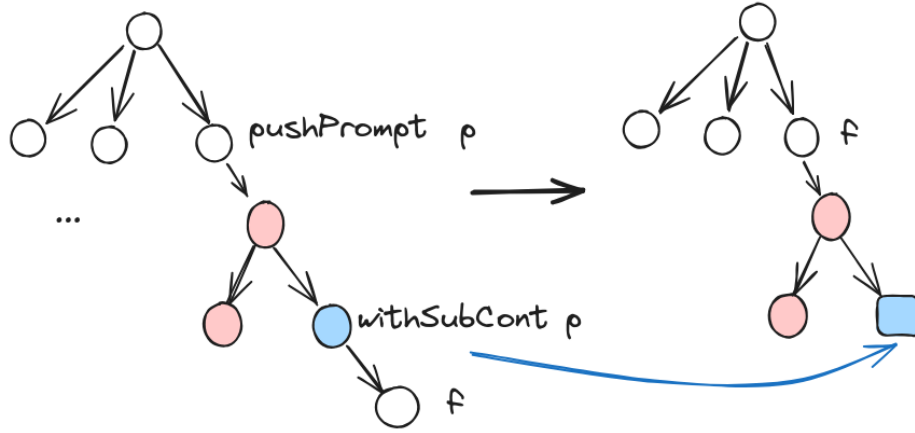


Рис. 25: Пример работы withSubCont.

текущего продолжения (мб до конкретного ограничителя с определённой меткой). Таких операторов придумано много [Hillerström, 2022, приложение A], но они более-менее все сводятся друг к другу.

Например, работа с исключениями подразумевает использование двух конструкций: продолжение ограничивается с помощью **try-catch**, а **throw** выкидывает соответствующее частичное продолжение, не захватывая его (см. рис. 21):

$$E_1[\text{try}\{E_2[\text{throw}(v)]\}\text{catch}(x)\{t\}\} \rightarrow E_1[[x \mapsto v] t]$$

Мы же рассмотрим универсальные операторы из Dyvbig et al. [2007]. Работа вводит следующий набор синтаксических конструкций (рис. 24) для работы с ограниченными продолжениями в дополнение к чистому call-by-value лямбда-исчислению:

- $\text{newPrompt}^{184}$  — создаёт свежий идентификатор (метку) ограничения;
- $\text{pushPrompt } p \text{ } e$  — устанавливает ограничение с меткой  $p$  и исполняет выражение  $e$ ;
- $\text{withSubCont } p \text{ } f$  — захватывает частичное продолжение до ограничения с меткой  $p$  и передаёт в функцию  $f$ , возвращает результат  $f$  (рис. 25);
- $\text{pushSubCont } k \text{ } v$  — исполняет композицию текущего продолжения и  $k$  на значении  $v$ .

<sup>184</sup>Исторически в лиспах неограниченные продолжения были ограничены лишь REPL, отсюда название ограничений — “prompt”.

```

(λp. 2 + pushPrompt p
 if (withSubCont p
 (λk. (pushSubCont k False) + (pushSubCont k True)))
 then 3
 else 4)
newPrompt

```

Рис. 26: Пример выражения (результат — 9).

Операторы ограниченных продолжений легко понять как **resumable exceptions**, исключения, которые можно поймать, а программу возобновить с того места, где исключение было выброшено (с некоторым значением). Необычно то, что классические операторы ограниченных продолжений принимают код, работающий с частичным продолжением в месте “кидания исключения”, а не в месте “поимки”. То есть блок обработки пишется не с `catch`, а с `throw`:

$$\begin{aligned}
E_1[\text{pushPrompt } p \{E_2[\text{withSubCont } p \ f]\}] &\rightarrow E_1[f \ E_2] \\
E_1[\text{try}\{E_2[\text{throw } v]\}\text{catch}(x, k)\{t\}] &\rightarrow E_1[[x \mapsto v] [k \mapsto \lambda y. E_2[y]] \ t]
\end{aligned}$$

**Упражнение 72** Поредуцируйте пример рис. 26.

### 6.3.1 Реализация операторов

Расширим язык 6.1.3 рассмотренными операторами для работы с ограниченными. Для простоты вместо свежих меток промптов будем использовать имена, а вместо `pushSubCont` обычную аппликацию:

```

1 data Expr = ... | PushPrompt String Expr | WithSubCont String Expr
2 data Frame = ... | PushPromptFrame String | WithSubContFrame String

```

Поиск редексов расширяется очевидным образом:

```

1 evalK :: (?env :: Env) => Expr -> K -> Value
2 evalK e k = case e of
3 -- ...
4 PushPrompt promptName body -> evalK body (PushPromptFrame promptName : k)
5 WithSubCont promptName f -> evalK f (WithSubContFrame promptName : k)

```

Вся задача `PushPrompt` — это застрять в продолжении в виде `PushPromptFrame`, но когда до него доходит исполнение, он просто игнорируется:

```

1 appK :: (?env :: Env) => K -> Value -> Value
2 appK k result = case k of
3 -- ...
4 PushPromptFrame _ : k' -> k' `appK` result

```

Чтобы захватить фрагмент продолжения, нам нужна операция, которая найдёт первый промт с соответствующим именем в продолжении и вернёт **subcontinuation** — до этого промта и **metacontinuation** — после:

```

1 splitByPrompt :: K -> String -> (K, K)
2 splitByPrompt k targetPromptName = go [] k
3 where
4 go _ [] = error $ "Prompt " ++ targetPromptName ++ " not found"
5 go subcont (PushPromptFrame promptName : metacont)
6 | promptName == targetPromptName = (subcont, metacont)
7 go subcont (frame : metacont) = go (subcont ++ [frame]) metacont

```

Теперь WithSubCont должен просто применить пользовательскую функцию к subcontinuation, но при этом не забыть дальше исполнить metacontinuation:

```

1 appK :: (?env :: Env) => K -> Value -> Value
2 appK k result = case k of
3 -- ...
4 WithSubContFrame promptName : k' ->
5 let (subcont, metacont) = k' `splitByPrompt` promptName in
6 (RApp result : metacont) `appK` K ?env subcont

```

**Упражнение 73** Поредуцируйте следующий пример:

```

1 exampleDelimited = let ?env = Map.empty in flip evalK [] $
2 c 10 -. PushPrompt "p"
3 (c 5 -. WithSubCont "p"
4 (lam "k" $ v "k" @ c 1 -. v "k" @ c 3))

```

Многие классические операторы, например, shift/reset, prompt/control и т.д., можно получить, оставив фрейм промта в subcontinuation или в metacontinuation в реализации splitByPrompt Dyvbig et al. [2007].

### 6.3.2 В Monad Cont

Для примера реализуем два классических оператора для работы с ограниченными продолжениями — **shift-reset**. Воспользуемся продолжениями, собираемыми монадой Cont. shift просто захватывает текущее продолжение и передаёт его в пользовательское вычисление подобно cont:

```

1 shift :: ((a -> r) -> Cont r r) -> Cont r a
2 shift f = Cont \k -> runCont (f k) id

```

reset же в качестве продолжения вычислению-аргументу передаёт id, тем самым это вычисление не имеет доступа к продолжению после reset (в то время как продолжение самого Haskell запоминает, что после comp нужно исполнить k):

```

1 reset :: Cont a a -> Cont r a
2 reset comp = Cont \k -> k (runCont comp id)

```

**Упражнение 74** *Каким будет результат исполнения следующей функции:*

```

1 exampleShiftReset = flip runCont id $
2 (1 +) <$> reset ((2 +) <$> shift \k -> pure (k 3 + k 5))

```

## 6.4 Приложения продолжений

Продолжения полезны для понимания смысла программ, которые мы пишем каждый день (см. 137). Кроме того, как мы увидим в этой главе, продолжения первого класса можно использовать как средство построения выразительных встроенных языков.

### 6.4.1 Всё через продолжения

Реализуем ряд полезных встроенных языков с помощью продолжений первого класса. Чтобы получить доступ к продолжениям, воспользуемся shallow-embedded языком `Cont`<sup>185</sup>. Таким образом, будем работать с башней языков Haskell, `Cont`, X, где X — рассматриваемый встроенный язык.

Начнём с тривиального встроенного императивного языка:

```

1 runIdentityC :: (forall r . Cont r a) -> a
2 runIdentityC comp = runCont comp id

3 exampleIdentity :: Int
4 exampleIdentity = runIdentityC do
5 x <- pure 4
6 y <- pure 5
7 pure (x + y)

```

Реализуем язык с исключениями. Для этого модифицируем первый типовый параметр `Cont`, **answer (response) type**<sup>186</sup>. Функция `abort` игнорирует продолжение программы до `runExn`, которая завершает продолжение оборачиванием успешного результата в `Just`:

<sup>185</sup><https://blog.poisson.chat/posts/2019-10-26-reasonable-continuations.html>

<sup>186</sup>[https://wiki.haskell.org/Cont\\_computations\\_as\\_question-answering\\_boxes](https://wiki.haskell.org/Cont_computations_as_question-answering_boxes)

```

1 abort :: Cont (Maybe r) a
2 abort = Cont $ const Nothing

3 runExn :: (forall r . Cont (Maybe r) a) -> Maybe a
4 runExn comp = runCont comp Just

5 exampleExn :: Int -> Maybe Int
6 exampleExn n = runExn do
7 when (n < 0) abort
8 pure (n + 1)

```

Обратите внимание, что семантика операции обрыва вычисления полностью сосредоточена в функции `abort`, в то время как в `Monad Maybe` соответствующая функция бы отвечала лишь за конструирование элемента домена, а уже монадическое связывание — за работу с продолжениями. Имея доступ к продолжениям первого класса, нам уже не нужно реализовывать монадическое связывание вручную. Говорят, что для произвольного `m`, `Cont (m r) a` — “бесплатная” реализация монады<sup>187</sup>.

Реализация работы с ошибками через возвращение специального результата (`Either` в Haskell, `Result` в Rust, `nil` в Go...) имеет монадическую семантику (условные ветвления на каждом выбрасывают только часть продолжения), в то время как полноценный механизм исключений имеет доступ ко всему ограниченному продолжению до соответствующего блока `try-catch`, что напоминает нашу текущую реализацию. С точки зрения количества синтаксического шума на уровне исходного кода, вариант с исключениями явно предпочтительнее, а вопрос типизации исключений мы рассмотрим далее (см. 8). С точки зрения производительности всё не так очевидно<sup>188189</sup>, однако `stack unwinding` средствами рантайма как будто имеет большее пространство для оптимизаций.

Теперь попробуем вызвать продолжение несколько раз. Получим язык с недетерминизмом (или `backtracking`). Так, `choice` дважды продолжает остаток программы и аккумулирует все полученные результаты в списке:

```

1 choice :: Cont [r] Bool
2 choice = Cont \k -> k True ++ k False

3 runNondet :: (forall r . Cont [r] a) -> [a]
4 runNondet comp = runCont comp (:[[]])

5 exampleNondet :: Int -> [Int]
6 exampleNondet n = runNondet do
7 b <- choice
8 if b then pure n else pure (n - 1)

```

<sup>187</sup><https://hackage.haskell.org/package/kan-extensions-5.2/docs/Control-Monad-Codensity.html>

<sup>188</sup><https://www.serpentine.com/2011/02/25/cps-is-great-cps-is-terrible/>

<sup>189</sup><https://stackoverflow.com/questions/13835817/are-exceptions-in-c-really-slow>

Удивительно, но имея только продолжения первого класса можно реализовать даже язык с изменяемой ячейкой памяти:

```
1 -- s -> (s, a) ~ ((s, a) -> r) -> (s -> r)
2 -- ~ (a -> s -> r) -> (s -> r) ~ Cont (s -> r) a

3 get :: Cont (s -> r) s -- (s -> s -> r) -> (s -> r)
4 get = Cont \k s -> k s s

5 put :: s -> Cont (s -> r) ()
6 put s' = Cont \k _s -> k () s'

7 runStateC :: (forall r . Cont (s -> r) a) -> s -> (s, a)
8 runStateC comp = runCont comp \a s -> (s, a)

9 exampleState :: Int -> (Int, ())
10 exampleState = runStateC do
11 s <- get
12 put (s + 1)
```

Идея довольно простая: захват продолжения оставляет после себя функцию, в которую предыдущий вызов продолжения подкладывает текущее состояние. И действительно, изменяемое состояние это просто фрагмент продолжения (например, аппаратного стека), к которому мы имеем непосредственный доступ на чтение и изменение. Здесь же мы разместили в начале продолжения аппликацию, к которой, захватывая продолжение, прыгаем, чтобы получить аргумент и обновить её на новую.

**Упражнение 75** *Поредумируйте пример выше, чтобы понять, как это работает.*

Интуитивно могущество продолжений можно попробовать объяснить следующим образом: поддерживать продолжение — это основная обязанность системы исполнения языка; когда продолжения передаются пользователю, его код как бы вовлекается в деятельность runtime'a и становится его частью.

## 6.4.2 The mother of all monads

В предыдущем параграфе (6.4.1) мы реализовали возможности классических монад в языке с продолжениями. Сработает ли это для произвольной монады? Оказывается, что да<sup>190</sup><sup>191</sup> Filinski [1994].

<sup>190</sup><http://www.schoolofhaskell.com/school/to-infinity-and-beyond/pick-of-the-week/the-mother-of-all-monads>

<sup>191</sup><https://blog.poisson.chat/posts/2019-10-26-reasonable-continuations.html>

Вспомним, что монада — это конструктор типа `m`, которым мы представляем денотацию вычисления. Операции `pure` и `bind` позволяют создавать и композировать денотации. Таким образом, мы имеем вычисления, представленные в виде first-class данных.

Если наш язык предоставляет продолжения первого класса (для примера возьмём `Cont (m r)` как такой язык), мы можем определить две операции: `reflect` — исполнить монадическое вычисление в языке с продолжениями и получить результат `a`; `reify` — по вычислению в языке с продолжениями получить денотацию<sup>192</sup>. В Haskell можно это выразить как перегрузку этой пары функций для различных монад:

```
1 class MonadicReflection m where
2 -- m a -> (a -> m r) -> m r
3 reflect :: m a -> forall r . Cont (m r) a
4 -- (forall r . (a -> m r) -> m r) -> m a
5 reify :: (forall r . Cont (m r) a) -> m a
```

Например, для домена `State` реализация будет выглядеть следующим образом:

```
1 newtype State s a = State { runState :: s -> (s, a) }

2 instance MonadicReflection (State s) where
3 reflect :: State s a -> (forall r . Cont (State s r) a)
4 reflect comp = Cont \k -> State \s ->
5 let (s', a) = runState comp s in
6 runState (k a) s'

7 reify :: (forall r . Cont (State s r) a) -> State s a
8 reify comp = State $ runState (runCont comp (\x -> State (,x)))

9 exampleStateReflection :: Int -> (Int, Int)
10 exampleStateReflection = runState $ reify do
11 x <- reflect get
12 reflect $ put (x + 1)
13 pure x
```

Интуитивно это можно понимать следующим образом: реализации `bind` на самом деле не важно, ей подаётся “маленькое” продолжение (собранное с помощью рассаживания `do`-нотации), или “большое” продолжение, собранное мета-языком или монадой `Cont`. И в принципе монады можно задавать через их вложение в монаду `Cont`<sup>193</sup>.

Таким образом, если язык поддерживает продолжения первого класса, можно в `direct-стиле`<sup>194</sup> реализовать функциональность произвольной монады, без `do`-нотации, аппликатив-

<sup>192</sup>Для энергичного языка `reify` должна принимать `thunk (() -> a) -> m a`.

<sup>193</sup><https://blog.poisson.chat/posts/2019-10-27-continuation-submonads.html>

<sup>194</sup><https://www.unison-lang.org/docs/fundamentals/abilities/for-monadically-inclined/>

ных цепочек и прочего синтаксического шума<sup>195</sup><sup>196</sup>. Позже мы увидим, что этот подход, в отличие от монад, нативно поддерживает композицию (7), а также не накладывает ограничений на типизацию (8). Отсюда возникает вопрос: а нужны ли нам теперь в программировании монады?

### 6.4.3 Генераторы и корутины

Генераторы и корутины обычно определяют как вычисление, которое может быть остановлено и возобновлено снова в том же состоянии Moura and Ierusalimsky [2009]. Корутины же обобщают генераторы и, как правило, используются как примитив асинхронного программирования в виде `async/await` или других языковых конструкций Elizarov et al. [2021]. Различие синхронного и асинхронного программирования можно понимать так: в первом случае продолжениями управляет исключительно операционная система, а во втором — средства языка.

Для примера реализуем генераторы. В качестве результата генератора будем использовать ленивый список, итератор, закодированный в виде развёртки (см. 4.5):

```
1 data Box f = forall s . Box s (s -> f s)
2 data ListF a rec = Nil | Cons a rec
3 type Iterator a = Box (ListF a)

4 box2list :: Iterator a -> [a]
5 box2list (Box s next) = case next s of
6 Nil -> []
7 Cons x s' -> x : box2list (Box s' next)
```

Скрытым состоянием итератора будет ещё невыполненное продолжение. Доступ к продолжению будем получать в монаде `Cont`, где в качестве response type возьмём `GenState`. Так, операция `yield` останавливает вычисление и сохраняет продолжение в конструкторе `Yield`, а `makeGen` заканчивает данный генератор порождением конструктора `Stop`:

```
1 data GenState a = Stop | Yield a (() -> GenState a)

2 makeGen :: Cont (GenState a) () -> Iterator a
3 makeGen comp = Box
4 (\() -> runCont comp (\() -> Stop))
5 (\k -> case k () of
6 Stop -> Nil
7 Yield x k' -> Cons x k'
8)
```

<sup>195</sup><http://www.valuedlessons.com/2008/01/monads-in-python-with-nice-syntax.html>

<sup>196</sup><https://github.com/lampepfl/monadic-reflection/blob/main/TUTORIAL.md>

```

9 yield :: a -> Cont (GenState a) ()
10 yield x = cont \k -> Yield x k

```

Теперь мы можем писать код, последовательно порождающий множество результатов:

```

1 exampleGen :: [Int]
2 exampleGen = it2list $ makeGen do
3 yield 1
4 yield 2
5 yield 3

```

**Упражнение 76** Приведите вычисление `runCont exampleGen (\() -> Stop)` к нормальной форме.

## 6.5 Эффективная работа с продолжениями

На практике для реализации генераторов и корутин требуется останавливать и продолжать программу, иначе говоря, захватывать продолжения. Это нужно делать максимально эффективно.

### 6.5.1 Contiguous stack

Продолжение представляется в виде аппаратного стека. Как только требуется захватить продолжение, стек копируется в кучу (возможно, лениво)<sup>197</sup>. То есть этот подход полностью полагается на поддержку со стороны рантайма языка.

### 6.5.2 Сегментный стек

Стек вызовов представляется как связный список аллоцированных в куче сегментов, каждый соответствует ограничивающей операции. Так, не требуется делать копирования, достаточно подмены указателей. Однако, в таком случае стек нелокален, что не очень хорошо для работы кешей.

### 6.5.3 Finite state machine (FSM)

Данная реализация подразумевает автоматическую CPS трансформацию пользовательского кода средствами компилятора. Чтобы не аллоцировать большое количество замыканий, продолжения дефункционализируются и в рамках каждой функции представляются одним изменяемым объектом. Так, состояние функции целиком один раз аллоцируется в куче, а обновление кода тела функции в определённом месте реализовано как машина состояний — с помощью меток и прыжков<sup>198</sup>.

<sup>197</sup>(youtube) Иван Угланский - Java Project Loom.

<sup>198</sup><https://github.com/Kotlin/KEEP/blob/master/proposals/coroutines.md#state-machines>

Таким образом, например, реализованы корутины в Kotlin<sup>198</sup>, генераторы в C#<sup>199</sup>...

Даже в таком виде CPS остаётся тяжеловесной трансформацией, способной замедлить исполнение кода на порядки. Дело, в частности, в том, что переменные в таком подходе сложно размещать в регистрах (у функций много точек выходов и входов<sup>198</sup>), приходится постоянно записывать их в RAM — производить **spilling**<sup>200</sup>.

---

<sup>199</sup><https://csharpindepth.com/Articles/IteratorBlockImplementation>

<sup>200</sup>[https://en.wikipedia.org/wiki/Register\\_allocation](https://en.wikipedia.org/wiki/Register_allocation)

## 7 Эффекты и модульные интерпретаторы

Ранее мы признали работу со сложностью главной задачей программиста, а построение встроенных языков — основным инструментом её решения (5.1). В данной главе мы рассмотрим понятие эффекта. Оно тесно связано со встроенными языками и даст нам лучшее понимание, когда их конструировать, что это даёт, и с чем нужно быть осторожным.

Для реализации встроенных языков мы предпочли shallow embedding в форме tagless final (5.4), который максимально переиспользует возможности мета-языка и позволяет давать различные интерпретации одной программе. Далее мы исследовали процесс вычисления и извлекли понятие продолжения (6). Оказалось, что tagless final языки, которые мы строили вокруг монад, можно выразить через продолжения удобнее и проще (6.4.1, 6.4.2). В этой главе мы поймём, как это поможет решить expression problem (5.5) до конца.

С историческо-философской перспективы, теория эффектов является прочным мостом между функциональной и императивной парадигмами программирования. Они возникли порознь как Машина Тьюринга и  $\lambda$ -исчисление Чёрча, и оставались довольно изолированными школами мысли довольно долгое время. Это стало меняться в последние 30 лет, и мы стали понимать, как эти два мира дополняют друг друга. Теория эффектов до сих пор является крайне горячей темой<sup>201202</sup>.

### 7.1 Понятие эффекта

Начнём разговор от обратного, со свойства чистоты. **Чистая функция** обладает следующими свойствами:

- Её результат всегда одинаков при одинаковом наборе аргументов (никак более нетривиально не зависит ни от чего более);
- Её единственный наблюдаемый результат — её возвращаемое значение.

В целом стиль программирования с использованием чистых функций приветствуется, так как он обладает множеством хороших свойств. Так, про них можно удобно рассуждать с помощью equational reasoning; всё, что нужно для понимания кода, явно написано в этом коде; классические системы типов хорошо работают, предоставляя полноту абстракции, качественную документацию и частичную спецификацию. . . Также известно, что всё можно записать с помощью чистых вычислений, даже работу с IO Jones [2001].

Однако, используя только чистые функции, всё приходится делать вручную. В случае с IO (с состоянием аналогично) — передавать результирующий мир в аргументы раз за разом:

```
1 getList :: Int -> World -> (World, [Int])
2 getList n w | n == 0 = (w, [])
3 | otherwise =
4 let (w', x) = getInt w in
```

---

<sup>201</sup>(youtube) The Evolution of Effects — Nickolas Wu.

<sup>202</sup><https://github.com/yallop/effects-bibliography>

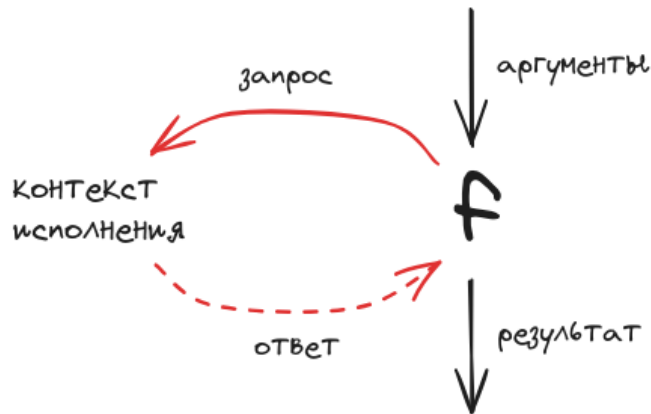


Рис. 27: Эффекты как клиент-серверное взаимодействие.

```

5 let (w', xs) = getList (n - 1) w' in
6 (w', x : xs)

```

Таким образом, код из чистых функций заполнен несущественными церемониями, за которыми не видно бизнес-логики и сути. Чтобы сосредоточиться на важных деталях, нужно делегировать весь этот bookkeeping стороннему коду, замести неинтересные детали под ковёр. Тогда код выше можно будет переписать, например, следующим образом:

```

1 getList :: Int -> IO [Int]
2 getList n | n == 0 = pure []
3 | otherwise = do
4 x <- getInt
5 xs <- getList (n - 1)
6 return (x : xs)

```

Абстрактную сущность, которой мы будем делегировать несущественные для данного фрагмента бизнес-логики детали, мы будем называть **контекстом исполнения (execution context)**<sup>203</sup>. А **эффектом** — взаимодействие функции с контекстом исполнения, которое происходит с помощью вызова **effect-операций** (например, `getInt` из примера выше)<sup>204</sup>. Так, на контекст исполнения можно смотреть как на сервер, которому функция-клиент шлёт запросы и получает ответы (см. рис. 27). В этой модели такая функция может нарушать оба свойства чистых функций.

На практике этим контекстом является интерпретатор (встроенного) языка, а effect-операциями — его конструкции. Если язык является встроенным, то говорят о **пользовательских (user-defined) эффектах**. Так мы снова возвращаемся к задаче построения модульных

<sup>203</sup><https://okmij.org/ftp/Computation/having-effect.html>

<sup>204</sup>Также эффектом иногда называют функцию перехода в dataflow-анализе Møller and Schwartzbach [2012]. По сути, она как раз описывает изменение контекста.

интерпретаторов (5.5). К тому же, реализация вычислительного контекста также может делегировать реализацию некоторой функциональности другому контексту исполнения, и так далее. Получаем уже знакомую нам башню интерпретаторов (5.1.1).

Рассмотрим некоторые примеры вычислительных контекстов и операций:

- Контекст — подсистема управления памятью, `modify` — effect-операция: контекст для нас поддерживает состояние ячеек памяти;
- Контекст — хендлер исключения, `throw MyException` — effect-операция: контекст за нас определяет, как ошибка будет обрабатываться (обратите внимание, что тут управление не возвращается терму);
- Контекст — настройки инъекции зависимостей, запрос функциональности — effect-операция: контекст за нас определяет реализацию функциональности, которой нам пользоваться<sup>205</sup>. Как мы увидим далее, этот и подобные простые эффекты можно реализовать просто и эффективно (см. 7.2.2).

### **Упражнение 77** *Приведите ещё примеры вычислительных контекстов и операций.*

Когда мы говорим про `standalone` язык, на котором мы программируем (например, Haskell), любое наше действие в программе исполняется им. То есть, например, сложение — effect-операция? В таком случае разумно выделить подмножество конструкций языка, достаточно “интересных”, чтобы считать, что они порождают эффект.

Какие конструкции языка считать “интересными”? Заметим, что с одной стороны это хорошо, что эффекты скрывают от нас некоторую сложность, позволяя сосредоточиться на других вещах. С другой стороны, это же и плохо, ведь мы эту сложность перестаём наблюдать, а она пронизывает наш код, поддерживает неявные зависимости между его частями. Таким образом, эффекты требуют дополнительной аккуратности со стороны программиста<sup>206</sup>. Соответственно, именно такие непростые конструкции и стоит считать “интересными”. Как минимум точно стоит считать “интересными” конструкции, использование которых выводит функцию из категории чистых. Также, это могут быть операции, делающие сложные нелокальные модификации потока управления (поддерживаемого интерпретатором в виде продолжения).

В конечном итоге выбор “интересных” конструкций зависит от задачи и перспективы разработчика<sup>203</sup>. Так, конструкции, влияющие на произвольные наблюдаемые свойства кода, как, например, терминируемость или вычислительная сложность, могут мотивировать считать рекурсивные вызовы или длинные операции эффектами.

Далее мы научимся отслеживать и контролировать использование эффектов на уровне типов с помощью систем эффектов (см. далее 8).

---

<sup>205</sup> Существует термин **contextual polymorphism** — код в разных контекстах может иметь различное поведение.

<sup>206</sup> (youtube) Kris Jenkins — Side-Effects Are The Complexity Iceberg.

## 7.2 Хендлеры эффектов

Хендлеры эффектов — это современный универсальный метод построения модульных интерпретаторов встроенных языков, напрямую реализующий клиент-серверную метафору. Как обычно бывает, хендлеры были изобретены множество раз. В этой главе мы посмотрим на основные реализации, которые лучше всего помогут нам понять концепцию.

Основная идея хендлеров эффектов довольно проста. Вводится языковая конструкция `handle`, позволяющая задать вычислительный контекст для определённого скоупа, предоставляющий реализации effect-операций. Также вводится конструкция `perform`, позволяющая вызвать effect-операцию (отправить запрос контексту). Каждая операция имеет набор параметров, а также “обратный адрес”, продолжение места вызова, в который она вернёт результат. Например, на экспериментальном языке Koka<sup>207</sup> контекст, предоставляющий некоторую константу, может быть реализован следующим образом (`resume` — имя продолжения места вызова, `perform` вставляется неявно):

```
1 with handler
2 ctl ask() resume(21)
3 ask() + ask()
```

Если ближайший контекст нужный запрос обработать не может, запрос делегируется внешнему контексту, и так пока подходящий контекст не будет найден. На этой идее основывается модульность интерпретаторов, заданных хендлерами.

### 7.2.1 Хендлеры через ограниченные продолжения

Как мы уже видели ранее, различные эффекты можно реализовывать с помощью доступа к текущему продолжению (6.4.1, 6.4.2). Хендлеры эффектов дополняют эту идею тем, что используют ограниченные продолжения, чтобы передавать управление различным интерпретаторам (хендлерам).

Известно, что классические операторы манипуляции ограниченными продолжениями, *monadic reflection* и хендлеры эффектов выразимы друг через друга Forster et al. [2017].

### 7.2.2 Эффективная реализация хендлеров

В общем виде скорость работы `perform` определяется скоростью захвата и восстановления ограниченных продолжений. Однако, существует класс операций, которые можно реализовать гораздо эффективнее.

Если мы посмотрим на реализацию операции `ask`, то мы увидим, что она последним действием вызывает продолжение, возвращая управление вызвавшему коду. Такие операции называют **tail-resumptive**, они очень сильно напоминают обычные функции, за исключением того, что их реализации определяются контекстом (хендлером). Таким образом, tail-

---

<sup>207</sup><https://koka-lang.github.io/koka/doc/index.html>

resumptive операции можно реализовать как неявную передачу словаря функций от хендлера к `perform`, и тем самым избежать дорогих манипуляций продолжениями Xie et al. [2020]<sup>208</sup>.

### 7.2.3 Встроенные хендлеры как явная клиент-серверная коммуникация

Чтобы лучшим образом понять семантику хендлеров, реализуем язык с хендлерами как встроенный в Haskell. Начнём с варианта, предложенного Олегом Киселёвым, максимально прямолинейно кодирующего идею клиент-серверной коммуникации термина и контекста Kiselyov et al. [2013].

Начнём с эффекта `ask`, запрашивающего числа у контекста. Зададим тип данных сообщений к контексту, это либо конечный результат вычисления, либо запрос `Ask`, содержащий “обратный адрес” — текущее продолжение:

```
1 data Message res = Val res | Ask (Int -> Message res)
```

Продолжения будем собирать в специализированный `Monad Cont` с подходящим `response type` (см. 6.2.2, 6.4.1):

```
1 newtype Eff res = Eff
2 { runEff :: forall res' . (res -> Message res') -> Message res' }
```

Тогда эффект `ask` реализуется просто как “отправка” запроса `Ask` с текущим продолжением.<sup>209</sup>

```
1 ask :: Eff Int
2 ask = Eff \k -> Ask k
```

Хендлер мы реализуем как “сервер”, который в цикле обрабатывает запросы, пока вычисление не пришлёт конечный результат:

```
1 run :: Eff res -> Message res
2 run comp = runEff comp Val

3 runReader :: Eff res -> Int -> res
4 runReader comp env = loop (run comp)
5 where
6 loop = \case
7 Val res -> res
8 Ask k -> loop (k env)
```

Наконец, мы можем писать `effectful` код:

---

<sup>208</sup>Хендлеры `tail-resumptive` операций напоминают `co-pattern-matching` и, соответственно, объекты (см. 4.6.2).

<sup>209</sup>Ранее мы аналогично реализовывали генераторы, см. 6.4.3.

```

1 exampleReader :: Int -> Int
2 exampleReader = runReader do
3 x <- ask
4 y <- ask
5 pure (x + y)

```

Монада **Eff** превратит его в ленивый список сообщений:

```

1 exampleReader :: Int -> Int
2 exampleReader = loop $
3 Ask \x ->
4 Ask \y ->
5 Val (x + y)

```

## 7.2.4 Расширяемые сообщения и пересылка

Абстрагируем тип сообщений по “форме” запросов, которые в них могут участвовать (см. 4.3.2):

```

1 data Message effs res = Val res | Request (effs (Message effs res))

```

Предыдущий тип сообщений получается передачей следующего функтора формы:

```

1 newtype Reader env msg = Ask (env -> msg)

```

**Упражнение 78** Убедитесь, что **Message (Reader Int) res** эквивалентно предыдущему типу сообщений.

Копроизведение функторов формы является функтором формы (см. 5.5.1):

```

1 data (eff |> effs) a = L (eff a) | R (effs a)

```

Теперь операция **ask** допускает существование других типов запросов **effs**:

```

1 ask :: Eff (Reader env |> effs) env
2 ask = Eff \k -> Msg $ L $ Ask k

```

Новый хендлер обрабатывает только часть запросов, остальные пересылает хендлеру выше (скомпозировав правильным образом продолжения):

```

1 runReader
2 :: forall effs env res . Functor effs
3 => Eff (Reader env |> effs) res
4 -> env -> Eff effs res
5 runReader comp env = loop (run comp)
6 where

```

```

7 loop :: Request (Reader env |> effs) res -> Eff effs res
8 loop = \case
9 Val res -> pure res
10 Msg (L (Ask k)) -> loop (k env)
11 Msg (R unknownReq) -> do
12 response <- Eff \k -> Msg (fmap k unknownReq)
13 loop response

```

Заметьте, что в результирующем домене остались непроинтерпретированные эффекты. Конечный домен получится применением всех необходимых интерпретаторов. Так решается *stable denotations problem* (5.5).

### 7.2.5 Свободные монады

Чтобы получить полноценное решение более простым и минималистичным образом, перейдём к самой классической реализации хендлеров через свободные монады. Для начала обсудим сами свободные монады.

Рассмотрим некоторую алгебраическую структуру, например, моноид (нейтральный элемент и ассоциативная бинарная операция). По произвольному множеству  $X$  можно построить некоторый моноид  $M(X)$  наиболее “экономичным” образом — **свободный моноид**. Это делается следующим образом: к множеству  $X$  добавляют деревья выражений с операциями моноида:

```

1 data M x = Element x | Mempty | Mappend (M x) (M x)
2 instance Monoid (M x) where
3 mempty = Mempty
4 mappend l r = Mappend l r

```

Ещё только нужно организовать новое множество таким образом, чтобы в нём не было одинаковых с точки зрения алгебры деревьев (например, `Mappend Mempty Mempty` и `Mempty`). В случае моноида в этом случае можно выбрать тип списка:

```

1 type M x = [x]
2 instance Monoid (M x) where
3 mempty = []
4 mappend = (++)

```

Свободные монады строятся аналогично свободным моноидам. Используем определение монады как функтора с операциями `pure` и `join`. Тогда по произвольному функтору  $F$  можно получить монаду  $Free(F)$  (покажем это с помощью эквивалентного определения монад из Haskell):

```

1 data Free f a = Pure a | Join (f (Free f a))
2 instance Functor f => Monad (Free f) where

```

```

3 return = Pure
4 Pure x >>= k = k x
5 Join f >>= k = Join (fmap (>>= k) f)

```

Переименуем конструкторы:

```

1 data Term sig var = Var var | Op (sig (Term sig var))
2 (>>=) :: Term sig var -> (var -> Term sig var') -> Term sig var'
3 data MonoidSig subtree = Mempty | MAppend subtree subtree -- (Bool -> subtree)

```

Это не что иное, как кодирование алгебраических термов над сигнатурой `sig` и переменными из множества `var`. А монадическое связывание — это подстановка.

## 7.2.6 Хендлеры через свободные монады

Заметим, что следующие типы изоморфны: `Message`  $\cong$  `Free`  $\cong$  `Term`. Хендлеры в классическом виде как раз и возникли в процессе изучения того, как описывать эффекты в виде алгебраических структур Bauer [2018]<sup>210</sup>.

Вместо переменных мы будем хранить чистый результат вычисления, а сигнатуры будем записывать в виде  $P \times (A \rightarrow K)$ , где  $P$  — параметр операции, а  $A$  — результат операции, по которому хендлер выбирает нужный подтерм-продолжение для возобновления вычисления:

```

1 data Comp effs res = Res res | Op (effs (Comp effs res))
2 data Reader env comp = Ask () (env -> comp)
3 data State s comp = Get () (s -> comp) | Put s (() -> comp)

```

Реализация `Monad Comp` будет как раз композировать продолжения для нас в `do`-нотации:

```

1 ask = Op (Ask () (\e -> Val e)
2 example = do
3 x <- ask
4 y <- ask
5 pure (x + y)
6 -- $\alpha\beta$ -эквивалентно
7 example =
8 Op (Ask () (\x ->
9 Op (Ask () (\y ->
10 Res (x + y))))))

```

Хендлер также просто сворачивает список операций. Операции, которые он не умеет обрабатывать, он оставляет в дереве. Чтобы пропустить неизвестную операцию и интерпретировать поддереву (продолжение), используется `fmap`:

<sup>210</sup>(youtube) What is algebraic about algebraic effects and handlers — Andrej Bauer.

```

1 runReader
2 :: Functor effs => Comp (Reader env |> effs) res -> env -> Comp effs res
3 runReader comp env = case comp of
4 Res res -> Res res
5 Op (L (Ask () k)) -> runReader (k env) env
6 Op (R other) -> Op (fmap (`runReader` env) other)

```

**Упражнение 79** Постройте пример вычисления из **Reader** и **State**. Какое дерево получится после интерпретации одного из эффектов?

Как обычно, свёртку можно обобщить в виде катаморфизма:

```

1 handle
2 :: Functor effs => (res -> d) -> (effs d -> d) -> Comp effs res -> d
3 handle val alg = \case
4 Pure res -> val res
5 Op eff -> alg $ fmap (handle val alg) eff

```

Теперь в реализации продолжение (поддерево) уже проинтерпретировано в нужный домен:

```

1 runReader
2 :: Functor effs => Eff (Reader env |> effs) res -> env -> Eff effs res
3 runReader = handle (\res _env -> pure res) \case
4 L (Ask k) -> \env -> k env env
5 R other -> \env -> Op (fmap ($ env) other)

```

Когда хендлеры реализуют как built-in возможность в языке, нужно принять дизайн-решение, в предоставляемом продолжении текущий эффект уже проинтерпретирован, или нет. Первый вариант называют **deep handlers**, второй — **shallow handlers**, они выразимы друг через друга Hillerström and Lindley [2018].

К сожалению, поскольку различные варианты эффектов упорядочены как на уровне термов, так и на уровне типов, нам нужна операция, превращающая вычисление от меньшего количества эффектов в вычисление с большим:

```

1 liftF :: Functor effs => Comp effs res -> Comp (eff |> effs) res
2 liftF = \case
3 Pure x -> Pure x
4 Op effs -> Op $ R $ liftF <$> effs

5 example :: Comp (Reader Int |> State Int) ()
6 example = do
7 env <- ask
8 liftF (put env)

```

От порядка можно избавиться с помощью классов типов Swierstra [2008] (3.4.8).

Можно заметить, что помимо большого количества аллокаций, использование `Monad Free` может приводить к квадратичной сложности кода из-за линейных проходов в каждом `bind`'е. Существуют различные альтернативные схемы кодирования Ploeg and Kiselyov [2014], Kiselyov and Ishii [2015].

Свободные монады находят и другие, правда, аналогичные применения: trampolining Bjarnarson [2012] и пайплайны Kiselyov [2012b] [Bragilevsky, chapter 14].

### 7.2.7 Приложения хендлеров

Рассмотрим хендлеры `tail-resumptive` операций, обращающиеся с продолжениями тривиальным образом. Они нужны для распространения значений и функциональности вниз по стеку. Как мы обсуждали ранее 7.2.2, такие хендлеры аналогичны наличию динамических свободных переменных или неявных аргументов функций 3.1.2, и, вместе с ними, рекордов или анонимных классов 4.6.2.

Среди нетривиальных сценариев использования продолжений выделим следующие:

- Продолжение можно не вызвать, так, можно реализовать механизм исключений. Однако, на практике продолжения могут содержать логику финализации ресурсов. В этом случае всё равно нужна какая-то специальная обработка <sup>211</sup>.
- Можно вызвать несколько раз для эмуляции недетерминизма. Это ограничивает использование эффективных мутабельных продолжений 6.5 Leijen [2018]<sup>212</sup>, а также требует нетривиальной обработки при работе с ресурсами.
- Можно вызвать не сразу. Это нужно для реализации изменяемого состояния, генераторов и корутин. Для изменяемого состояния это слишком дорого. Генераторы — хорошее применение, но `built-in` реализация генераторов может быть эффективнее за счёт экономии аллокаций продолжений (создаётся сразу `Iterator`)<sup>213</sup>. Реализация корутин поверх хендлеров даёт возможность пользователям писать собственные планировщики, что самое котируемое применение хендлеров на данный момент Sivaramakrishnan et al. [2021], Phipps-Costin et al. [2023].

### 7.2.8 Трансформеры монад

В классической реализации хендлеров через свободные монады 7.2.6, каждый хендлер порождает промежуточное дерево из непроинтерпретированных операций. Естественным образом возникает желание дефорестировать 4.4.1 эти промежуточные деревья. Можно пойти до конца и полностью избавиться от свободных монад и функторов сигнатур Wu and Schrijvers [2015]. Так, мы фактически получим `tagless final shallow embedding` 5.4, другое популярное решение `expression (stable denotations) problem` — **трансформеры монад** Liang et al. [1995],

<sup>211</sup><https://koka-lang.github.io/koka/doc/book.html#sec-resource>

<sup>212</sup><https://koka-lang.github.io/koka/doc/book.html#sec-multi-resume>

<sup>213</sup><https://csharpindepth.com/Articles/IteratorBlockImplementation>

Jones [1995]<sup>214</sup>. В целом эти подходы равносильны по выразительности, однако могут иметь на практике различные особенности встраивания в язык Schrijvers et al. [2019]. Однако, в отличие от трансформеров, хендлеры можно сделать удобной built-in возможностью языка.

## 7.2.9 Алгебраичность и эффекты высших порядков

В начале нулевых появилась идея описывать effect-операции не монадами сразу, а алгебраически, с помощью сигнатур и уравнений Plotkin and Power [2002], Bauer [2018]. Эффекты в этом формализме являются композируемыми по построению, как композируемы сигнатуры алгебраических теорий (конкатенация сигнатур — сигнатура 7.2.4), так мы имеем расширяемый синтаксис. Однако, на операции накладывается ограничение в виде **свойства алгебраичности** — операция коммутрует с продолжением и “всплывает” вверх:

$$E[op(v, k)] \equiv op(v, \lambda x. E[k(x)])$$

Соответствующие эффекты называют **алгебраическими**.

В нашей реализации (7.2.6) ограничение на алгебраичность можно увидеть следующим образом. У нас сигнатуры операций являются функторами по своим продолжениям. Этим пользуются как монадическое связывание для накапливания продолжений, так и хендлеры для интерпретации продолжений неизвестных операций.

Многие полезные эффекты являются алгебраическими, но не все. Например, поимка исключений `catch` не является алгебраической операцией, так как её сигнатура запишется следующим образом:

```
1 data Catch e comp = Catch { try :: comp, onExn :: (e -> comp), next :: comp }
```

В таком случае единственная разумная реализация функтора будет работать, в том числе, и с вложенными продолжениями, и вся программа, в результате работы `bind`, окажется внутри блока `try`.

Иначе говоря, *алгебраические эффекты не могут принимать другие `effectful` вычисления в качестве аргументов*. Как раз, чтобы моделировать `catch`, и были оригинально предложены хендлеры Plotkin and Pretnar [2013]<sup>215</sup>. Таким образом, у хендлеров две задачи: ограничивать скоуп некоторой функциональности и интерпретировать эффекты. Однако, как мы знаем, порядок хендлеров определяет результирующий домен и, соответственно, семантику. В то же время задача ограничения скоупа фиксирует позицию хендлера, что ограничивает выразительность и делает некоторые домены недоступными.

Операции, не удовлетворяющие свойству алгебраичности, соответствуют **эффектам высших порядков (higher-order effects)**, которые могут принимать другие `effectful` вычисления в качестве аргументов. Существуют встраивания хендлеров таких эффектов и обширный набор исследований на тему Wu et al. [2014]<sup>216</sup><sup>217</sup> Yang et al. [2022].

<sup>214</sup><https://hackage.haskell.org/package/mtl>

<sup>215</sup>Несмотря на то, что алгебраические эффекты — это просто класс “хороших” операций, часто при их упоминании подразумевают хендлеры эффектов.

<sup>216</sup><https://github.com/fused-effects/fused-effects>

<sup>217</sup>(youtube) Building Haskell Programs with Fused Effects — Patrick Thomson

Ключ к эффектам высших порядков состоит в возможности исполнять эффекты вычислений-аргументов в контексте тех хендлеров, которые доступны на колсайте эффекта высших порядков van der Rest et al. [2022]. Это связано с другой идеей, *bidirectional effects*, позволяющей реализации операции порождать эффекты на своём колсайте (например, бросать там исключения), что крайне необходимая на практике возможность Zhang et al. [2020]<sup>218</sup>.

---

<sup>218</sup><https://effekt-lang.org/docs/concepts/bidirectional>

## 8 Системы эффектов TODO

TODO

## 9 Datatype-generic programming TODO

Ранее мы говорили о том, что всё, что пишут программисты, — интерпретаторы. Оказывается, многие из них можно получать автоматически, выйдя на новый уровень обобщения. А код, который может не быть написан, — не должен быть написан.

Как правило, такая ситуация возникает, когда интерпретация полностью определяется описанием (формой) типа. То есть тем, какие конструкторы данных для него определены, с какими полями, как они называются. . . . Такими интерпретациями являются, например, сериализация в конкретный формат (e.g. json), структурное равенство и т.д.

Решением является поддержка в языке **структурного полиморфизма (structural polymorphism)** [Макарова, глава 13], или, как это иногда называют, **datatype-generic программирования**.

Можно вообразить язык, в котором data-generic программирование является обычным программированием Chapman et al. [2010]. Для этого берётся язык с зависимыми типами, с выделенной вселенной описаний **Desc**. Конкретные типы же получаются применением функции интерпретации **Desc -> Type**. Таким образом, параметрический полиморфизм воплощается обобщением по типу, а структурный полиморфизм — обобщением по его описанию.

TODO

### 9.1 Специализация

$$\llbracket \text{spec} \rrbracket(p, x)(y) \equiv \llbracket p \rrbracket(x, y)$$

TODO Liu [2023]

### 9.2 Ad-hoc решения

TODO

### 9.3 Подходы к datatype-generic в Haskell

TODO

#### 9.3.1 Template Haskell

TODO<sup>219</sup>

#### 9.3.2 Deriving strategies

TODO<sup>220</sup>

---

<sup>219</sup><https://markkarpov.com/tutorial/th.html>

<sup>220</sup>[https://ghc.gitlab.haskell.org/ghc/doc/users\\_guide/exts/deriving\\_strategies.html](https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/deriving_strategies.html)

### 9.3.3 GHC.Generics

TODO<sup>221</sup>

### 9.3.4 Sum Of Products

TODO

### 9.3.5 Uniplat

TODO

---

<sup>221</sup>[https://ghc.gitlab.haskell.org/ghc/doc/users\\_guide/exts/generics.html](https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/generics.html)

## 10 Персистентность и оптика TODO

В функциональном программировании принято персистентно работать с данными. В этой главе мы рассмотрим, когда и почему это важно. А также — функциональную оптику — набор абстракций для удобной работы с данными.

### 10.1 Персистентные структуры данных

Структуры данных разделяют на **изменяемые (mutable)** и **неизменяемые (immutable)**. Это классификация лишь по внутренней реализации, используются изменяемые ячейки памяти или нет.

По использованию выделяют **эфемерные** и **персистентные** структуры данных. Изменения эфемерной структуры данных можно наблюдать по той же ссылке:

```
1 let xs = MutableList<Int>.empty()
2 xs.add(42)
3 print(xs)
```

В то время как изменение персистентной структуры каждый раз порождает новые ссылки, а по старым остаются доступны предыдущие версии структуры:

```
1 let xs = []
2 let xs' = 42 : xs
3 print xs
```

Может показаться, что эфемерные являются изменяемыми, а персистентные — неизменяемыми. На самом деле это более-менее ортогональные классификации. Так, с помощью копирования можно к изменяемой структуре данных предоставить персистентный интерфейс, а неизменяемой — эфемерный (с помощью монады **State**).

В речи, когда говорят о персистентных структурах данных, часто имеют в виду структуры данных с персистентным интерфейсом, специально оптимизированные для него (не требуют полного копирования на каждую операцию). Такие структуры данных бывают на удивление оптимальными и разнообразными Okasaki [1999].

Например, можно реализовать эффективный персистентный массив с логарифмической сложностью всех операций. В Haskell такой структурой данных является **Seq**<sup>222223</sup>. Если в вершинах хранить небольшие массивы, которые современные архитектуры процессоров могут эффективно копировать, можно существенно уменьшить высоту дерева и алгоритмическую сложность операций (e.g. `scala.immutable.Vector`).

**Упражнение 80** Реализуйте на Haskell персистентное декартово дерево по неявному ключу. Какие особенности Haskell усложняют использование этой структуры?

<sup>222</sup><https://hackage.haskell.org/package/containers-0.7/docs/Data-Sequence.html>

<sup>223</sup><http://www.staff.city.ac.uk/~ross/papers/FingerTree.html>

Когда использовать эфемерные, а когда персистентные структуры данных? Если сравнивать, то можно обнаружить, что эфемерные можно реализовывать эффективнее во многих случаях: кеш процессоров лучше работает с локальными данными, меньше аллокаций и индирекций. В то же время персистентные структуры обречены быть деревьями, чтобы реаллоцировать не структуру целиком, а только путь до корня.

Персистентные структуры же позволяют писать более модульный и безопасный с точки зрения многопоточности код, который может не учитывать возможность изменения структуры по ссылке. В то время как работа с эфемерной структурой не является чистым кодом, а включает в себя порождение наблюдаемых побочных эффектов. Также объединение персистентных результатов разных вызовов может быть дешевле, как, например, конкатенация персистентных массивов дешевле конкатенации эфемерных (логарифмическая сложность против линейной).

Таким образом, в рамках ограниченного, легко обозреваемого скоупа лучше использовать эфемерные структуры ввиду их эффективности (например, чтобы изначально заполнить коллекцию элементами). Однако, через границы абстракции лучше пропускать только персистентные структуры (или же положиться на систему эффектов, см. 8). То есть каждая структура данных должна поддерживать две фазы своей жизни. Например, так сделано в Scala, где у многих персистентных коллекций есть [Builder](#) версия.

## 10.2 Идея оптики

**Функциональная оптика** позволяет строить функциональные ссылки, фокусирующиеся на определённых свойствах структур данных. В дальнейшем, с помощью элиминирующих функций, можно по функциональной ссылке и объекту прочесть свойство или персистентно обновить его. Например, мы можем сфокусироваться на первой компоненте пары с помощью ссылки `_1` и прочесть первую компоненту:

```
1 ghci> view _1 (42, 43)
2 42
```

Или можем сфокусироваться на свойстве наличия в множестве определённого элемента `member`, выставить свойство в `True` и получить новое множество с нужным элементом:

```
1 ghci> set (member 42) (Set.fromList [1, 2]) True
2 Set.fromList [1, 2, 42]
```

Функциональные ссылки композируются (тут, с помощью `%`). Так, если множество будет в первой компоненте пары, мы всё ещё можем добавить в него элемент:

```
1 ghci> set (_1 % member 42) (Set.fromList [1, 2], 3) True
2 (Set.fromList [1, 2, 42], 3)
```

Важно заметить, что функциональные ссылки, в отличие от обычных указателей, с одной стороны более абстрактны и не обязательно просто ссылаются на область памяти, а с

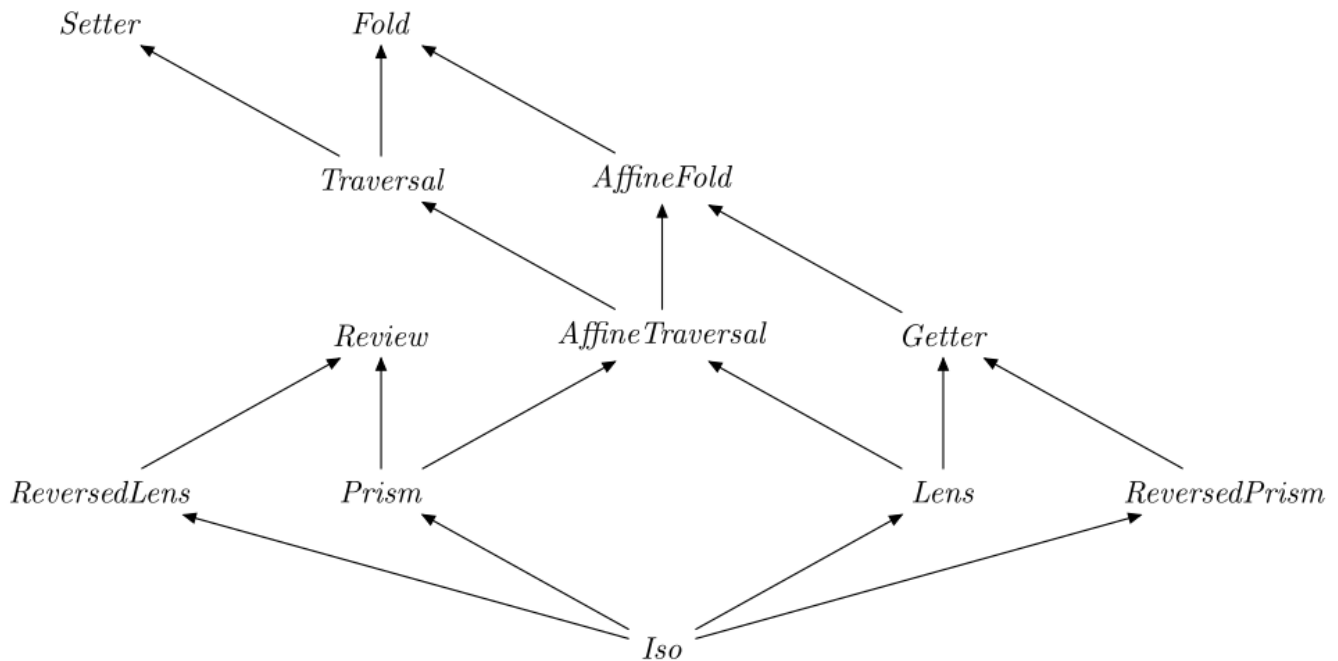


Рис. 28: Иерархия оптических девайсов библиотеки `optics`.

другой — отделены от конкретного объекта, чем скорее напоминают указатели на методы в C++.

Таким образом, оптика помогает приблизить удобство программирования с персистентными структурами данных к удобству программирования с мутабельными. Это важно, поскольку хорошие практики программирования должны быть как минимум так же доступны, как и менее оптимальные.

Функциональные ссылки могут предоставлять различные интерфейсы использования, которые называют **оптическими девайсами**. Девайсы образуют иерархию (рис. 28): при композиции двух девайсов, результирующий является ближайшим общим предком с функциональностью пересечения функциональностей исходных.

Обе ссылки выше являются оптическим девайсом **линза**, который исторически появился первым. Изначально линзы были предложены как решение проблемы view-update в базах данных Bohannon et al. [2006], Foster et al. [2008]. А именно — как нужно изменить реальную базу при изменении view. Или как восстанавливать целое после извлечения и изменения части. Линзы тут являются средством двустороннего программирования — позволяют одновременно описывать как view, так и способ обновления. Параллельно<sup>224</sup>, линзы упоминаются в серии блог-постов, описывающих попытку удобнее работать с изменяемым состоянием при написании игр<sup>225226</sup> (линзы в играх продолжают радовать<sup>227</sup>).

<sup>224</sup><https://github.com/ekmett/lens/wiki/History-of-Lenses>

<sup>225</sup>(post) Making Haskell nicer for game programming

<sup>226</sup>(post) Haskell State Accessors (second attempt: Composability)

<sup>227</sup><http://www.timphilipwilliams.com/posts/2019-07-25-minecraft.html>

Существует множество библиотек оптики как для Haskell<sup>228229230</sup>, так и для других языков: Kotlin<sup>231</sup>, Scala<sup>232</sup>, Swift<sup>233</sup> (языковая поддержка).

## 10.3 Использование оптики

Рассмотрим для примера использование нескольких оптических девайсов библиотеки `optics`, имеющей прекрасную документацию<sup>234</sup>.

Примеры:

```
1 data Pet = Cat String | Dog Int deriving Show
2 newtype UserName = UserName { _getUserName :: String } deriving Show
3 data User = User { _userName :: UserName, _userCats :: [Pet] } deriving Show

4 exampleUser :: User
5 exampleUser = User (UserName "Bob") [Cat "Kitty", Dog 42, Dog 4]

6 exampleName :: User -> User
7 exampleName = over (userName % getUserName) (++ "!")

8 exampleIx :: User -> User
9 exampleIx = set (userName % getUserName % ix 2) 'k'

10 exampleFold :: User -> Int
11 exampleFold = getSum $ foldMapOf (userName % getUserName % folded) (const 1)

12 examplePrint :: User -> IO User
13 examplePrint = traverseOf (userCats % traversed % _Dog) (\x -> x <$ print x)
```

TODO

## 10.4 Data optics

### 10.4.1 Линзы

Простейшую линзу образуют пара функций — просмотр и установка свойства:

---

<sup>228</sup><https://hackage.haskell.org/package/optics-0.1/docs/Optics.html>

<sup>229</sup><http://lens.github.io/>

<sup>230</sup><https://github.com/marcosh/existential-optics/tree/main>

<sup>231</sup><https://arrow-kt.io/>

<sup>232</sup><https://github.com/optics-dev/Monocle>

<sup>233</sup><https://github.com/swiftlang/swift-evolution/blob/main/proposals/0161-key-paths.md>

<sup>234</sup><https://hackage.haskell.org/package/optics-0.4.2.1/docs/Optics.html>

```

1 data SimpleLens' s a = SimpleLens'
2 { view' :: s -> a
3 , set' :: s -> a -> s
4 }

```

На эти функции накладываются естественные законы:

```

1 view l (set l s x) ≡ x
2 set l s (view l s) ≡ s
3 set l (set l s x) y ≡ set l s y

```

Например, можно легко изменить возраст пользователя:

```

1 newtype Age = Age { _getAge :: Int }
2 data User = User { _userName :: String, _userAge :: Age }

3 userAge :: SimpleLens' User Age
4 userAge = SimpleLens' { view' = _userAge, set' = \s x -> s { _userAge = x } }

5 getAge :: SimpleLens' Age Int
6 getAge = SimpleLens' { view' = _getAge, set' = \s x -> s { _getAge = x } }

7 ghci> set (userAge % getAge) user 1

```

Можно обобщить линзы до полиморфных линз, которые позволяют пересоздавать структуру с новыми типовыми параметрами:

```

1 data SimpleLens s t a b = SimpleLens
2 { view :: s -> a
3 , set :: s -> b -> t
4 }

5 _1 :: SimpleLens (a, c) (b, c) a b
6 _1 = SimpleLens { view = \(x, _) -> x, set = \(x, c) y -> (y, c) }

```

Можно заметить, что линза в таком представлении является комонадой коалгеброй  $\text{kostate}^{235}$ . Действительно, пару функций можно заменить на функцию, возвращающую пару:

```

1 data Store a s = Store (a, a -> s)
2 data DataLens s a = DataLens (s -> Store a s)

```

Если определить инстанс комонады для `Store a s` и выписать законы совместимости с коалгеброй, мы получим в точности законы линз.

TODO

---

<sup>235</sup>(youtube) Category Theory II 9.1: Lenses.

### 10.4.2 Призмы

Призмы позволяют получить свойство, которое может отсутствовать, и устанавливать его при наличии. Так, конкретное поле типа-суммы может не удастся извлечь, при передаче не ожидаемого конструктора.

Простые призмы можно определить аналогично линзам, с учётом возможного отсутствия свойства:

```
1 data SimplePrism' s a = SimplePrism'
2 { preview' :: s -> Maybe a
3 , review' :: a -> s
4 }
```

Полиморфная призма должна предоставить структуру с обновлённым типом в случае неуспеха просмотра:

```
1 data SimplePrism s t a b = SimplePrism
2 { preview :: s -> Either t a
3 , review :: b -> t
4 }
```

Можно определить призму для работы с содержимым конструктора `Left`:

```
1 _Left :: SimplePrism (Either a c) (Either b c) a b
2 _Left = SimplePrism
3 { preview = \case Left a -> Right a; Right c -> Left (Right c)
4 , review = Left
5 }
```

Композиция призм определяется естественным образом.

### 10.4.3 Композиция линз и призм

Заведём класс типов, с помощью которого научимся композировать различные комбинации линз и призм. Результирующий девайс однозначно определяется операндами композиции.

```
1 class Composable o1 o2 o3 | o1 o2 -> o3 where
2 compose :: o1 s t a b -> o2 a b c d -> o3 s t c d
```

Теперь можем естественным образом определить композицию для различных девайсов. Можно заметить, что этот подход требует квадратичного количества инстансов по числу оптических девайсов.

```

1 instance Composable SimpleLens SimpleLens SimpleLens where
2 compose :: SimpleLens s t a b -> SimpleLens a b c d -> SimpleLens s t c d
3 compose l2 l1 = SimpleLens
4 { view = view l1 . view l2
5 , set = \s x -> set l2 s (set l1 (view l2 s) x)
6 }

```

Композиция линз и призм образует другой девайс — аффинные траверсы, которые являются чем-то средним. Наличие призмы в композиции не позволяет гарантированно просматривать свойство, а линзы — создавать новое значение без изначального.

```

1 data AffineTraversal s t a b = AffineTraversal
2 { apreview :: s -> Either t a
3 , aset :: s -> b -> t
4 }

5 instance Composable SimplePrism SimpleLens AffineTraversal where
6 compose :: SimplePrism s t a b -> SimpleLens a b c d -> AffineTraversal s t c d
7 compose p l = AffineTraversal
8 { apreview = fmap (view l) . preview p
9 , aset = \s d -> case preview p s of
10 Left t -> t
11 Right a -> review p (set l a d)
12 }

```

## 10.5 Путь к profunctor optics

TODO

## 10.6 Генерация оптики

TODO

*To be continued...*

## Список литературы

- Christopher Strachey. Fundamental concepts in programming languages. *Higher-order and symbolic computation*, 13:11–49, 2000. URL <https://facweb.cdm.depaul.edu/smitsch/courses/csc447fa23/assets/articles/strachey-fundamental-concepts-in-programming-languages.pdf>.
- Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys (CSUR)*, 17(4):471–523, 1985. URL <https://doi.org/10.1145/6041.6042>.
- Benjamin C Pierce. *Types and programming languages*. MIT press, 2002.
- John Launchbury and Simon L Peyton Jones. State in haskell. *Lisp and symbolic computation*, 8(4): 293–341, 1995. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/state-lasc.pdf>.
- Sandy Maguire. *Thinking with Types, Type-Level Programming in Haskell*. a. URL <https://leanpub.com/thinking-with-types/>
- Simon Peyton Jones. Type inference as constraint solving: how ghc's type inference engine actually works. Keynote talk at Zurihac 2019, 2019. URL <https://simon.peytonjones.org/type-inference/>.
- Benjamin C Pierce and David N Turner. Local type inference. *Acm transactions on programming languages and systems (toplas)*, 22(1):1–44, 2000. URL <https://doi.org/10.1145/345099.345100>.
- David Raymond Christiansen. Bidirectional typing rules: A tutorial. 2013. URL <https://davidchristiansen.dk/tutorials/bidirectional.pdf>.
- Jana Dunfield and Neelakantan R Krishnaswami. Sound and complete bidirectional typechecking for higher-rank polymorphism with existentials and indexed types. *Proceedings of the ACM on Programming Languages*, 3(POPL):1–28, 2019. URL <https://doi.org/10.1145/3290322>.
- Alejandro Serrano, Jurriaan Hage, Simon Peyton Jones, and Dimitrios Vytiniotis. A quick look at impredicativity. *Proceedings of the ACM on Programming Languages*, 4(ICFP):1–29, 2020. URL <https://www.microsoft.com/en-us/research/publication/a-quick-look-at-impredicativity/>.
- Frank Emrich, Sam Lindley, Jan Stolarek, James Cheney, and Jonathan Coates. Freezeml: Complete and easy type inference for first-class polymorphism. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 423–437, 2020. URL <https://link.springer.com/article/10.1208/s12249-010-9382-3>.

- Brent A Yorgey, Stephanie Weirich, Julien Cretin, Simon Peyton Jones, Dimitrios Vytiniotis, and José Pedro Magalhães. Giving haskell a promotion. In *Proceedings of the 8th ACM SIGPLAN Workshop on Types in Language Design and Implementation*, pages 53–66, 2012. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/p53-yorgey.pdf>.
- Stephanie Weirich, Justin Hsu, and Richard A Eisenberg. System fc with explicit kind equality. *ACM SIGPLAN Notices*, 48(9):275–286, 2013. URL <https://doi.org/10.1145/2544174.2500599>.
- Vitaly Bragilevsky. *Haskell in Depth*. Manning. URL <https://www.manning.com/books/haskell-in-depth>.
- Bruno Blanchet. Escape analysis for object-oriented languages: application to java. *Acm Sigplan Notices*, 34(10):20–34, 1999. URL <https://doi.org/10.1145/320385.320387>.
- Richard A Eisenberg and Simon Peyton Jones. Levity polymorphism. *ACM SIGPLAN Notices*, 52(6):525–539, 2017. URL <https://doi.org/10.1145/3140587.3062357>.
- Paul Hudak, John Hughes, Simon Peyton Jones, and Philip Wadler. A history of haskell: being lazy with class. In *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, pages 12–1, 2007. URL [https://www.researchgate.net/profile/Simon-Peyton-Jones/publication/221501761\\_A\\_history\\_of\\_Haskell\\_Being\\_lazy\\_with\\_class/links/0c960517e31f50f743000000/A-history-of-Haskell-Being-lazy-with-class.pdf](https://www.researchgate.net/profile/Simon-Peyton-Jones/publication/221501761_A_history_of_Haskell_Being_lazy_with_class/links/0c960517e31f50f743000000/A-history-of-Haskell-Being-lazy-with-class.pdf).
- Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 60–76, 1989. URL <https://doi.org/10.1145/75277.75283>.
- Cordelia V Hall, Kevin Hammond, Simon L Peyton Jones, and Philip L Wadler. Type classes in haskell. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 18(2):109–138, 1996. URL <https://doi.org/10.1145/227699.227700>.
- Simon Peyton Jones, Mark Jones, and Erik Meijer. Type classes: an exploration of the design space. In *In Haskell Workshop*, 1997. URL <https://courses.cs.washington.edu/courses/cse590p/06sp/multi.pdf>.
- Jeffrey R Lewis, John Launchbury, Erik Meijer, and Mark B Shields. Implicit parameters: Dynamic scoping with static types. In *Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 108–118, 2000. URL <https://doi.org/10.1145/325694.325708>.
- Martin Sulzmann, Gregory J Duck, Simon Peyton-Jones, and Peter J Stuckey. Understanding functional dependencies via constraint handling rules. *Journal of functional programming*, 17(1): 83–129, 2007a. URL <https://doi.org/10.1017/S0956796806006137>.

- Oleg Kiselyov and Chung-chieh Shan. Functional pearl: implicit configurations—or, type classes reflect the values of types. In *Proceedings of the 2004 ACM SIGPLAN workshop on Haskell*, pages 33–44, 2004. URL [https://d1wqtxts1xzle7.cloudfront.net/43582096/Functional\\_pearl\\_implicit\\_configurations20160310-32037-1bu6179-libre.pdf](https://d1wqtxts1xzle7.cloudfront.net/43582096/Functional_pearl_implicit_configurations20160310-32037-1bu6179-libre.pdf).
- Filip Křikava, Heather Miller, and Jan Vitek. Scala implicits are everywhere: A large-scale study of the use of scala implicits in the wild. *Proceedings of the ACM on Programming Languages*, 3 (OOPSLA):1–28, 2019. URL <https://doi.org/10.1145/3360589>.
- Bruno CdS Oliveira, Adriaan Moors, and Martin Odersky. Type classes as objects and implicits. *ACM Sigplan Notices*, 45(10):341–360, 2010. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=d30d65ca9ce7891352024a5c71ebe0ae8c41f7ac>.
- Dominique Devriese and Frank Piessens. On the bright side of type classes: instance arguments in agda. *ACM SIGPLAN Notices*, 46(9):143–155, 2011. URL <https://archive.alvb.in/msc/thesis/reading/typeclasses-agda-Devriese.pdf>.
- Conor McBride. Faking it simulating dependent types in haskell. *Journal of functional programming*, 12(4-5):375–392, 2002. URL <https://doi.org/10.1017/S0956796802004355>.
- Simon Peyton Jones, Andrew Tolmach, and Tony Hoare. Playing by the rules: rewriting as a practical optimisation technique in ghc. In *Haskell workshop*, volume 1, pages 203–233, 2001. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2001/09/rules.pdf>.
- Sandy Maguire. *Algebra-Driven Design, Elegant Solutions from Simple Building Blocks*. b. URL <https://leanpub.com/algebra-driven-design/>.
- Li-yao Xia. *Defunctionalization*. URL <https://poisson.chat/aquarium/defunctionalization.pdf>.
- John C Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM annual conference-Volume 2*, pages 717–740, 1972. URL <https://doi.org/10.1145/800194.805852>.
- John C Reynolds. Definitional interpreters revisited. *Higher-Order and Symbolic Computation*, 11: 355–361, 1998. URL <https://doi.org/10.1023/A:1010075320153>.
- Jeremy Yallop and Leo White. Lightweight higher-kinded polymorphism. In *International Symposium on Functional and Logic Programming*, pages 119–135. Springer, 2014. URL <https://www.cl.cam.ac.uk/~jdy22/papers/lightweight-higher-kinded-polymorphism.pdf>.
- Manuel MT Chakravarty, Gabriele Keller, and Simon Peyton Jones. Associated type synonyms. *ACM SIGPLAN Notices*, 40(9):241–253, 2005a. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2005/01/at-syns.pdf>.

- Manuel MT Chakravarty, Gabriele Keller, Simon Peyton Jones, and Simon Marlow. Associated types with class. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 1–13, 2005b. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2005/01/assoc.pdf>.
- Tom Schrijvers, Simon Peyton Jones, Manuel Chakravarty, and Martin Sulzmann. Type checking with open type functions. In *Proceedings of the 13th ACM SIGPLAN international conference on Functional programming*, pages 51–62, 2008. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2008/01/icfp2008.pdf>.
- Richard A Eisenberg, Dimitrios Vytiniotis, Simon Peyton Jones, and Stephanie Weirich. Closed type families with overlapping equations. *ACM SIGPLAN Notices*, 49(1):671–683, 2014. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/pop1137-eisenberg.pdf>.
- Oleg Kiselyov, Simon Peyton Jones, and Chung-chieh Shan. Fun with type functions. *Reflections on the Work of CAR Hoare*, pages 301–331, 2010. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/typefun.pdf?from=https://research.microsoft.com/~simonpj/papers/assoc-types/fun-with-type-funs/typefun.pdf&type=exact>.
- Mark P Jones. Type classes with functional dependencies. In *European Symposium on Programming*, pages 230–244. Springer, 2000. URL [https://doi.org/10.1007/3-540-46425-5\\_15](https://doi.org/10.1007/3-540-46425-5_15).
- Nada Amin, Tiark Ropmf, and Martin Odersky. Foundations of path-dependent types. *Acm Sigplan Notices*, 49(10):233–249, 2014. URL [http://lampwww.epfl.ch/~amin/dot/fpdt\\_post.pdf](http://lampwww.epfl.ch/~amin/dot/fpdt_post.pdf).
- Jan Stolarek, Simon Peyton Jones, and Richard A Eisenberg. Injective type families for haskell. *ACM SIGPLAN Notices*, 50(12):118–128, 2015. URL [https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1070&context=compsci\\_pubs](https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1070&context=compsci_pubs).
- Richard A Eisenberg and Jan Stolarek. Promoting functions to type families in haskell. *ACM SIGPLAN Notices*, 49(12):95–106, 2014. URL [https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1000&context=compsci\\_pubs](https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1000&context=compsci_pubs).
- Csongor Kiss, Tony Field, Susan Eisenbach, and Simon Peyton Jones. Higher-order type-level programming in haskell. *Proceedings of the ACM on Programming Languages*, 3(ICFP):1–26, 2019. URL <https://dl.acm.org/doi/pdf/10.1145/3341706>.
- Dominic Orchard and Tom Schrijvers. Haskell type constraints unleashed. In *International Symposium on Functional and Logic Programming*, pages 56–71. Springer, 2010. URL <https://kar.kent.ac.uk/57498/1/constraint-families.pdf>.

- Edsko de Vries and Andres Löh. True sums of products. In *Proceedings of the 10th ACM SIGPLAN workshop on Generic programming*, pages 83–94, 2014. URL <https://doi.org/10.1145/2633628.2633634>.
- Gert-Jan Bottu, Georgios Karachalias, Tom Schrijvers, Bruno C d S Oliveira, and Philip Wadler. Quantified class constraints. *ACM SIGPLAN Notices*, 52(10):148–161, 2017. URL <https://www.pure.ed.ac.uk/ws/portalfiles/portal/42495988/quantcc.pdf>.
- Martin Sulzmann, Manuel MT Chakravarty, Simon Peyton Jones, and Kevin Donnelly. System f with type equality coercions. In *Proceedings of the 2007 ACM SIGPLAN international workshop on Types in languages design and implementation*, pages 53–66, 2007b. URL <https://doi.org/10.1145/1190315.1190324>.
- Dimitrios Vytiniotis, Simon Peyton Jones, Tom Schrijvers, and Martin Sulzmann. Outsidein (x) modular type inference with local assumptions. *Journal of functional programming*, 21(4-5): 333–412, 2011. URL <https://doi.org/10.1017/S0956796811000098>.
- Joachim Breitner, Richard A Eisenberg, Simon Peyton Jones, and Stephanie Weirich. Safe zero-cost coercions for haskell. In *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*, pages 189–202, 2014. URL [https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1013&context=compsci\\_pubs](https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1013&context=compsci_pubs).
- Simon Peyton Jones, Stephanie Weirich, Richard A Eisenberg, and Dimitrios Vytiniotis. A reflection on types. In *A List of Successes That Can Change the World: Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday*, pages 292–317. Springer, 2016. URL [https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1002&context=compsci\\_pubs](https://repository.brynmawr.edu/cgi/viewcontent.cgi?article=1002&context=compsci_pubs).
- Simon Marlow, Simon Peyton Jones, Andrew Moran, and John Reppy. Asynchronous exceptions in haskell. In *Proceedings of the ACM SIGPLAN 2001 conference on Programming language design and implementation*, pages 274–285, 2001. URL <https://classes.cs.uchicago.edu/archive/2007/spring/32102-1/papers/p274-marlow.pdf>.
- Simon Marlow. An extensible dynamically-typed hierarchy of exceptions. In *Proceedings of the 2006 ACM SIGPLAN Workshop on Haskell*, pages 96–106, 2006. URL [https://archive.alvb.in/msc/03\\_infoafp/papers/2012-12-18\\_HoorCollege\\_MarlowExtensibleExceptions\\_dk.pdf](https://archive.alvb.in/msc/03_infoafp/papers/2012-12-18_HoorCollege_MarlowExtensibleExceptions_dk.pdf).
- Simon Peyton Jones. Tackling the awkward squad: monadic input/output, concurrency, exceptions, and foreign-language calls in haskell. *NATO SCIENCE SERIES SUB SERIES III COMPUTER AND SYSTEMS SCIENCES*, 180:47–96, 2001. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=2e6c9d76f9cb690dc18019fc894ba9572a8c2812>.
- Simon Marlow. Parallel and concurrent programming in haskell. In *Central European Functional Programming School*, pages 339–401. Springer, 2011.

- URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=bd127ea1952996864c1542e1453973a78973ad5c>.
- Ralf Hinze, Jennifer Hackett, and Daniel WH James. Functional pearl: F for functor. 2012. URL [www.cs.ox.ac.uk/people/daniel.james/functor/functor.pdf](http://www.cs.ox.ac.uk/people/daniel.james/functor/functor.pdf).
- Ralf Hinze and Daniel WH James. Reason isomorphically! In *Proceedings of the 6th ACM SIGPLAN workshop on Generic programming*, pages 85–96, 2010. URL <http://www.cs.ox.ac.uk/people/ralf.hinze/publications/WGP10.pdf>.
- G rard Huet. The zipper. *Journal of functional programming*, 7(5):549–554, 1997. URL <https://doi.org/10.1017/S0956796897002864>.
- Conor McBride. The derivative of a regular type is its type of one-hole contexts. *Unpublished manuscript*, pages 74–88, 2001. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=7de4f6fddb11254d1fd5f8adfd67b6e0c9439eaa>.
- Michael Abbott, Thorsten Altenkirch, Neil Ghani, and Conor McBride. Derivatives of containers. In *International Conference on Typed Lambda Calculi and Applications*, pages 16–30. Springer, 2003. URL <http://www.strictlypositive.org/derivcont.pdf>.
- Erik Meijer, Maarten Fokkinga, and Ross Paterson. Functional programming with bananas, lenses, envelopes and barbed wire. In *Conference on functional programming languages and computer architecture*, pages 124–144. Springer, 1991. URL <https://ris.utwente.nl/ws/portalfiles/portal/6142049/meijer91functional.pdf>.
- Erik Meijer and Graham Hutton. Bananas in space: Extending fold and unfold to exponential types. In *Proceedings of the seventh international conference on Functional programming languages and computer architecture*, pages 324–333, 1995. URL <https://dl.acm.org/doi/pdf/10.1145/224164.224225>.
- Olivier Danvy and Kevin Millikin. Refunctionalization at work. *Science of Computer Programming*, 74(8):534–549, 2009. ISSN 0167-6423. doi: <https://doi.org/10.1016/j.scico.2007.10.007>. URL <https://www.sciencedirect.com/science/article/pii/S0167642309000227>. Special Issue on Mathematics of Program Construction (MPC 2006).
- Philip Wadler. Deforestation: Transforming programs to eliminate trees. In *European Symposium on Programming*, pages 344–358. Springer, 1988. URL [https://link.springer.com/content/pdf/10.1007/3-540-19027-9\\_23.pdf](https://link.springer.com/content/pdf/10.1007/3-540-19027-9_23.pdf).
- СА Романенко. ВЫЯВЛЕНИЕ И ДОКАЗАТЕЛЬСТВО СВОЙСТВ ФУНКЦИОНАЛЬНЫХ ПРОГРАММ МЕТОДАМИ СУПЕРКОМПИАЦИИ. URL <https://www.keldysh.ru/council/1/klyuchnikov-diss.pdf>.

- Andrew Gill, John Launchbury, and Simon L Peyton Jones. A short cut to deforestation. In *Proceedings of the conference on Functional programming languages and computer architecture*, pages 223–232, 1993. URL <https://dl.acm.org/doi/pdf/10.1145/165180.165214>.
- Jeremy Gibbons. Unfolding abstract datatypes. In *International Conference on Mathematics of Program Construction*, pages 110–133. Springer, 2008. URL <https://ora.ox.ac.uk/objects/uuid:b51d2af6-25de-41ae-b3e7-9ac997de87b3/files/m7c19077c7bfba562a6cc9ceb3db62641>.
- William R Cook. On understanding data abstraction, revisited. In *Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications*, pages 557–572, 2009. URL [https://www.phaazon.net/media/uploads/on\\_understanding\\_data\\_abstraction.pdf](https://www.phaazon.net/media/uploads/on_understanding_data_abstraction.pdf).
- Duncan Coutts, Roman Leshchinskiy, and Don Stewart. Stream fusion: From lists to streams to nothing at all. *ACM SIGPLAN Notices*, 42(9):315–326, 2007. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=a4fad0d182605fcd155bebe3d620b7ffa0456968>.
- Oleg Kiselyov, Aggelos Biboudis, Nick Palladinos, and Yannis Smaragdakis. Stream fusion, to completeness. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, pages 285–299, 2017. URL <https://arxiv.org/pdf/1612.06668>.
- Paul Downen, Zachary Sullivan, Zena M Ariola, and Simon Peyton Jones. Codata in action. In *European Symposium on Programming*, pages 119–146. Springer International Publishing Cham, 2019. URL <https://library.oapen.org/bitstream/handle/20.500.12657/23330/1006825.pdf?sequence=1#page=132>.
- Jon Louis Bentley. Little languages. *Commun. ACM*, 29(8):711–721, 1986. URL <https://doi.org/10.1145/6424.315691>.
- Jeremy Gibbons and Nicolas Wu. Folding domain-specific languages: deep and shallow embeddings (functional pearl). In *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*, pages 339–347, 2014. URL <https://ora.ox.ac.uk/objects/uuid:0547e439-48d5-4749-a5eb-811e65cfaaee/files/mf31b32c6dcdbd81dd6a8ef43166a8cc91>.
- Jeremy Gibbons. Functional programming for domain-specific languages. In *Central European Functional Programming School*, pages 1–28. Springer, 2013. URL <https://ora.ox.ac.uk/objects/uuid:489909da-8818-4d6e-a4fd-0107b589ee76/files/m00be42b36cabbc6d11c8748fac8c7c71>.
- Benjamin C Pierce, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, and Brent Yorgey. Software foundations. Webpage: <http://www.cis.upenn.edu/bcpierce/sf/current/index.html>, 16, 2010. URL <https://idris-hackers.github.io/software-foundations/pdf/sf-idris-2018.pdf>.

- Graham Hutton. Fold and unfold for program semantics. *ACM SIGPLAN Notices*, 34(1):280–288, 1998. URL <https://dl.acm.org/doi/abs/10.1145/291251.289457>.
- Robert Nystrom. *Crafting Interpreters*. URL <https://craftinginterpreters.com/contents.html>.
- Dom Hutchinson. Language engineering - notes. Course COMS22201 notes. URL <https://github.com/dajhutchinson/Language-Engineering>.
- Arthur Charguéraud. The locally nameless representation. *Journal of automated reasoning*, 49: 363–408, 2012. URL <https://www.chargueraud.org/research/2009/ln/main.pdf>.
- Stephen Chong. *CS153: Compilers Lecture 12: Closures and Environments*. URL <https://groups.seas.harvard.edu/courses/cs153/2018fa/lectures/Lec12-Functions.pdf>.
- Oleg Kiselyov. Typed tagless final interpreters. In *Generic and indexed programming: International spring school, sSGIP 2010, oxford, uK, march 22-26, 2010, revised lectures*, pages 130–174. Springer, 2012a. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=f0b6a04331821a294bad4d08975637b4f363ec5a#page=136>.
- Frank Pfenning and Conal Elliott. Higher-order abstract syntax. *ACM sigplan notices*, 23(7): 199–208, 1988. URL <https://doi.org/10.1145/960116.54010>.
- Jeff Epstein, Andrew P Black, and Simon Peyton-Jones. Towards haskell in the cloud. In *Proceedings of the 4th ACM symposium on Haskell*, pages 118–129, 2011. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/remote.pdf?from=https://research.microsoft.com/en-us/um/people/simonpj/papers/parallel/remote.pdf&type=exact>.
- Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages. In *Programming Languages and Systems: 5th Asian Symposium, APLAS 2007, Singapore, November 29-December 1, 2007. Proceedings 5*, pages 222–238. Springer, 2007. URL <https://www.cambridge.org/core/journals/journal-of-functional-programming/article/finally-tagless-partially-evaluated-tagless-staged-interpreters-for-simpler-typed-languages/7B2DC44A2127EBBA71ADE63809D9425F>.
- Eugenio Moggi. *Computational lambda-calculus and monads*. University of Edinburgh, Department of Computer Science, Laboratory for ..., 1988. URL <http://www.lfcs.inf.ed.ac.uk/reports/88/ECS-LFCS-88-66/ECS-LFCS-88-66.pdf>.
- Philip Wadler. Comprehending monads. In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming*, pages 61–78, 1990. URL <https://dl.acm.org/doi/pdf/10.1145/91556.91592>.

- Philip Wadler. The essence of functional programming. In *Proceedings of the 19th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 1–14, 1992. URL <https://dl.acm.org/doi/pdf/10.1145/143165.143169>.
- Conor McBride and Ross Paterson. Applicative programming with effects. *Journal of functional programming*, 18(1):1–13, 2008. URL <https://doi.org/10.1017%2FS0956796807006326>.
- John C Reynolds. The discoveries of continuations. *Lisp and symbolic computation*, 6:233–247, 1993. URL [https://ics.uci.edu/~jajones/INF102-S18/readings/08\\_histcont.pdf](https://ics.uci.edu/~jajones/INF102-S18/readings/08_histcont.pdf).
- Peter J Landin. Histories of discoveries of continuations: Belles-lettres with equivocal tenses. In *Proceedings of the Second ACM SIGPLAN Workshop on Continuations (CW'97), Technical report BRICS NS-96-13, University of Aarhus*, page 1. Citeseer, 1997. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=14c609276166517ed1afb56e6b84a41ba157030b>.
- Steven E Ganz, Daniel P Friedman, and Mitchell Wand. Trampolined style. In *Proceedings of the fourth ACM SIGPLAN international conference on Functional programming*, pages 18–27, 1999. URL <https://doi.org/10.1145/317636.317779>.
- RO Bjarnarson. Stackless scala with free monads. *Scala Days*, 2012. URL [https://days2012.scala-lang.org/sites/days2012/files/bjarnason\\_trampoline.pdf](https://days2012.scala-lang.org/sites/days2012/files/bjarnason_trampoline.pdf).
- Conor McBride. Clowns to the left of me, jokers to the right (pearl) dissecting data structures. In *Proceedings of the 35th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 287–295, 2008. URL [https://web.archive.org/web/20170705104305id\\_/http://www.cis.upenn.edu/~cis39903/static/clowns.pdf](https://web.archive.org/web/20170705104305id_/http://www.cis.upenn.edu/~cis39903/static/clowns.pdf).
- Daniel Hillerström. Foundations for programming and implementing effect handlers. 2022. URL <http://dx.doi.org/10.7488/era/2122>.
- Serkan Muhcu, Philipp Schuster, Michel Steuwer, and Jonathan Immanuel Brachthäuser. Multiple resumptions and local mutable state, directly. *Proceedings of the ACM on Programming Languages*, 9(ICFP):704–733, 2025. URL <https://doi.org/10.1145/3747529>.
- Olivier Danvy. Defunctionalized interpreters for programming languages. *ACM Sigplan Notices*, 43(9):131–142, 2008. URL <http://jfla.inria.fr/2014/danvy-ICFP08.pdf>.
- Jeremy Gibbons. Continuation-passing style, defunctionalization, accumulations, and associativity. *arXiv preprint arXiv:2111.10413*, 2021. URL <https://doi.org/10.48550/arXiv.2111.10413>.
- Olivier Danvy. An analytical approach to program as data objects, 2006. URL [https://www.cs.tufts.edu/comp/150FP/archive/olivier-danvy/d\\_scientia.pdf](https://www.cs.tufts.edu/comp/150FP/archive/olivier-danvy/d_scientia.pdf).

- Atze van der Ploeg and Oleg Kiselyov. Reflection without remorse: revealing a hidden sequence to speed up monadic reflection. In *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell*, pages 133–144, 2014. URL <https://doi.org/10.1145/2633357.2633360>.
- Pierre-Louis Curien and Hugo Herbelin. The duality of computation. *ACM sigplan notices*, 35(9): 233–243, 2000. URL <https://doi.org/10.1145/357766.351262>.
- R Kent Dyvbig, Simon Peyton Jones, and Amr Sabry. A monadic framework for delimited continuations. *Journal of functional programming*, 17(6):687–730, 2007. URL <https://doi.org/10.1017/S0956796807006259>.
- Andrzej Filinski. Representing monads. In *Proceedings of the 21st ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 446–457, 1994. URL <https://doi.org/10.1145/174675.178047>.
- Ana Lúcia De Moura and Roberto Ierusalimsky. Revisiting coroutines. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 31(2):1–31, 2009. URL <https://doi.org/10.1145/1462166.1462167>.
- Roman Elizarov, Mikhail Belyaev, Marat Akhin, and Ilmir Usmanov. Kotlin coroutines: design and implementation. In *Proceedings of the 2021 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, pages 68–84, 2021. URL [https://www.researchgate.net/profile/Mikhail-Belyaev-3/publication/355361443\\_Kotlin\\_coroutines\\_design\\_and\\_implementation/links/624c63c34f88c3119ce12a49/Kotlin-coroutines-design-and-implementation.pdf](https://www.researchgate.net/profile/Mikhail-Belyaev-3/publication/355361443_Kotlin_coroutines_design_and_implementation/links/624c63c34f88c3119ce12a49/Kotlin-coroutines-design-and-implementation.pdf).
- Anders Møller and Michael I Schwartzbach. Static program analysis. *Notes*. Feb, 2012. URL <https://users-cs.au.dk/amoeller/spa/spa.pdf>.
- Yannick Forster, Ohad Kammar, Sam Lindley, and Matija Pretnar. On the expressive power of user-defined effects: Effect handlers, monadic reflection, delimited control. *Proceedings of the ACM on Programming Languages*, 1(ICFP):1–29, 2017. URL <https://doi.org/10.1145/3110257>.
- Ningning Xie, Jonathan Immanuel Brachthäuser, Daniel Hillerström, Philipp Schuster, and Daan Leijen. Effect handlers, evidently. *Proceedings of the ACM on Programming Languages*, 4(ICFP):1–29, 2020. URL <https://doi.org/10.1145/3408981>.
- Oleg Kiselyov, Amr Sabry, and Cameron Swords. Extensible effects: an alternative to monad transformers. *ACM SIGPLAN Notices*, 48(12):59–70, 2013. URL <https://doi.org/10.1145/2578854.2503791>.
- Andrej Bauer. What is algebraic about algebraic effects and handlers? *arXiv preprint arXiv:1807.05923*, 2018. URL <https://doi.org/10.48550/arXiv.1807.05923>.

- Daniel Hillerström and Sam Lindley. Shallow effect handlers. In *Asian Symposium on Programming Languages and Systems*, pages 415–435. Springer, 2018. URL [https://www.research.ed.ac.uk/files/76099718/shallow\\_effect\\_handlers.pdf](https://www.research.ed.ac.uk/files/76099718/shallow_effect_handlers.pdf).
- Wouter Swierstra. Data types à la carte. *Journal of functional programming*, 18(4):423–436, 2008. URL <https://www.cs.tufts.edu/~nr/cs257/archive/wouter-swierstra/DataTypesALaCarte.pdf>.
- Oleg Kiselyov and Hiromi Ishii. Freer monads, more extensible effects. *ACM SIGPLAN Notices*, 50(12):94–105, 2015. URL <https://doi.org/10.1145/2887747.2804319>.
- Oleg Kiselyov. Iteratees. In *FLOPS’12: Proceedings of the 11th international conference on Functional and Logic Programming*, 2012b. URL <https://okmij.org/ftp/Haskell/Iteratee/describe.pdf>.
- Daan Leijen. Algebraic effect handlers with resources and deep finalization. Technical report, Tech. Rep. MSR-TR-2018-10, Microsoft Research (April 2018), 2018. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2018/04/resource-v1.pdf>.
- KC Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. Retrofitting effect handlers onto ocaml. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 206–221, 2021. URL <https://doi.org/10.1145/3453483.3454039>.
- Luna Phipps-Costin, Andreas Rossberg, Arjun Guha, Daan Leijen, Daniel Hillerström, KC Sivaramakrishnan, Matija Pretnar, and Sam Lindley. Continuing webassembly with effect handlers. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA2):460–485, 2023. URL <https://doi.org/10.1145/3622814>.
- Nicolas Wu and Tom Schrijvers. Fusion for free: Efficient algebraic effect handlers. In *International Conference on Mathematics of Program Construction*, pages 302–322. Springer, 2015. URL <https://lirias.kuleuven.be/retrieve/322544>.
- Sheng Liang, Paul Hudak, and Mark Jones. Monad transformers and modular interpreters. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 333–343, 1995. URL <https://dl.acm.org/doi/pdf/10.1145/199448.199528>.
- Mark P Jones. Functional programming with overloading and higher-order polymorphism. In *International School on Advanced Functional Programming*, pages 97–136. Springer, 1995. URL <http://web.cecs.pdx.edu/mpj/pubs/springschool95.pdf>.
- Tom Schrijvers, Maciej Piróg, Nicolas Wu, and Mauro Jaskelioff. Monad transformers and modular algebraic effects: what binds them together. In *Proceedings of the 12th ACM SIGPLAN International Symposium on Haskell*, pages 98–113, 2019. URL <https://doi.org/10.1145/3331545.3342595>.

- Gordon Plotkin and John Power. Notions of computation determine monads. In *International Conference on Foundations of Software Science and Computation Structures*, pages 342–356. Springer, 2002. URL [https://era.ed.ac.uk/bitstream/handle/1842/196/Comp\\_Eff\\_Monads.pdf?sequence=1](https://era.ed.ac.uk/bitstream/handle/1842/196/Comp_Eff_Monads.pdf?sequence=1).
- Gordon D Plotkin and Matija Pretnar. Handling algebraic effects. *Logical methods in computer science*, 9, 2013. URL [https://doi.org/10.2168/LMCS-9\(4:23\)2013](https://doi.org/10.2168/LMCS-9(4:23)2013).
- Nicolas Wu, Tom Schrijvers, and Ralf Hinze. Effect handlers in scope. In *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell*, pages 1–12, 2014. URL <https://doi.org/10.1145/2633357.2633358>.
- Zhixuan Yang, Marco Paviotti, Nicolas Wu, Birthe van den Berg, and Tom Schrijvers. Structured handling of scoped effects. In *European Symposium on Programming*, pages 462–491. Springer International Publishing Cham, 2022. URL <https://library.oapen.org/bitstream/handle/20.500.12657/54028/1/978-3-030-99336-8.pdf#page=476>.
- Cas van der Rest, Jaro Reinders, and Casper Bach Poulsen. Handling higher-order effects. *arXiv preprint arXiv:2203.03288*, 2022. URL <https://arxiv.org/pdf/2203.03288>.
- Yizhou Zhang, Guido Salvaneschi, and Andrew C Myers. Handling bidirectional control flow. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–30, 2020. URL <https://doi.org/10.1145/3428207>.
- James Chapman, Pierre-Évariste Dagand, Conor McBride, and Peter Morris. The gentle art of levitation. *ACM Sigplan Notices*, 45(9):3–14, 2010. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=55ab751e0af3808c741585df0210ca1785c235d6>.
- Yanhong A Liu. Incremental computation: What is the essence? *arXiv preprint arXiv:2312.07946*, 2023. URL <https://arxiv.org/pdf/2312.07946>.
- Chris Okasaki. *Purely functional data structures*. Cambridge University Press, 1999. URL <https://www.cs.tufts.edu/~nr/cs257/archive/chris-okasaki/dissertation.pdf>.
- Aaron Bohannon, Benjamin C Pierce, and Jeffrey A Vaughan. Relational lenses: a language for updatable views. In *Proceedings of the twenty-fifth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 338–347, 2006. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=8ee979ec0a1d8288a07a813c7bde91e4a2ae3187>.
- J Nathan Foster, Alexandre Pilkiewicz, and Benjamin C Pierce. Quotient lenses. *ACM Sigplan Notices*, 43(9):383–396, 2008. URL <https://repository.upenn.edu/server/api/core/bitstreams/31dcfba8-8b4f-4143-bd2c-9443dc19c499/content>.